

AI at the Wheel, Engineers in Control
AI-Orchestrated Development for Software-Defined Vehicles

Your Name

2026

Contents

Preface	1
Why this book	1
Who this book is for	1
How to read it	1
The companion code	2
A note on sources and a fast-moving field	2
Chapter 1: The Software-Defined Vehicle and Its Development Problem	3
A car that improves while it's parked	3
1.1 From fixed-function ECUs to software-defined platforms	3
1.2 Modern E/E architecture: domain and zonal compute	4
1.3 Service-oriented vehicle software	4
1.4 The SDV lifecycle: features and value over the vehicle's life	4
1.5 The SDV development bottleneck: complexity, safety, and pace	5
1.6 Why SDV teams need a new development methodology	5
Key takeaways	6
What's next	6
Chapter 2: AWS AI-DLC — AI as Development Orchestrator	7
From assistant to orchestrator	7
2.1 What AI-DLC is, and how it differs from “AI coding assistants”	7
2.2 The three phases: Inception, Construction, Operations	8
2.3 Bolts: replacing sprints as the unit of delivery	8
2.4 Mob Elaboration and Mob Construction: human-in-the-loop by design	8
2.5 Documentation-first and the audit trail	9
2.6 The open-source adaptive workflows	9
2.7 What AI-DLC asks of the team	10
Key takeaways	10
What's next	10
Chapter 3: Why AI-DLC Fits the SDV	11
3.1 Mapping AI-DLC phases onto the SDV lifecycle	11
AI-DLC Phases → Software Defined Vehicle Lifecycle	11
3.2 AI-DLC and the automotive V-model: where they fit, and where they don't	12
3.3 Where AI orchestrates and where engineers must decide	13
3.4 Governance, traceability, and the automotive compliance fit	15
The compliance challenge in SDVs	15
Regulatory landscape	16
Traceability requirements by domain	17
How AI-DLC supports compliance in SDVs	17

The OTA governance challenge	17
Key takeaways	18
What's next	18
Chapter 4: Deciding Where AI-DLC Fits — Value, Risk, and Maturity	20
4.1 A value, risk, and feasibility framework for AI-DLC in SDV	20
Framework structure	20
Pillar 1 — Value (primary weight)	21
Pillar 2 — Risk (secondary weight)	21
Pillar 3 — Feasibility (secondary weight)	21
Scoring worksheet	22
SDV workload positioning map	22
Governance and review cadence	23
Customizing the framework	23
4.2 What is realistic today, and what is not	24
At a glance	24
Realistic today	24
Emerging but not yet reliable	24
Not realistic today	25
The maturity curve	26
Where to start	26
Claims you can defend — and overpromises to avoid	26
4.3 Controlling the cost of AI-DLC: measure, cap, and alert	26
Key takeaways	28
What's next	28
Chapter 5: Capturing SDV Intent and Requirements	29
5.1 Intent bootstrap for vehicle features	29
The running example: EV charge scheduling	29
Seeding the intent	29
Structured questions tuned to vehicle context	30
The captured intent artifact	30
5.2 Mob Elaboration with cross-functional automotive teams	31
Who is in the mob	31
Surfacing assumptions early	31
The questionnaire the AI writes	32
5.3 Structured requirements for safety- and timing-critical functions	33
Anatomy of an SDV requirement	33
Example requirements for EV charge scheduling	33
Capturing SOTIF and edge-case intent	35
5.4 Removing ambiguity before design (the Inception goal)	35
5.5 Traceability from intent to requirement	35
Key takeaways	36
What's next	36
Chapter 6: User Stories and Functional Design for Vehicle Software	37
6.1 From requirements to user stories for in-vehicle and fleet features	37
6.2 Technology-agnostic functional design	39
6.3 Decomposing vehicle features into units of work	40
6.4 A worked thread: one story from altitude to units	41
6.5 Anti-patterns in stories and functional design	42
6.6 Approval gates: what the team signs off in Inception	42

Key takeaways	43
What's next	43
Chapter 7: Application and Infrastructure Design via Mob Construction	44
7.1 Architecture design for SDV: in-vehicle, edge, and cloud	44
7.2 Designing within AUTOSAR Adaptive and service-oriented middleware	45
7.3 Infrastructure design: cloud backend, OTA, data pipelines	47
7.4 Mob Construction: AI proposes, the team decides	48
7.5 A worked design pass: from units to approved architecture	49
7.6 Design anti-patterns to avoid	49
7.7 How the design maps to the code	50
Key takeaways	50
What's next	50
Chapter 8: Code Generation for SDV	51
8.1 Deterministic code generation with verification	51
8.2 Generating embedded, middleware, and backend code	51
8.3 Working with C/C++, config-heavy toolchains, and existing code	54
8.4 Unit and test generation	55
8.5 Build, test, and the verification loop	55
8.6 Traceability that survives into the code	56
Key takeaways	56
What's next	56
Chapter 9: Non-Functional Requirements as First-Class Citizens	57
9.1 NFR assessment for SDV: safety, real-time, reliability, security	57
9.2 NFR design: turning constraints into logical components	58
9.3 Functional safety and timing as explicit AI-DLC inputs	59
9.4 Where GenAI must not be in the loop	60
9.5 The boundary as an enabler, not just a restriction	60
9.6 NFR anti-patterns in SDV	62
Key takeaways	62
What's next	63
Chapter 10: Operations — Deploy, Monitor, Document	64
10.1 OTA deployment in the Operations phase	64
10.2 Monitoring, telemetry, and feedback into the next Bolt	66
10.3 Documentation-first operations and accumulated context	66
10.4 Incident response and the difference between a fix and a rollback	67
10.5 Closing the loop back to Inception	67
Key takeaways	68
What's next	68
Chapter 11: Understanding Productivity Gains — AI-DLC and DORA Metrics	69
11.1 Why measure the gain, not just feel it	69
11.2 The four DORA metrics	69
11.3 What AI-DLC moves — and what it doesn't	70
11.4 Measuring without gaming: guardrail metrics and SDV caveats	71
11.5 Baselining honestly: attributing the gain	72
Key takeaways	72
What's next	73

Chapter 12: Safety, Validation, and Compliance Inside AI-DLC	74
12.1 The hard problem: trusting AI-orchestrated, non-deterministic work	74
12.2 ISO 26262 and ISO 21448 (SOTIF) within AI-DLC gates	74
12.3 UNECE regulation, type approval, and the AI-DLC audit trail	76
12.4 Using documentation-first artifacts as evidence	76
12.5 Validation becomes the constraint — and must be resourced	78
12.6 Safety and compliance anti-patterns	79
Key takeaways	79
What's next	79
Chapter 13: Security, Privacy, and Responsible AI in the Workflow	80
13.1 Securing the AI-DLC pipeline and generated artifacts	80
13.2 IP, licensing, and provenance of generated code	81
13.3 In-cabin and fleet data privacy	82
13.4 Governance, roles, and auditability	82
13.5 Responsible AI as a property of the workflow, not a policy on top	83
Key takeaways	83
What's next	84
Chapter 14: Brownfield — Evolving Existing Vehicle Software with AI-DLC	85
No such thing as a blank workspace	85
14.1 Brownfield is the SDV default	85
14.2 Workspace detection and the reverse-engineering stage	86
14.3 Grounding Inception in the existing system	87
14.4 Evolving the code: modify in place, never duplicate	88
14.5 Brownfield governance and pitfalls	89
Key takeaways	91
What's next	91
Chapter 15: The SDV Operational Framework and How AI-DLC Embeds Into It	92
The standing machinery	92
15.1 What the SDV operational framework is	92
15.2 The pillars of the framework	93
15.3 Where AI-DLC embeds: the Operations phase as the per-change engine	93
15.4 The audit trail as the framework's evidence spine	95
15.5 Human gates, roles, and the safety boundary in operations	96
15.6 Continuous improvement: closing the loop to the next Bolt	97
Key takeaways	97
What's next	97
Chapter 16: Selecting and Evaluating LLM Models for SDV Workloads	98
The wrong default	98
16.1 Why model selection is a first-class decision	98
16.2 A workload taxonomy for AI-DLC	99
16.3 The selection criteria	99
16.4 Building an evaluation harness	101
16.5 Matching models to the maturity gradient and the safety boundary	101
16.6 Governing model choice	101
Key takeaways	103
What's next	103

Chapter 17: A Worked Example and an Adoption Roadmap	104
17.1 End-to-end Bolt: building an SDV feature with AI-DLC	104
17.2 Lessons, anti-patterns, and a maturity model	105
17.3 Rolling AI-DLC out across an SDV organization	105
17.4 The road ahead: agents, smaller models, and evolving regulation	106
17.5 A checklist for your first Bolt	107
Key takeaways	107
Glossary	109
AI-DLC and SDV concepts	109
Abbreviations and acronyms	109

Preface

Two shifts are reshaping how vehicles get built. The car is becoming software-defined, evolving through updates long after it leaves the factory, and at the same time a new way of building software is emerging in which artificial intelligence (AI) acts not as a coding assistant but as the orchestrator of the development lifecycle. Amazon Web Services (AWS) calls that second shift the AI-Driven Development Life Cycle, or AI-DLC. This book is about deliberately bringing the two together: infusing AI-DLC into the engineering of software-defined vehicles (SDVs).

Why this book

Most writing about AI in automotive stops at “use a copilot to write code faster.” AI-DLC asks a more ambitious question: what if AI led the workflow, generating requirements, proposing architecture, and writing code, while humans kept authority over every decision that matters? That model is powerful, but the automotive world adds constraints that general software does not: functional safety, hard real-time behavior, type approval, and an unforgiving bar for evidence and traceability. The interesting work is in the fit, and that fit is what this book is about.

Who this book is for

This book is written primarily for automotive software engineers, architects, and engineering leads who are adopting, or evaluating, AI-assisted development on SDV platforms. It is also for program, quality, and safety leads who need to understand how AI-DLC’s phases, gates, and documentation-first model line up with automotive compliance.

You do not need prior AI-DLC experience; Chapter 2 covers the methodology in full. You should be comfortable with software engineering and the general shape of how vehicle software is built.

How to read it

Part I builds the foundations: the SDV and its development problem (Chapter 1), the AI-DLC methodology itself (Chapter 2), why the two fit (Chapter 3), and how to decide which SDV workloads to run through AI-DLC and how aggressively (Chapter 4). Parts II, III, and IV then walk AI-DLC’s three phases as applied to SDV: Inception (what and why), Construction (how), and Operations (deploy, monitor, document), with safety, security, and governance treated as first-class concerns throughout.

For clarity, that walkthrough follows a *greenfield* path (a single feature, EV charge scheduling, built as if from a blank workspace) so each phase can be seen in isolation. Real SDV work is rarely greenfield, though: new features are almost always added into an existing vehicle software

estate. Chapter 14 therefore turns to the *brownfield* case, showing how AI-DLC reverse-engineers existing code and evolves it in place without disturbing the certified, safety-critical parts. The book then closes with the operational framework that runs a fleet (Chapter 15) and how to select and evaluate the large language models that power the workflow (Chapter 16), and an end-to-end worked example with an organizational adoption roadmap (Chapter 17).

If you already know SDV architecture, skim Chapter 1 and start at Chapter 2. If you already know AI-DLC, skim Chapter 2 and start at Chapter 3.

The companion code

The book's running example, EV charge scheduling, is available as a complete, runnable companion repository at <https://github.com/your-org/genai-for-sdv-code> (placeholder; final URL to be confirmed before publication). It holds both halves this book describes: the AI-DLC documentation-first artifact trail under `aidlc-docs/`, named exactly as the chapters reference it, and the six units of work (U1 through U6) spanning cloud, in-vehicle, and OTA, with the safety boundary isolated and human-engineered. Every `aidlc-docs/...` path cited in the chapters resolves to a real file there. Later chapters point back to this note rather than repeating the link.

A note on sources and a fast-moving field

AI-DLC is an AWS methodology, and its open-source adaptive workflows are published under `awslabs/aidlc-workflows` (repository: <https://github.com/awslabs/aidlc-workflows>; placeholder, confirm before publication). The methodology is evolving quickly, and specific tools will date faster than the underlying ideas. Where possible this book emphasizes durable judgment, how to elaborate intent, where to place human decisions, how to validate safely, over any particular tool version. Generative AI can make mistakes; AI-DLC's whole design assumes human review, and so should you.

Chapter 1: The Software-Defined Vehicle and Its Development Problem

After this chapter you can describe what makes a vehicle “software-defined,” how modern vehicle architecture is organized, and why the way SDV software is built is straining under complexity, safety, and pace, setting up the case for a new methodology.

A car that improves while it’s parked

A decade ago, the capabilities of a car were essentially fixed the moment it left the factory. The features you bought were the features you kept. Today, a vehicle can gain new driver-assistance behavior, a better charging curve, or an entirely new interface overnight, while it sits in the owner’s driveway. That shift, from fixed-function hardware to a platform that evolves through software, is the essence of the software-defined vehicle.

It also creates a development problem. The promise of continuous improvement means the software never really stops changing, and it must change against an unforgiving backdrop of safety, real-time behavior, and regulation. This chapter establishes what an SDV is and why building its software the traditional way is buckling, which is the gap the rest of the book fills with AI-DLC.

1.1 From fixed-function ECUs to software-defined platforms

Traditional vehicles grew their electronics organically. Each new feature arrived as its own electronic control unit (ECU): a small dedicated computer with firmware written for one job. A premium car could end up with seventy to a hundred ECUs from dozens of suppliers, wired together over buses like CAN (Controller Area Network). The software was tightly bound to the hardware it shipped on and rarely changed after production.

That sprawl carries a real cost. Every added ECU is another processor to source, another supplier relationship to manage, more wiring harness to route, and more weight to carry; a premium vehicle’s harness can run to several kilometers of wire and rank among the heaviest and most expensive components in the car. More insidiously, each ECU is a software island: functions that must cooperate (say, regenerative braking coordinating with stability control) do so by passing signals across buses between separately developed, separately versioned firmware. Integrating dozens of such islands, each from a different vendor on a different release schedule, turns even a modest feature change into a cross-supplier negotiation. The complexity is not in any one ECU; it is in the seams between them.

That model does not scale to a world where customers expect their vehicle to improve like their phone, and where automakers want to add features and revenue after the sale. The software-defined vehicle inverts the old relationship: powerful, general-purpose compute runs software that can be updated, and hardware becomes a platform for that software rather than the other way around. The unit of value shifts from the part to the program running on it.

1.2 Modern E/E architecture: domain and zonal compute

To make this work, the electrical/electronic (E/E) architecture consolidates. Instead of scattering function across many small ECUs, the industry is moving to fewer, more powerful computers:

- **Domain architectures** group functions by purpose, for example a single domain controller for cockpit/infotainment, another for ADAS (advanced driver-assistance systems), another for the powertrain.
- **Zonal architectures** go further, placing controllers by physical location in the car and concentrating heavy computation in one or more high-performance central compute units.

Consider what consolidation does to a feature like adaptive cruise or cabin pre-conditioning. In the federated world it might touch five or six ECUs, each needing a coordinated firmware release. On a domain or zonal architecture the same logic runs as software on a central controller, calling other functions through defined interfaces and updated as a unit. The wiring shrinks, the compute grows, and, crucially for this book, the feature becomes a piece of *software* that can be specified, generated, tested, and shipped on its own cadence rather than a hardware change frozen at production.

The practical consequence: there is now real compute in the vehicle, and a clearer place to run demanding workloads. That matters later when AI-DLC produces software targeting these platforms.

1.3 Service-oriented vehicle software

With consolidated compute, vehicle software increasingly looks like modern distributed software. Functions are exposed as services that communicate over standardized middleware rather than as monolithic firmware bound to a wire. Standards such as AUTOSAR (AUTomotive Open System ARchitecture) Adaptive bring POSIX (Portable Operating System Interface)-based operating systems, service-oriented communication, and over-the-air (OTA) updatability to high-performance controllers, while the classic AUTOSAR platform still serves the deeply real-time, safety-critical edges.

This service orientation matters for our subject because it gives generated software defined interfaces to target, and it makes the system describable in the kind of structured artifacts AI-DLC produces and consumes.

1.4 The SDV lifecycle: features and value over the vehicle's life

The deepest change the SDV brings is not technical but temporal. A traditional vehicle's software was essentially complete at the start of production; the SDV's software is never complete. Its lifecycle stretches across the whole life of the car: develop a capability, deploy it over the air, operate and observe it in the field, then feed what you learn back into the next improvement. The same vehicle receives dozens of updates over the years it is owned, fixing defects, sharpening existing behavior, and adding features that did not exist when it was sold.

This reshapes the engineering problem in three ways. First, **the software must be designed to keep changing**: interfaces, update mechanisms, and safety arguments all have to accommodate versions that ship long after the vehicle does. Second, **the field becomes an input, not just a destination**: telemetry from the fleet reveals how features actually behave across climates, driving styles, and edge cases, and that evidence drives the next cycle. Third, **value and obligation both accrue over time**: an automaker can offer capabilities and revenue after the sale, but it also carries a lifelong duty to keep every deployed version safe, secure, and compliant. A vehicle that improves in the driveway is also a vehicle whose maker must prove, for years, exactly what software each car is running and why. That standing obligation is a theme the rest of the book returns to repeatedly.

1.5 The SDV development bottleneck: complexity, safety, and pace

Here is the tension at the heart of the book. The SDV demands three things at once that traditional development struggles to deliver together:

- **Complexity.** Tens of millions of lines of code, layered standards, multi-supplier integration, and tight hardware constraints.
- **Safety and compliance.** Functional safety (International Organization for Standardization standard ISO 26262), safety of the intended functionality (ISO 21448 / SOTIF), and regulatory type approval, all demanding rigorous traceability and evidence.
- **Pace.** Continuous, OTA-delivered improvement on a cadence closer to consumer software than to traditional automotive release cycles.

These three pull in different directions. The traditional automotive answer to complexity and safety is process: the V-model, stage gates, extensive documentation, and long validation campaigns that trade speed for rigor. That rigor is not optional, it is why cars are as safe as they are, but it was built for a world of infrequent, hardware-locked releases. Bolt a consumer-software release cadence onto a V-model-paced organization and something has to give: either the pace collapses back to annual model-year rhythms, or the rigor erodes under schedule pressure. Neither is acceptable when the product is a two-tonne machine that updates itself in a driveway.

Conventional methods, designed for human-driven, long-running processes, spend a large share of effort on planning, meetings, and lifecycle rituals rather than core engineering. Adding more people scales coordination cost, not throughput: the classic result that a late project grows later when staffed up. The demand is also relentless in a way it never was before: because the software is never finished, the backlog of defects to fix, features to add, and field learnings to act on never empties. The result is a structural bottleneck: the demand for safe, traceable change outpaces the ability of human-driven workflows to supply it, and no amount of the same kind of effort closes the gap.

1.6 Why SDV teams need a new development methodology

If the bottleneck is the human-driven workflow itself, then faster typing or a code-completion tool only nibbles at the edges. What is needed is a change to the workflow, one that lets AI take on the heavy lifting of requirements, design, and code while humans keep authority over the decisions that carry safety and regulatory weight, and that produces the traceable evidence automotive compliance demands as a by-product rather than an afterthought.

Note the shape of what is required. It is not simply “more automation”: the industry already

has generators, static analyzers, and continuous-integration pipelines. It is a change to *who does what*: letting AI carry the generative and coordinating load across the whole lifecycle, while engineers concentrate their scarce judgment on the safety, timing, and regulatory decisions that must remain human. And it must do this while producing traceable evidence continuously, because in a vehicle that evidence is not paperwork bolted on at the end; it is part of the product, demanded by the very regulations that let the car be sold and kept on the road.

That is exactly the proposition of AI-DLC. The next chapter introduces it on its own terms; from Chapter 3 onward we infuse it into the SDV.

Key takeaways

- A software-defined vehicle is one whose capabilities are primarily defined and evolved through updatable software on consolidated, general-purpose compute.
- E/E architectures are consolidating toward domain and zonal designs, creating real in-vehicle compute and clean service interfaces.
- The SDV lifecycle spans the whole life of the car: software is never finished, the field feeds the next improvement, and value and compliance obligations accrue for as long as the vehicle is on the road.
- SDV development must satisfy complexity, safety/compliance, and consumer-like pace simultaneously, a combination traditional, human-driven workflows handle poorly.
- The bottleneck is the workflow itself, motivating a methodology like AI-DLC.

What's next

Chapter 2 introduces AI-DLC on its own terms: its philosophy, its three phases, and the practices, Bolts and Mobs, that make it work.

Chapter 2: AWS AI-DLC — AI as Development Orchestrator

After this chapter you can explain what AI-DLC is, how it differs from an AI coding assistant, its three phases, and the core practices, Bolts, Mob Elaboration, and Mob Construction, that make it work.

From assistant to orchestrator

Most AI in software today plays a supporting role: it autocompletes a function, suggests a fix, answers a question. The developer still drives every step of the process. AI-DLC, introduced by AWS, proposes something more fundamental. It repositions AI from *coding assistant* to *development orchestrator*: the AI leads the workflow, generating requirements, proposing architecture, and producing code, while humans make the decisions at clearly defined points. The aim is to remove the non-core overhead that dominates traditional development, planning ceremonies, status meetings, lifecycle rituals, and let engineers spend their judgment where it counts.

This chapter explains AI-DLC on its own terms. We hold the automotive specifics until Chapter 3, but keep the SDV in the back of your mind: every property we describe here, structured artifacts, human approval gates, a documentation-first audit trail, will matter enormously when safety and compliance enter the picture.

2.1 What AI-DLC is, and how it differs from “AI coding assistants”

The distinction is easy to blur, so it is worth making sharp. An assistant lives *inside* a step: you are writing a function, and it proposes the next lines. You still decide what to build, break it into tasks, sequence the work, write the tests, and assemble the evidence. The assistant makes each keystroke cheaper but leaves the shape of the process, and all of its overhead, untouched. Studies of assistant adoption bear this out: developers accept a great deal of generated code yet report that the surrounding work (understanding requirements, reviewing, integrating, verifying) barely moves. The bottleneck was never typing speed.

An AI coding assistant accelerates tasks inside a process you still own end to end. AI-DLC changes the process. It defines a structured, largely deterministic sequence of phases in which AI does the generative heavy lifting and the team provides direction and approval. Two consequences follow immediately:

- **AI leads, humans decide.** The workflow is AI-led but human-centric: nothing advances without explicit human approval at defined points.
- **Documentation is the product, not a side effect.** Each phase emits artifacts, requirements, designs, decisions, that form a complete record. If the generated code were

deleted, the record of what was built and why would remain.

2.2 The three phases: Inception, Construction, Operations

AI-DLC organizes work into three phases, each with clear inputs, outputs, and human decision points.

- **Inception (What & Why).** AI breaks a project description down into structured requirements, user stories, and tasks. The explicit goal is to remove ambiguity before any design or code exists, so that later phases have as little uncertainty as possible. The team reviews and approves.
- **Construction (How).** AI proposes architecture and component design, then generates code and tests. The team decides among options and approves the result.
- **Operations.** AI handles deployment and produces documentation from the context accumulated across the earlier phases; the team signs off.

The phases are gated. You do not slide from requirements into code; you pass through a review where humans approve, modify, or send work back.

The ordering is deliberate, and it front-loads the cheapest place to fix a mistake. An ambiguity caught in Inception costs a sentence to correct; the same ambiguity discovered after code generation costs a rework of design, code, and tests; discovered in the field, it can cost a recall. By forcing clarity before design and design before code, AI-DLC spends effort where errors are cheapest to remove, the opposite of a rush-to-code workflow where requirements are implicit and get “discovered” during debugging. This is also why the gates are not bureaucratic friction: each one is the last inexpensive moment to change direction before the next phase makes change expensive.

2.3 Bolts: replacing sprints as the unit of delivery

AI-DLC replaces the sprint with a unit of delivery it calls a **Bolt**. Where a sprint is a fixed-length time box sized to human working rhythms, a Bolt is a focused pass through the phases for a coherent piece of work, sized to how AI and humans actually collaborate. A Bolt carries a feature or change from intent through requirements, design, code, and into operation, accumulating its artifacts as it goes. Delivery is a sequence of Bolts rather than a calendar of sprints.

The shift matters because a sprint is sized to people (two weeks because that is a comfortable human planning horizon), whereas a Bolt is sized to a *coherent change*. When AI does the generative work, the natural unit is no longer “what can the team finish in a fortnight” but “what is one self-contained thing worth carrying through the gates together.” A Bolt might complete in hours or span several days; what defines it is coherence and a clean set of accumulated artifacts, not a fixed duration. Because each Bolt leaves behind a complete record of intent, decisions, and verification, a program’s history becomes a sequence of traceable changes rather than a calendar of time boxes whose contents blur together. In an SDV, where every deployed change must be accountable for the life of the vehicle, that per-change record is exactly the granularity regulators and safety engineers want.

2.4 Mob Elaboration and Mob Construction: human-in-the-loop by design

AI-DLC borrows the spirit of mob programming, the whole team engaging together, and applies it at two points:

- **Mob Elaboration** happens in Inception. The cross-functional team works with the AI to elaborate intent into requirements and stories, surfacing assumptions and resolving ambiguity together before anything is designed.
- **Mob Construction** happens in Construction. The team works with the AI as it proposes architecture and writes code, steering and deciding rather than watching.

The “mob” framing is deliberate: it keeps humans collectively in the loop at the moments of highest leverage, intent and design, rather than reducing review to a rubber stamp after the fact.

Both mobs share a concrete mechanism. AI-DLC never asks its questions loosely in chat; it writes them to a question file as a structured, multiple-choice questionnaire, each question carrying meaningful options plus a final “Other” for anything the options miss. The team answers by filling a letter after an `[Answer]:` tag, and the workflow reads those answers back to drive the next step. Crucially, the questionnaire is itself editable: because it is just a file, the team can **add** a question the AI did not think to ask, **modify** an option, or **remove** a question that does not apply, and the workflow re-reads the edited file. So the mob shapes both the questions and the answers, which matters in an SDV where a safety or supplier voice often needs to ask something the model would not. Because the questions are generated, their exact wording and ordering can vary from one run to the next; the same concern may surface as “Question 2” here and a differently phrased “Question 3” there. That non-determinism is expected and harmless: what is stable is not the prose but the decisions the questions force and the `[Answer]:` choices recorded against them, and it is those documented answers, not the generated phrasing, that flow into requirements and the audit trail. When answers conflict, the workflow writes a follow-up clarification questionnaire rather than guessing.

2.5 Documentation-first and the audit trail

Because every phase emits artifacts, AI-DLC is documentation-first by construction. The open-source workflows write these artifacts to a structured location alongside the project, capturing requirements, designs, non-functional analysis, and an audit trail of decisions. Two benefits stand out:

- **Traceability.** You can follow a line from original intent through requirement, design, and code.
- **Evidence.** The accumulated record is exactly the kind of artifact that regulated industries need, a point we will exploit heavily for automotive compliance in Part IV.

2.6 The open-source adaptive workflows

AWS has open-sourced adaptive workflows for AI-DLC (the repository link is in the Preface). “Adaptive” is the operative word: the workflow tailors itself to the work. A small bug fix runs a minimal path, intent, light requirements, code, build and test, while a greenfield application runs the full sequence, comprehensive requirements, user stories, design, unit decomposition, and per-unit construction. The pipeline executes only the stages that add value, and humans approve transitions at each gate. A deterministic process checker verifies that each stage’s outputs meet expectations before the next begins.

This adaptivity is why AI-DLC can serve both a quick SDV defect fix and a ground-up new vehicle feature without forcing either through the wrong amount of process, a property we will lean on throughout the book.

2.7 What AI-DLC asks of the team

Repositioning AI as orchestrator does not reduce the human role; it changes its shape. Three shifts are worth naming early, because they shape how well a team adopts the method.

First, **the centre of gravity moves upstream**. Time that used to go into typing implementation now goes into stating intent precisely, answering the elaboration questionnaires well, and reviewing generated artifacts with genuine attention. Teams that treat the gates as rubber stamps get rubber-stamp results; teams that engage at intent and design get the method's real leverage. The scarce resource becomes good judgment at the gates, not keystrokes.

Second, **review becomes the throughput constraint**. Because AI can generate faster than people can validate, an organization's ability to deliver is bounded by how quickly and well it reviews, a dynamic later chapters return to as the "validation bottleneck." This is uncomfortable but honest: adopting AI-DLC without strengthening review capacity simply relocates the queue.

Third, **trust is earned through the trail, not assumed**. The value of a documentation-first workflow is that no one has to take the AI's word for anything. Every requirement, decision, and test result is recorded and inspectable, so confidence rests on evidence a human approved rather than on faith in the generator. That property is what makes an AI-led workflow defensible in a regulated setting at all, and it is the bridge from this chapter's mechanics to the automotive case the rest of the book builds.

Key takeaways

- AI-DLC repositions AI from assistant to **orchestrator**: AI leads the workflow, humans decide at defined gates.
- Work flows through three phases, **Inception** (what/why), **Construction** (how), and **Operations** (deploy, monitor, document).
- **Bolts** replace sprints as the unit of delivery; **Mob Elaboration** and **Mob Construction** keep humans collectively in the loop at the highest-leverage moments.
- AI-DLC is **documentation-first**, producing a traceable audit trail, the seed of compliance evidence.
- The open-source **adaptive workflows** scale the process up or down to fit the work.
- AI-DLC changes the human role rather than shrinking it: judgment moves upstream to intent and design, review becomes the throughput constraint, and trust rests on the inspectable trail rather than on the generator.

What's next

With both halves established, the SDV in Chapter 1 and AI-DLC here, Chapter 3 makes the case for infusing them: mapping AI-DLC's phases onto the SDV lifecycle and marking where AI orchestrates versus where engineers must decide.

Chapter 3: Why AI-DLC Fits the SDV

After this chapter you can map AI-DLC's phases onto the SDV lifecycle, see how AI-DLC complements the automotive V-model, and mark where AI orchestrates and coordinates versus where engineers must decide.

3.1 Mapping AI-DLC phases onto the SDV lifecycle

AI-DLC Phases → Software Defined Vehicle Lifecycle

AI-DLC Phase	SDV Lifecycle Stage	Application
Requirements & Specification	Vehicle Architecture & Feature Definition	AI-assisted generation of software requirements from vehicle feature specs, safety standards (ISO 26262, SOTIF), and regulatory constraints
Design & Planning	System/Software Design	AI-driven architecture decomposition across ECUs, systems-on-chip (SoCs), and domains (ADAS, infotainment, powertrain); auto-generation of interface contracts and service application programming interfaces (APIs)
Code Generation	Implementation	AI-assisted generation of embedded C/C++, AUTOSAR-compliant code, middleware configurations, and vehicle abstraction layers
Testing & Validation	V&V (Verification & Validation)	AI-generated test cases for hardware-in-the-loop/software-in-the-loop (HIL/SIL) simulations, scenario-based testing for autonomous functions, fuzz testing for vehicle communication protocols (CAN, Ethernet, SOME/IP — Scalable service-Oriented Middleware over IP)
Code Review & Security	Safety & Security Assurance	AI-powered static analysis for MISRA (Motor Industry Software Reliability Association) compliance, vulnerability scanning for connected vehicle attack surfaces, TARA (Threat Analysis & Risk Assessment) automation

AI-DLC Phase	SDV Lifecycle Stage	Application
Deployment & CI/CD (continuous integration / continuous delivery)	OTA Updates & Fleet Management	AI-optimized deployment strategies for over-the-air updates, A/B rollout policies, rollback decision-making based on fleet telemetry
Monitoring & Observability	In-Field Monitoring	AI-driven anomaly detection from vehicle telemetry, predictive maintenance triggers, real-time performance monitoring across fleet
Iteration & Feedback	Continuous Vehicle Improvement	AI-analyzed field data feeding back into requirements — feature usage patterns, failure modes, and driver behavior informing next release cycles

Key considerations for the SDV context

- **Safety-critical constraints:** AI-generated code in ASIL-rated (Automotive Safety Integrity Level) domains requires additional human review gates and formal verification.
- **Multi-domain orchestration:** Unlike typical software, SDVs span multiple domains (chassis, ADAS, body, infotainment), each with different real-time and safety requirements.
- **Hardware–software co-design:** AI-DLC must account for heterogeneous compute (microcontrollers [MCUs], graphics processing units [GPUs], and custom application-specific integrated circuits [ASICs]) and resource constraints.
- **Regulatory compliance:** the United Nations Economic Commission for Europe (UNECE) WP.29 (World Forum for Harmonization of Vehicle Regulations), ISO 21434 (cybersecurity), and type-approval processes add mandatory checkpoints that AI-DLC must integrate with.

3.2 AI-DLC and the automotive V-model: where they fit, and where they don't

Most automotive teams do not organize work around three phases; they organize it around the **V-model**, the lifecycle that ISO 26262 is built on. The left leg descends through definition and decomposition: item definition and requirements, system architecture, software architecture, and unit design; implementation sits at the base; and the right leg climbs back up through verification and validation (unit test, integration test, system test, and acceptance), with each right-side activity verifying the left-side artifact directly across from it.

AI-DLC does not replace the V. It traverses it faster and more often: a Bolt carries one feature down the left leg and up the right, and the mob's approval gates sit where the V's reviews already are. The useful question for an SDV team is therefore not “V-model *or* AI-DLC?” but “which arms of the V can AI-DLC accelerate, and which stay human-owned?” The answer follows the same maturity gradient as the rest of this chapter.

V-model stage (arm)	What AI-DLC contributes	Verdict
Item definition & requirements (left)	Structured, traceable requirements from intent; coverage-gap detection	Works well — human verifies safety content
HARA & ASIL assignment (left)	Context and cross-referencing only	Human-owned — AI-DLC does not decide

V-model stage (arm)	What AI-DLC contributes	Verdict
System & software architecture (left)	Proposed decomposition across ECUs and domains; interface contracts	Works with oversight — safety architecture human-owned
Unit / detailed design (left)	Designs and stubs for QM and ASIL-A/B with a safety monitor	Works for QM — ASIL-C/D design human-owned
Implementation (base)	Cloud/QM code, boilerplate, configuration	Works well for QM — ASIL-C/D control off the generative path (Chapter 9)
Unit & integration test (right)	Test generation at every layer	Works well
System test, SIL/HIL scenarios (right)	Scenario and fuzz-test generation	Works with oversight — final HIL gate human-run
Safety validation & homologation (right)	Assembles evidence from the audit trail	Human-accountable — AI-DLC does not sign off

Two things stand out. First, the V’s **horizontal correspondence**, every left-side artifact has a matching right-side verification, is exactly the bidirectional traceability AI-DLC produces as documentation-first artifacts (Section 3.4). AI-DLC strengthens a property the V-model already demands rather than working against it. Second, the boundary is the one drawn throughout this book: AI-DLC accelerates the *definition* and *verification* arms wherever specs are formal and tests can be automated, but the **decisions the V exists to protect** (hazard analysis and risk assessment (HARA), Automotive Safety Integrity Level (ASIL) assignment and decomposition, the safety case, and validation sign-off) stay human-accountable, and the **base of the V for ASIL-C/D control code** is off the generative path entirely (Chapter 9).

The figure overlays that verdict on the V: the definition and verification arms are AI-DLC-accelerated, the middle arms proceed under human oversight, and the safety-decision and certified-control elements remain human-owned.

In short, AI-DLC and the V-model are complementary, not competing. AI-DLC is a faster, iterative way to *walk* the V while keeping its reviews intact; it does not remove any of the V’s safety obligations, and it stops exactly where those obligations demand a human decision.

3.3 Where AI orchestrates and where engineers must decide

AI-DLC is AI-led but human-centric. Across all three phases, the AI does the generative and coordinating work, while engineers retain authority over the decisions that carry business, safety, and regulatory weight. As AWS puts it, developers “retain the ultimate responsibility of validation, decision-making and oversight,” and the AI “defers critical decisions to humans who possess the contextual understanding of business requirements.”

The table below draws that line explicitly for SDV work, phase by phase.

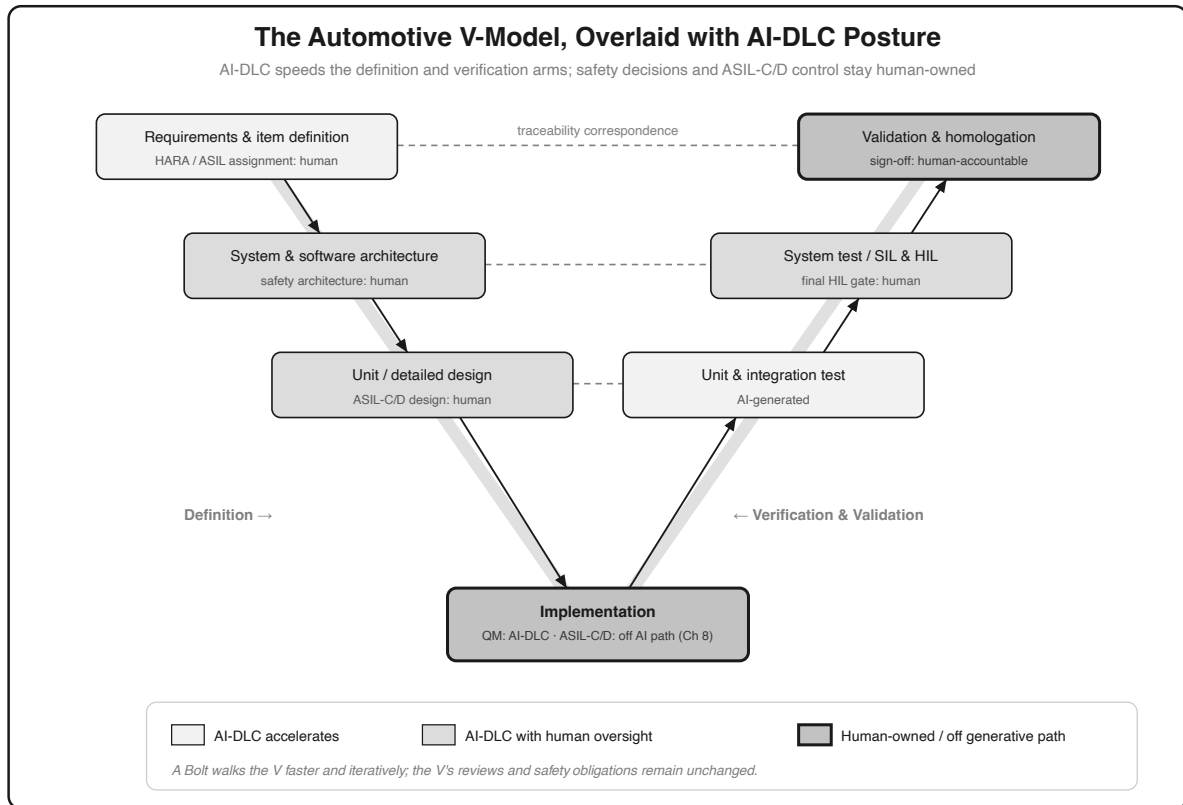


Figure 1: A V-model diagram overlaid with AI-DLC posture. The left, descending leg lists Requirements and item definition, System and software architecture, and Unit and detailed design; the base is Implementation; the right, ascending leg lists Unit and integration test, System test with SIL and HIL, and Validation and homologation. Boxes are shaded in three tones: lightest for stages AI-DLC accelerates (requirements, unit and integration test), mid-gray for stages that proceed with human oversight (architecture, unit design, implementation of QM code, system and SIL/HIL test), and darkest with a thick border for human-owned elements (HARA and ASIL assignment carved out of requirements, ASIL-C/D control at the base marked off the generative path, and validation and homologation sign-off). Dashed horizontal lines connect each left-side stage to its matching right-side verification, labeled traceability correspondence. A footer states that AI-DLC speeds the definition and verification arms while safety decisions and ASIL-C/D control stay human-accountable.

Phase	Where AI orchestrates	Where engineers must decide (human oversight)
Inception — planning & requirements	- Context building over existing vehicle software codebases (ECU firmware, middleware, AUTOSAR stacks) - AI-generated specifications — translating high-level vehicle feature intents into structured user stories and work units - Task decomposition — breaking complex SDV features (e.g., OTA update systems, ADAS modules) into manageable “Units” and atomic “Bolts” - Architectural suggestions — proposing component models for domain/zonal architectures	Defining the intent — what vehicle feature to build and why (e.g., prioritize parking assistance or lane-keeping for this model year?) Verifying AI-generated specs — ensuring requirements capture safety-critical constraints (ISO 26262, SOTIF) Architecture trade-offs — centralized vs. zonal architectures; hardware–software partitioning Scoping decisions — what is in/out of scope for each iteration
Construction — building & testing	- Code and test generation — embedded software, integration tests, hardware abstraction layers - Domain modeling — component models for vehicle software platforms - Adding architectural components — infrastructure patterns for edge-to-cloud connectivity - Deployment with IaC (infrastructure as code) — provisioning virtual ECU environments and digital twins - Mob programming orchestration — coordinating sessions where AI handles routine implementation	Verifying each Bolt output before proceeding — “verify before you move on” Safety-critical code review — real-time control, ADAS, and powertrain software Approving domain models — confirming component architectures meet automotive safety standards Mob testing decisions — test-coverage adequacy for safety-critical functions Integration decisions — how components interact across vehicle domains
Operations — deploy & monitor	- Deployment automation — CI/CD pipelines for vehicle software releases and OTA updates - AI observability — monitoring production vehicle software, detecting anomalies - Incident management — proposing rollout strategies and rollback plans - Proposing infrastructure and deployment plans for production releases	Approving releases — the Quality Steward confirms operational acceptance criteria and approves deployment Rollout strategy approval — phased vs. full-fleet OTA deployment Incident response judgment — severity, customer impact, recall vs. software patch Regulatory compliance sign-off — meeting automotive safety and cybersecurity regulations

The pattern is consistent: AI proposes, generates, and coordinates; humans direct, verify, and approve, especially wherever safety, timing, or regulation is at stake. Chapter 9 sharpens the hardest version of this boundary, the safety-critical control paths where generative AI must not be in the loop at all.

3.4 Governance, traceability, and the automotive compliance fit

The compliance challenge in SDVs

Software Defined Vehicles introduce a paradigm shift; vehicles are no longer static hardware products but continuously evolving software platforms. This creates unprecedented governance and traceability demands across the entire lifecycle.

Regulatory landscape

Standard / Regulation	Focus Area	Traceability Requirement
ISO 26262	Functional Safety	Bidirectional traceability from hazard analysis → safety requirements → implementation → verification
ISO 21434	Cybersecurity Engineering	Asset identification → threat scenarios → security requirements → validation evidence
UNECE WP.29 R155 (UN Regulation No. 155)	Cyber Security Management System (CSMS)	Documented processes for risk management across vehicle lifetime, including OTA
UNECE WP.29 R156 (UN Regulation No. 156)	Software Update Management System (SUMS)	Full traceability of every software version deployed to every vehicle in the field
SOTIF (ISO 21448)	Safety of the Intended Functionality	Scenario-based validation traceability for AI/machine-learning (ML)-driven functions
ASPICE (Automotive SPICE)	Process Maturity	Work product traceability across all engineering disciplines
European Union (EU) AI Act	AI System Governance	Transparency, explainability, and audit trails for high-risk AI systems in vehicles

The governance layers below show how policy, change control, and release approval sit above a shared traceability backbone, with audit, monitoring, and incident response drawing evidence from the same spine.

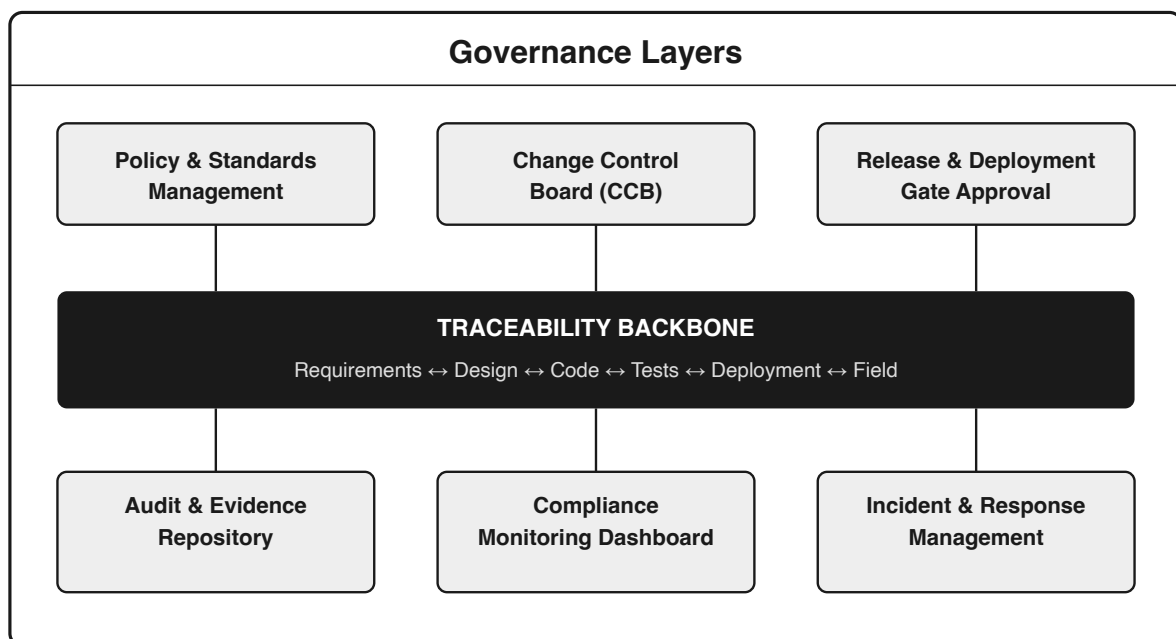


Figure 2: Governance layers: policy, change control, and release gates above a traceability backbone linking requirements, design, code, tests, deployment, and field, with audit, compliance monitoring, and incident response beneath.

Traceability requirements by domain

SDV Domain	ASIL Level	Traceability Depth	Key Artifacts
ADAS / Autonomous Driving	ASIL-D	Full formal	Safety case, hazard log, hazard analysis and risk assessment (HARA), verification evidence
Powertrain / electric vehicle (EV) control	ASIL-C/D	Full formal	Emissions compliance, battery safety, thermal models
Chassis & Dynamics	ASIL-B/C	Structured	Stability control verification, braking system validation
Body & Comfort	ASIL-A/B	Standard	Functional test evidence, requirement coverage
Infotainment / Connectivity	QM (Quality Management; non-safety)	Lightweight	Feature specs, integration tests, security scans
OTA Update System	ASIL-B + CSMS	Full formal	Update packages, rollback evidence, fleet deployment records

How AI-DLC supports compliance in SDVs

Compliance Need	AI-DLC Enabler
Bidirectional Traceability	AI auto-links requirements → code → tests; detects coverage gaps
Change Impact Analysis	AI identifies downstream effects of requirement changes across ECUs and domains
Safety Case Generation	AI assists in constructing structured safety arguments (Goal Structuring Notation / Claims-Arguments-Evidence, GSN/CAE, notation)
Audit-Ready Documentation	AI auto-generates compliance documentation from development artifacts
Configuration Management	AI tracks software bill of materials (SBOM) per vehicle variant/VIN (Vehicle Identification Number)
Continuous Compliance	AI monitors field data against safety assumptions; flags deviations
SOTIF Validation	AI generates edge-case scenarios for ML-based perception systems

The OTA governance challenge

Software updates in the field create a **living compliance obligation**:

- Pre-deployment gate**, every OTA update must demonstrate:
 - No regression in safety functions (ASIL impact assessment)
 - Cybersecurity vulnerability assessment
 - Type-approval validity (does this update change the vehicle “type”?)
 - Rollback capability proven
- In-field monitoring**, post-deployment:
 - Fleet-wide anomaly detection
 - Safety KPI (key performance indicator) tracking per software version
 - Regulatory reporting readiness (recall triggers)
- Lifetime traceability**, for every VIN:
 - Complete software version history

- Which requirements/tests apply to the currently deployed version
- Evidence that each deployed version passed all gates

In the SDV world, **governance and traceability are not overhead; they are the product**. The ability to prove compliance continuously, across millions of vehicles receiving frequent updates, is a competitive differentiator. AI-DLC tools that embed traceability natively (rather than bolt-on documentation) become essential enablers for automotive OEMs (original equipment manufacturers) and Tier-1 suppliers navigating this landscape.

The figure below brings these threads together, showing AI-DLC as an orchestration layer that drives a specific governance and traceability action at each of the seven phases, mapped to the compliance standard it satisfies.

Key takeaways

- AI-DLC's phases map cleanly onto the full SDV lifecycle, from requirements through field monitoring and feedback.
- AI-DLC does not replace the automotive V-model; it walks the V faster and more iteratively, accelerating the definition and verification arms while the safety decisions and ASIL-C/D control the V exists to protect stay human-owned.
- AI proposes, generates, and coordinates; humans direct, verify, and approve, especially where safety, timing, or regulation is at stake.
- In SDVs, governance and traceability are the product: AI-DLC's documentation-first audit trail fits ISO 26262, ISO 21434, and UNECE R155/R156 evidence needs.

What's next

Chapter 4 turns from *whether* AI-DLC fits to *where and how much* to apply it: a value, risk, and feasibility framework for choosing workloads, and an honest read of what is realistic today and what is not.

AI-DLC Orchestrating SDV Governance & Compliance How an AI-powered development lifecycle automates traceability, governance gates, and regulatory compliance across the Software Defined Vehicle		
AI-DLC Phase	Governance & Traceability Action	Compliance Standard
Requirements & Specification	Auto-Link Hazards → Safety Goals → Requirements AI parses HARA outputs to generate structured safety requirements with ASIL tagging. Creates bidirectional trace matrix from hazard IDs to requirement IDs. Flags gaps in coverage automatically.	ISO 26262 Part 3 HARA, Safety Goals, ASIL Assignment
Design & Architecture	Trace Requirements → Architecture Decisions AI maps each safety/security requirement to system architecture elements (ECUs, domains, interfaces). Auto-generates ASIL decomposition rationale. Validates that no safety requirement is orphaned across the architecture.	ISO 26262 Part 4 Technical Safety Concept, ASPICE SWE.2
Code Generation & Implementation	Enforce Coding Standards & Generate SBOM AI generates MISRA-compliant code with inline requirement trace tags. Auto-creates Software Bill of Materials per build variant/VIN. Embeds traceability metadata in every code artifact linking back to design decisions.	MISRA C/C++ ISO 21434 SBOM, Config Management
Security & Review	Automate TARA & Vulnerability Tracing AI performs static analysis for MISRA violations, scans for CVEs against SBOM, maps each vulnerability to affected vehicle functions. Auto-generates cybersecurity assessment per UNECE R155. Creates audit-ready evidence packages.	ISO 21434 UNECE WP.29 R155, CSMS Certification
Testing & Validation	Generate Test Cases & Prove Coverage AI derives test cases from requirements ensuring 100% ASIL-level coverage. Generates SOTIF edge-case scenarios for ML perception. Auto-traces test results back to requirements — producing verification evidence matrix per release.	ISO 21448 (SOTIF) ASPICE SWE.6, HIL/SIL Evidence
Release Gate & OTA Deploy	Automated Compliance Gate & Deployment Tracing AI evaluates release readiness: safety regression analysis, type-approval impact check, rollback proof. Logs every OTA package to every VIN with full version lineage. Generates SUMS compliance report per update campaign.	UNECE WP.29 R156 SUMS, Type-Approval, Fleet Records
Field Monitoring & Feedback	Continuous Compliance & Closed-Loop Tracing AI monitors fleet telemetry against safety assumptions. Detects anomalies triggering trace-back to root cause (requirement, code, test). Auto-generates regulatory incident reports. Feeds field data into requirements — closing the loop.	Lifetime CSMS Regulatory Reporting, Recall Triggers

Figure 3: AI-DLC orchestrating SDV governance and compliance: for each of the seven phases (Requirements, Design, Code Generation, Security, Testing, Release/OTA, Field Monitoring), the matching governance and traceability action and the compliance standard it satisfies, from ISO 26262 and ISO 21434 to UNECE R155/R156 and SOTIF.

Chapter 4: Deciding Where AI-DLC Fits — Value, Risk, and Maturity

After this chapter you can decide which SDV workloads to run through AI-DLC using a value, risk, and feasibility framework, keep the running cost of an AI-led workflow under control, and set honest expectations about what is realistic today and what is not.

Chapter 3 argued that AI-DLC *fits* the SDV lifecycle. Fitting is not the same as applying it everywhere. This chapter is the decision layer: a structured way to judge which vehicle-software workloads should run through AI-DLC and how aggressively, followed by an honest assessment of where the tooling genuinely delivers today and where it does not yet. Together they keep adoption anchored to value and risk rather than enthusiasm.

4.1 A value, risk, and feasibility framework for AI-DLC in SDV

Not every SDV workload should run through AI-DLC, and the decision is rarely binary. The framework below evaluates a workload across three pillars (Value, Risk, and Feasibility, or VRF) and combines them into a single recommendation. Each sub-criterion uses qualitative anchors (Very Low to Very High) rather than hard numbers, so the model stays relevant as tools and technology evolve. The question it answers is simple: *should this workload be developed using AI-DLC, and how aggressively?*

Framework structure

The three pillars combine into a composite recommendation. Each sub-criterion is rated on a five-level scale (Very Low, Low, Medium, High, Very High), and pillar scores are weighted qualitative averages. The final band is derived from the composite position across all three pillars.

Composite position	Band	Recommendation
High Value + High Feasibility + Low Risk	Accelerate	Full AI-DLC adoption; assign to next sprint
Moderate-High across pillars	Pilot	Time-boxed AI-DLC pilot; measure before scaling
Mixed signals; one or more pillars weak	Selective	Apply AI-DLC to specific sub-tasks only
Low Feasibility or High Risk dominates	Defer	Traditional SDLC (software development life cycle); re-evaluate as tooling matures

Hard veto: if Functional Safety is Very Low (ASIL-C/D with no safety monitor) *or* Regulatory

is Very Low (homologation-blocking), the workload is automatically deferred regardless of how the other pillars score. Safety and regulatory constraints override value and feasibility every time.

Pillar 1 — Value (primary weight)

What business outcome does running this workload through AI-DLC unlock? Value carries the primary weight: a workload has to be worth doing before risk and feasibility matter.

#	Sub-criterion	Weight	Very Low	Medium	Very High
V1	Time-to-Market	High	No schedule pressure	Next model-year deadline	Critical-path; contractual SOP (start of production)
V2	Cost Reduction	High	Marginal savings	Meaningful reduction	Transformative; headcount reallocation
V3	Quality Impact	Medium	Unlikely to improve	Moderate via test gen	Catches bug classes humans miss
V4	Differentiation	Medium	Commodity; no edge	Visible, not sole factor	First-mover advantage
V5	Reusability	Low	One-off; no reuse	Across 2–3 programs	Platform; entire portfolio

Pillar 2 — Risk (secondary weight)

What could go wrong if AI-generated code enters this workload? Risk is scored so that Very Low is the safest outcome. Functional Safety and the hard veto live here.

#	Sub-criterion	Weight	Very High Risk	Medium Risk	Very Low Risk
R1	Functional Safety	High	ASIL-C/D; steering, braking	ASIL-A/B with safety monitor	QM; no safety relevance
R2	Cybersecurity	Medium	Vehicle bus (CAN/Ethernet); UN R155	Cloud endpoint; bounded	Internal; no attack surface
R3	Regulatory	Medium	Homologation-critical path	Awareness needed, not blocking	No regulatory touchpoint
R4	Intellectual-property (IP) sensitivity	Medium	OEM proprietary; joint-venture (JV) boundaries	Standard; clear ownership	Open-source friendly
R5	Blast Radius	Low	Recall / fleet OTA rollback	Degraded for subset; fallback	Contained; auto-retry

Pillar 3 — Feasibility (secondary weight)

Can we actually deliver this workload through AI-DLC today? Feasibility is the pillar most likely to shift over time as tooling, specs, and team proficiency improve, which is why it should be re-scored periodically.

#	Sub-criterion	Weight	Very Low	Medium	Very High
F1	Code Patterns	High	Novel algorithm R&D (research and development)	Domain-specific; templates exist	Well-known (REST — representational state transfer — pipelines, IaC)
F2	Spec Completeness	High	No formal spec; negotiating	Requirements exist; gaps remain	Full spec (OpenAPI, ARXML [AUTOSAR XML], IDL [interface definition language])
F3	Verification	Medium	Manual/physical only (HIL)	SIL + partial; HIL final gate	Fully automated CI; mocks ready
F4	Team Proficiency	Medium	No training; skepticism	Some trained; org approved	Proficient; AI pull-request (PR) culture
F5	Toolchain	Medium	Proprietary; no AI path	Standard CI/CD; moderate work	Agent PRs + auto-review live

Scoring worksheet

To apply the framework, rate each sub-criterion (Very Low to Very High), roll the ratings up into a weighted assessment per pillar, then read off the overall band.

Pillar	#	Sub-criterion	Rating	Weight
Value	V1	Time-to-Market acceleration	_____	High
	V2	Development cost reduction	_____	High
	V3	Quality & defect-density impact	_____	Medium
	V4	Strategic differentiation	_____	Medium
	V5	Reusability & platform leverage	_____	Low
		<i>Value pillar assessment</i>	_____	
Risk	R1	Functional safety (ASIL)	_____	High
	R2	Cybersecurity exposure	_____	Medium
	R3	Regulatory & certification	_____	Medium
	R4	IP & supply-chain sensitivity	_____	Medium
	R5	Defect blast radius	_____	Low
		<i>Risk pillar assessment</i>	_____	
Feasibility	F1	Code-pattern maturity	_____	High
	F2	Specification completeness	_____	High
	F3	Automated verification coverage	_____	Medium
	F4	Team AI-DLC proficiency	_____	Medium
	F5	Toolchain integration	_____	Medium
		<i>Feasibility pillar assessment</i>	_____	
		Overall VRF band	_____	

SDV workload positioning map

Applying the framework across common SDV workloads produces a consistent gradient: cloud-side work accelerates, the mid-stack pilots or applies selectively, and safety-critical control defers.

Workload	Value	Risk	Feasibility	Band
Cloud fleet management	High	Low	High	Accelerate
OTA orchestration (cloud)	High	Low	High	Accelerate
Connected data pipelines	High	Low	High	Accelerate
Digital twin simulation	Medium-High	Low	Medium-High	Pilot
Infotainment / HMI (human-machine interface) apps	Medium	Low	Medium	Pilot
Diagnostic services	Medium	Low	Medium	Pilot
Vehicle middleware (SOA — service-oriented architecture)	Medium	Medium	Medium	Selective
Body/comfort ECU	Low-Medium	Medium	Low	Selective
Powertrain/chassis control	High	Very High	Low	Defer
ADAS/AD algorithms	High	Very High	Low	Defer

Governance and review cadence

The framework is not a one-time gate. Because feasibility and tooling move, bands must be revisited on a regular cadence with clear ownership.

Activity	Cadence	Owner
Score new workloads entering backlog	Per intake	Product Owner + AI-DLC Coach
Re-score existing workloads	Quarterly	Architecture Review Board
Adjust weights for org context	Semi-annually	VP (Vice President) Engineering / CTO (Chief Technology Officer)
Update thresholds as tooling matures	Per platform release	AI-DLC Platform Team
Audit AI-generated code in production	Monthly	Quality and Safety Lead

Customizing the framework

The defaults above are a starting point. Tune the model to your organization and regulatory context:

- **Industry-specific risks:** add sub-criteria where needed (e.g., Export Control / ITAR, International Traffic in Arms Regulations, for defense-adjacent SDV programs).
- **OEM maturity:** increase the Feasibility weight for organizations early in AI adoption.
- **Supplier vs. OEM:** suppliers tend to weight IP sensitivity higher; OEMs weight time-to-market higher.
- **Regional regulation:** adjust the regulatory anchors for EU (UNECE), US (FMVSS, Federal Motor Vehicle Safety Standards), or China (GB, Guobiao national standards).
- **Portfolio scaling:** normalize ratings across programs to support portfolio-level prioritization.
- **Evolving tooling:** as AI-DLC tools mature, shift the feasibility anchors upward and re-evaluate previously deferred workloads.

4.2 What is realistic today, and what is not

AI-DLC is not equally effective across the SDV software stack. Its value tracks a clear gradient: generability is highest where specifications are formal and verification is automated (cloud services), and it falls away as you move toward safety-critical, real-time, and certified layers. The honest assessment below sorts SDV workloads into three bands (proven today, emerging but not yet reliable, and not realistic with current tooling) so teams can target effort where it pays off and avoid overpromising where it does not.

At a glance

	Realistic Today	Emerging / Partial	Not Yet Realistic
Scope	Cloud services, test generation, docs, boilerplate, data pipelines	Vehicle middleware (SOA), infotainment, SIL-testable modules	Safety-critical embedded, ADAS algorithms, certified code
Gains	Substantial effort reduction	Modest gains with human-in-the-loop	Negligible or negative
Trust	High — CI validates output	Medium — expert review required	Low — needs physical validation

Realistic today

In these areas a formal specification exists, verification can be automated, and no safety-certification dependency blocks the work. AI-DLC delivers measurable gains now.

Category	What works	Gain
Cloud services	REST/gRPC from OpenAPI, IaC (Terraform/CDK), data pipelines (Kafka/Kinesis), OTA orchestration, fleet management APIs	Major effort reduction
Test generation	Unit, integration, contract tests — any layer	Major effort saved
Documentation	API docs, architecture docs from structured codebases	High quality, minimal editing
Code review	Bug detection, style, security scanning	Meaningfully faster reviews
Boilerplate	Project scaffolding, CI config, schema migrations	Near-complete accuracy
Spec-to-code	OpenAPI / Franca IDL to implementation stubs	Works when spec is formal

Conditions for success: a formal spec exists, automated verification is possible, the team is trained, and there is no safety-certification dependency.

Emerging but not yet reliable

Vehicle middleware and platform software is where AI-DLC shows promise but still needs expert review. It handles the happy path and generates plausible skeletons, but misses the domain-specific concerns that matter most.

Area	Status	Gap
AUTOSAR Adaptive stubs	Can generate skeletons from ARXML	<code>ara::com</code> under-represented; partial quality
SOA integration	Handles happy path	Misses timing, QoS (quality of service), service discovery
Vehicle abstraction layer	Template generation possible	Real-time constraints not modeled
Infotainment / HMI	UI (user interface) generation works	Vehicle HAL (hardware abstraction layer) integration fragile

The productivity paradox

A counter-intuitive dynamic has emerged as adoption has scaled:

- AI generates code several times faster than teams can review it.
- Tool adoption is widespread, but developer trust remains low: only a minority of developers trust the output.
- Experienced developers can be measurably slower on complex codebases because of context-window limits.
- Review, security scanning, and compliance pipelines cannot absorb the volume of AI-generated code.

The implication is decisive for automotive: the bottleneck shifts from *writing* code to *validating* it, and in this domain validation is already the dominant cost. Generating more code faster does not help if it cannot be verified faster.

Not realistic today

These areas are blocked by technical limits, certification requirements, or both. Treating them as candidates for primary code generation invites risk.

Area	Blocker	Horizon
ASIL-C/D code (braking, steering, powertrain)	Zero latent-defect tolerance; no MISRA guarantee; no formal verification	Long-term
AUTOSAR Classic BSW (basic software)	Proprietary, MCU-coupled; vendor toolchains have no AI path	Medium-term
Real-time control algorithms	Cannot reason about worst-case execution time (WCET), interrupt handling, determinism	Long-term
ISO 26262 Part 6 compliant code	Traceability chain required; AI provenance unclear to assessors	Awaiting guidance
UN R155/R156 cybersecurity scope	TARA-to-code mapping not maintained by current tools	Medium-term
Homologation-blocking software	Type-approval requires human-accountable sign-off; AI role undefined	Regulatory clarity needed
ADAS perception (camera, LiDAR — light detection and ranging)	Novel architectures; AI cannot design new ML models	Frontier research
Motion planning	Requires formal safety proofs (responsibility-sensitive safety [RSS], safety force field [SFF])	Active research
Legacy embedded C migration (bulk)	Context windows cannot hold very large multi-file codebase intent	Near-to-medium term

The maturity curve

Effectiveness decreases and verification difficulty increases as you move from cloud-side workloads toward safety-critical layers.

Workload layer	Effectiveness	Band
Cloud services, data pipelines	High — full AI-DLC end-to-end	Accelerate
Test generation, documentation	High — any layer	Accelerate
Boilerplate, scaffolding, CI/CD	High — near-complete automation	Accelerate
Infotainment UI, diagnostics	Medium — works for standard patterns	Pilot
Vehicle middleware (AUTOSAR Adaptive)	Medium-Low — gaps in domain patterns	Pilot
Body control, comfort ECU	Low — limited to sub-tasks	Selective
Powertrain, chassis (docs only)	Very Low — documentation assist only	Defer
ADAS/AD algorithms	Not applicable for primary code gen	Defer

Where to start

Now	Next	Wait
Cloud fleet APIs	AUTOSAR Adaptive service layer	ASIL-C/D code
OTA services (cloud)	Infotainment app logic	Powertrain control
Data pipelines	Diagnostic services	ADAS algorithms
Test generation (all layers)	Middleware integration tests	Real-time embedded
Documentation (all layers)	HMI / UI generation	Legacy migration (bulk)
CI/CD automation	SIL test scenario generation	Homologation software

Claims you can defend — and overpromises to avoid

Defensible positions:

- AI-DLC accelerates cloud-side SDV services substantially, starting now.
- Vehicle middleware is a strong pilot candidate: moderate gains with guardrails.
- Test generation and documentation apply even where primary code generation does not.
- Selective sub-task application works within safety-relevant programs.
- Re-score the bands periodically as tooling evolves.

Overpromises to avoid:

- “AI can write safety-critical code that passes ISO 26262.”
- “End-to-end AI-DLC for ADAS with human review only.”
- “AI replaces embedded engineers.”
- “Across-the-board savings on the entire vehicle software stack.”
- “AI-generated code is production-ready without expert review.”

4.3 Controlling the cost of AI-DLC: measure, cap, and alert

The Value and Feasibility pillars quietly assumed a cost this section makes explicit. Traditional development’s cost was mostly salaried human time; an AI-led workflow adds a new, *variable* line item: the inference cost of every model call across Inception and Construction, which grows with the tokens consumed, the tool calls made, and the number of times the workflow loops to regenerate. As autonomous agents drive more of the loop between gates, the direction Chapter

17 anticipates, that cost can run away quietly: an agent that retries, expands its context, or spawns sub-tasks can post a large bill before a human notices. Treating cost as an afterthought is how a promising pilot produces a surprising invoice.

The discipline that keeps this in check is easy to state: **measure, cap, and alert**. It maps cleanly onto AI-DLC's structure.

Lever	What it means for AI-DLC	SDV example
Measure	Track tokens, tool calls, and regeneration iterations per Bolt; attribute cost per unit and per tier	A cost-per-Bolt line recorded in the same audit trail that already logs decisions
Cap	Per-Bolt and per-run budgets, a hard iteration limit, and right-sized model selection	Boilerplate and tests run on a small, cheap model; architecture and complex reasoning justify a stronger one
Alert	Anomaly thresholds with real-time, human-in-the-loop notification	A Bolt whose spend exceeds its budget pauses and asks a human before continuing

Three levers do most of the work.

Measure per Bolt, and attribute. Because a Bolt is already the unit of delivery and its artifacts accumulate in the audit trail, it is the natural unit of cost too. Recording tokens, tool calls, and regeneration count per Bolt, and attributing them per unit and per tier, turns cost from an opaque monthly total into something visible by feature and by layer. The per-layer split from the DORA discussion (Chapter 11) applies here: a cloud unit and an in-vehicle unit have very different cost profiles, and a single fleet-wide number hides both.

Cap before you spend. A budget checked only on a monthly dashboard is not a control; by then the money is gone. Effective caps sit in the workflow's path: a per-Bolt token budget, a hard limit on regeneration iterations (a Bolt that cannot converge in a set number of attempts stops and asks a human rather than looping), and, most powerful, right-sizing the model to the task. The maturity gradient that governs *where* AI-DLC applies also governs *what to spend*: boilerplate, tests, and documentation run well on smaller, cheaper models, while design and complex reasoning justify a stronger, costlier one. Paying a frontier model's rate to scaffold a REST endpoint is waste, not rigor.

Alert, with a human in the loop. An anomaly (a Bolt whose spend suddenly exceeds its budget, an agent stuck in a retry loop) should trip a real-time alert and pause, not silently continue. This is the same human-in-the-loop principle the book applies to safety, turned toward cost: the workflow proposes and generates, but a runaway is a decision point where a person chooses to continue, re-scope, or stop.

Two SDV-specific notes sharpen the picture. First, the safety boundary is also a cost boundary: because generative AI never authors the ASIL-C/D control path (Chapter 9), no inference is spent there, so cost concentrates in the QM layers where value is highest. Second, brownfield is a cost driver to watch: reverse-engineering a large legacy estate (Chapter 14) can consume enormous context, so scoping the work to the unit and its interfaces is a cost control as much as a capability one.

One caution closes the loop. Cost control must never become a reason to weaken the safety or review gates. The cheapest possible run is one that ships unreviewed code straight to a vehicle, precisely the outcome the whole book forbids. Right-size models, cap iterations, and alert on runaways; but money "saved" by thinning validation is a false economy in a domain where a

defect is measured in recalls. Cost is a pillar of the adoption decision, never an override of the safety veto.

Key takeaways

- Use the Value / Risk / Feasibility framework to decide what to run through AI-DLC, with a hard veto when functional safety or regulatory exposure is severe.
- Effectiveness follows a gradient: accelerate cloud-side and cross-cutting work (tests, docs), pilot the mid-stack, and defer safety-critical control.
- Be honest about maturity: claim cloud-side gains and middleware pilots; avoid promising certified safety-critical code or human-review-only ADAS.
- Re-score workloads on a regular cadence: feasibility and tooling move, so a Defer today may become a Pilot later, while the safety veto does not move.
- Control the running cost of AI-DLC by *measuring* tokens, tool calls, and iterations per Bolt, *capping* budgets and right-sizing models to the task, and *alerting* on runaways, without ever trading away the safety or review gates.

What's next

Part II walks the Inception phase for SDV, starting with intent and requirements.

Chapter 5: Capturing SDV Intent and Requirements

After this chapter you can run the AI-DLC Inception phase for a vehicle feature: bootstrapping intent, elaborating it as a team, and producing structured, traceable requirements for safety- and timing-critical functions.

5.1 Intent bootstrap for vehicle features

Inception begins with *intent*, not requirements. The intent bootstrap is the short, structured conversation that turns a one-line feature idea into a captured statement of what the team wants to build and why, with just enough vehicle context for the AI to elaborate it sensibly in the next step. The goal here is not completeness; it is to anchor everyone on the same problem before ambiguity gets expensive.

The running example: EV charge scheduling

We will carry one feature through the rest of this book: **EV charge scheduling**. A driver sets a departure time from a mobile app; the vehicle charges so the battery reaches its target state of charge (SoC) just before departure, optimizing for cheap off-peak electricity and battery longevity, and is pre-conditioned to the right temperature on arrival. The feature spans a cloud backend (scheduling, tariff data, the app API), an in-vehicle charge-management service, and an OTA channel that delivers and updates the in-vehicle component. This feature is realized end to end in the book's companion repository (see the Preface); the artifacts and source referenced in the coming chapters all live there.

We chose it deliberately. It carries real customer value, it is feasible to build with today's tooling, and it sits at an instructive boundary: the scheduling and orchestration logic is a strong AI-DLC candidate, while the battery's thermal and cell protection limits are safety-relevant and stay human-engineered. That contrast (most of the feature accelerated, one core firmly out of the loop) recurs throughout the book.

Seeding the intent

A bootstrap starts from a seed: a plain-language statement of the idea. For our example:

Let EV drivers schedule charging to a target state of charge by a chosen departure time, minimizing energy cost and battery wear, with the cabin pre-conditioned for departure.

That sentence is enough to begin, but it hides dozens of unstated assumptions. The AI's job in the bootstrap is to surface them with targeted questions rather than guessing, and in a vehicle, the questions are different from those for a typical cloud app.

Structured questions tuned to vehicle context

The bootstrap probes the dimensions that determine how the feature will later be designed, decomposed, and governed. For an SDV feature these include:

Dimension	What the bootstrap asks	For EV charge scheduling
Actors & triggers	Who or what initiates the feature, and from where?	Driver via mobile app; also a recurring schedule; possibly a utility demand-response signal
Vehicle domains & ECUs	Which compute and ECUs are involved?	Telematics/connectivity unit, charge-management/Battery Management System (BMS) gateway, Heating, Ventilation, and Air Conditioning (HVAC)/thermal controller
Signals & data	What vehicle signals are read or written?	State of charge, battery temperature, charger state, plug status, cabin temperature, location
Cloud vs. in-vehicle split	What runs off-board vs. on the vehicle?	Scheduling, tariff lookup, and app API in the cloud; charge sequencing on the vehicle
Safety relevance	Could this influence a hazard? At what ASIL?	Orchestration is QM/low-ASIL; battery thermal and cell protection limits are safety-relevant and owned by the BMS
Timing & real-time needs	Are there deadlines or latency constraints?	Soft real-time: readiness by departure; charge stop on thermal limit is hard and stays in the BMS
Connectivity & OTA	How is it delivered and updated? Offline behavior?	OTA-delivered in-vehicle service; must behave safely with no connectivity
Regulatory touchpoints	Which standards or approvals apply?	UNECE R156 for software updates; data-privacy rules for location/usage data
Value & priority	Why now, and how does success look?	Differentiator for the model year; success = readiness accuracy and measurable cost savings

These questions do double duty. They remove ambiguity early, and they begin populating the context that later phases and the compliance audit trail will draw on. The bootstrap poses them not as open chat but as a multiple-choice questionnaire the team answers with **[Answer]**: tags and can edit, adding a vehicle-specific question or removing one that does not apply, the same mechanism Mob Elaboration uses in the next section.

The captured intent artifact

The bootstrap's output is a concise, documentation-first intent record, the first artifact in the feature's trail. It states the problem, the actors, the in/out-of-scope boundaries, the domains and signals touched, the safety and regulatory flags raised, and the open questions to resolve in Mob Elaboration. A captured intent for our example reads, in essence:

- **Intent:** charge an EV to a target SoC by a driver-set departure time, minimizing cost and battery wear, with cabin pre-conditioning.
- **In scope:** mobile trigger, cloud scheduling, tariff optimization, in-vehicle charge sequencing, OTA delivery, status feedback.
- **Out of scope:** the battery thermal protection and cell-balancing control itself (owned by the BMS; safety-relevant), public-charger payment.

- **Flags:** safety boundary at the BMS interface; R156 applies; location and usage data are privacy-sensitive.
- **Open questions:** behavior with no connectivity, conflicting schedules, tariff data source, target SoC limits.

Crucially, the bootstrap does not try to *answer* the open questions or write requirements. It frames them. Resolving them is the work of Mob Elaboration, where the cross-functional team, including the people who own the safety-related boundaries this intent just flagged, turns intent into structured requirements. That is the subject of the next section.

5.2 Mob Elaboration with cross-functional automotive teams

Intent gets the team to the same problem; Mob Elaboration gets them to the same answer. This is the Inception working session where the cross-functional team sits with the AI and turns the captured intent, open questions and all, into agreed, structured requirements. The AI proposes; the people in the room decide, correct, and commit. Nothing here is a rubber stamp: the highest-leverage moment in the whole feature is settling what it must do before any design exists.

Who is in the mob

An SDV feature touches more disciplines than a typical application, so the mob is broader than a software team. For EV charge scheduling:

Role	What they own in the session
Product owner	The intent, scope boundaries, and priority; arbitrates trade-offs
Vehicle software engineer	The in-vehicle charge-management service and ECU interfaces
Cloud/backend engineer	Scheduling, tariff integration, the app API
Systems/battery engineer	Charging behavior, SoC targets, and the BMS interface
Functional safety engineer	ASIL classification, the safety boundary, and what must stay out of the AI's scope
Cybersecurity engineer	The vehicle bus and connectivity attack surface; UNECE R155 cybersecurity concerns
Test engineer	How each requirement will be verified — SIL/HIL testing or field
Supplier representative	The BMS/charger interface contract, where a Tier-1 owns the component

The safety and supplier voices matter especially early. The intent artifact flagged a safety boundary at the BMS; this is the session where that boundary becomes an explicit, agreed line rather than an assumption discovered later in code review.

Surfacing assumptions early

Mob Elaboration works by making the implicit explicit. The AI replays the open questions from intent and proposes candidate answers; the mob accepts, revises, or rejects each. A few examples from our feature:

- **No connectivity at departure.** Proposal: cache the last schedule on the vehicle and execute it locally; fall back to immediate charge if no schedule exists. *Decision:* accept, and add a requirement that local execution never depends on the cloud.

- **Conflicting schedules** (app vs. recurring vs. demand-response). Proposal: a documented precedence order. *Decision*: accept; safety/charge limits always win over cost optimization.
- **Target SoC bounds**. Proposal: allow 50–100%. *Decision*: revise; the systems engineer caps the default for battery longevity; the BMS still enforces the hard limit independently.
- **Who stops charging on a thermal event**. *Decision*: the BMS, autonomously, with no dependency on the scheduling logic. This is recorded as a constraint, not a feature requirement.

Each resolved assumption becomes a requirement or an explicit constraint, and each is captured with its rationale, the documentation-first habit that later feeds traceability.

The questionnaire the AI writes

These assumptions are not surfaced as free-form chat. The workflow writes them to a question file (for our feature, `requirements-questions.md`)* as a multiple-choice questionnaire, and the mob answers by filling in a letter after each `[Answer]:` tag. A small excerpt:

```
# Requirements Clarification Questions - EV Charge Scheduling
# (aidlc-docs/inception/requirements/requirements-questions.md)

## Question 1
With no connectivity at the departure time, what should the vehicle do?
A) Execute the most recent cached schedule locally, with no cloud dependency
B) Fall back to immediate charge within the BMS-permitted envelope
C) Wait for connectivity and take no action until a plan arrives
D) Other (please describe after [Answer]: tag below)
[Answer]: A

## Question 3
What target state-of-charge range should the app allow the driver to set?
A) 50–100% with no further restriction
B) 50–80% default for longevity, with 100% as an explicit opt-in
C) Fixed at 100% only
D) Other (please describe after [Answer]: tag below)
[Answer]: B
```

The questionnaire is editable, and in a vehicle that matters: the team can add a question the model did not pose, change an option, or remove one that does not apply, and the workflow re-reads the file. On this feature the functional-safety engineer *added* a question: “on a battery thermal event, which component stops charging?” (answer: the BMS, autonomously), and the mob *removed* an out-of-scope question about mobile-OS support. Each answered question then becomes a requirement or constraint with a recorded rationale: Q1’s answer drives CS-FR-021 (the cached plan behavior realized in `schedule_executor.cpp`), and the added safety question pins CS-SR-002 at the BMS boundary. Because the questionnaire is generated, its phrasing and numbering are not fixed; a later run might reorder these or word them differently, which is why the excerpt above jumps from Question 1 to Question 3; the recorded answers and the requirements they yield are authoritative, not the wording. When answers contradict one another, the workflow writes a follow-up clarification questionnaire rather than guessing.

*Requirement identifiers use the form CS-TYPE-NUMBER, where CS marks the charge-scheduling running example and TYPE is FR (functional requirement), NFR (non-functional requirement), SR (safety requirement), or SEC (security requirement). The numbers are stable handles reused across chapters, so each requirement can be traced from intent through design, code, and test.

5.3 Structured requirements for safety- and timing-critical functions

The output of elaboration is a set of structured requirements: uniquely identified, individually verifiable, and tagged with the attributes automotive engineering depends on. Generic software requirements capture *what* the system does; SDV requirements must also capture *how safely*, *how fast*, and *under what conditions*.

Anatomy of an SDV requirement

Each requirement carries more than a statement. A useful structure:

- **ID:** stable identifier for traceability (e.g., CS-FR-014); in our example CS marks the Charge-Scheduling feature and the middle code marks the requirement kind: FR functional requirement, NFR non-functional requirement, SR safety requirement, SEC security requirement.
- **Statement:** a single, testable assertion.
- **Type:** functional or non-functional.
- **ASIL / safety classification:** QM through ASIL-D, with the hazard it relates to.
- **Timing:** any deadline, latency, or rate, and whether it is soft or hard.
- **Trigger / precondition:** when the requirement applies.
- **Verification method:** how it will be checked (review, SIL, HIL, field).
- **Source:** link back to the intent and the elaboration decision.

Example requirements for EV charge scheduling

ID	Requirement	Type	ASIL	Timing	Verify
CS-FR-014	The vehicle shall begin charging so the battery reaches the target SoC no later than the set departure time, when a valid schedule and grid power are present.	Functional	QM	Soft: ready by departure	SIL + HIL
CS-FR-021	When no connectivity is available, the vehicle shall execute the most recent cached schedule without requiring the cloud.	Functional	QM	—	SIL + field
CS-NFR-007	Cabin pre-conditioning shall complete within a configurable window before departure without delaying readiness of the target SoC.	Non-functional	QM	Soft	SIL

ID	Requirement	Type	ASIL	Timing	Verify
CS-SR-002	The charge-management service shall treat BMS thermal and cell protection limits as authoritative and shall not command charging that the BMS has inhibited.	Safety constraint	Inherited (BMS ASIL-C/D)	Hard: honor inhibit immediately	HIL + review
CS-SR-003	A loss of the BMS interface shall cause the service to stop initiating charge and enter a safe idle state.	Safety constraint	ASIL-relevant	Hard	HIL
CS-SEC-005	All app-to-cloud and cloud-to-vehicle schedule commands shall be authenticated and integrity-protected.	Non-functional	QM (security)	—	Review + penetration test

Two things are worth noting. First, the **safety constraints (CS-SR-*)** are not features the AI will build; they are boundaries on what any generated code may do, and they point at the BMS, which stays human-engineered. Second, **timing is explicit and typed**: most of the feature is soft real-time, but honoring a BMS inhibit is hard, and that distinction drives both design and verification later.

AI-DLC captures this as a documentation-first artifact rather than as scattered notes. The requirements workflow writes `aidlc-docs/inception/requirements/requirements.md`; an excerpt for our feature:

```
# Requirements - EV Charge Scheduling

## Intent Analysis
- User request: Schedule EV charging to a target SoC by a driver-set departure
  ↪ time
- Request type: New Feature
- Scope: Multiple Components (cloud + in-vehicle + OTA)
- Complexity: Complex

## Functional Requirements
- CS-FR-014 Reach target SoC by departure when plugged in and grid power present
- CS-FR-021 Execute most recent cached schedule with no connectivity

## Non-Functional Requirements
- CS-NFR-007 Complete cabin pre-conditioning within the configured window
- CS-SR-002 Treat BMS thermal/cell protection limits as authoritative (safety)
- CS-SEC-005 Authenticate and integrity-protect all schedule commands (security)
```

Capturing SOTIF and edge-case intent

Beyond functional safety, requirements should record the *intended-functionality* concerns (ISO 21448 / SOTIF) and edge cases the mob surfaced: degraded sensors, stale tariff data, a plug pulled mid-charge, clock skew across cloud and vehicle. Each becomes either a requirement or a logged scenario for later test generation, so nothing identified in Inception is silently dropped.

5.4 Removing ambiguity before design (the Inception goal)

The explicit purpose of Inception is to remove ambiguity *before* any design or code exists. In ordinary software, ambiguity discovered late is expensive. In an SDV it can be unsafe, and it is expensive in a particular way: a requirement gap found after code generation may have already propagated into ECU interfaces, test plans, a safety case, and an OTA package, each of which must then be reworked and re-verified.

AI-DLC attacks ambiguity in two ways. The AI is good at *finding* it, replaying under-specified intent as concrete questions, spotting missing preconditions, and flagging requirements that cannot be verified as written. The mob is good at *resolving* it, bringing the domain, safety, and supplier knowledge that the AI does not have. The combination is the point: generation surfaces the gaps fast, and human judgment closes them with authority.

For our feature, an ambiguity left unresolved (say, “charge to full by morning” without defining *full*, *morning*, the no-connectivity fallback, or who owns the thermal limit) would surface as defects across several domains during Construction. Resolved in Inception, each becomes a clean, testable requirement with an owner. A practical test of readiness: every requirement is uniquely identified, individually verifiable, classified for safety and timing, and free of words like “fast,” “soon,” or “safe” that have no agreed meaning.

5.5 Traceability from intent to requirement

Because every Inception step emits artifacts, traceability is a by-product rather than a separate effort. The intent record links to the elaboration decisions; each decision links to the requirements it produced; each requirement carries its source and its verification method. The result is a chain that already points in both directions, from intent down to requirement, and from requirement back up to the reason it exists.

This is exactly the spine the automotive standards demand. The bidirectional links seed the ISO 26262 traceability from hazard through requirement to verification; the recorded decisions and rationale feed ASPICE work product traceability; and because the feature is OTA-delivered, the requirement set becomes the baseline that R156 expects every deployed software version to trace to. None of this is bolted on at the end; it accumulates as the feature moves through Inception.

Inception artifact	Links to	Seeds (standard)
Captured intent	Elaboration decisions	Rationale for the safety case
Elaboration decisions	Requirements + constraints	ASPICE work product traceability
Requirements (CS-FR/NFR/SR)	Design, code, tests (later phases)	ISO 26262 bidirectional traceability
Safety constraints (CS-SR-*)	BMS interface, verification	Hazard-to-requirement links
Requirement baseline	OTA package (later)	UNECE R156 version traceability

Carried forward, this chain lets later chapters answer questions regulators and auditors actually ask: *why does this code exist, which requirement does it satisfy, how was it verified, and which*

deployed vehicles are affected? Chapter 6 takes the approved requirements and turns them into user stories and functional design, extending the same chain into the units of work that Construction will build.

Key takeaways

- Inception starts with **intent**, not requirements: the bootstrap captures the what and why and frames open questions, using SDV-specific prompts (ECUs, signals, safety relevance, OTA).
- **Mob Elaboration** brings a broad cross-functional team, including safety and supplier voices, to resolve those questions into agreed requirements and explicit constraints.
- SDV requirements are **structured and typed**: uniquely identified, verifiable, and tagged with ASIL and timing; safety constraints bound what generated code may do rather than describing features to build.
- Removing ambiguity before design is the goal because, in a vehicle, late gaps are not just costly but potentially unsafe and ripple across domains.
- Inception's documentation-first artifacts **seed traceability** automatically, giving ISO 26262, ASPICE, and UNECE R156 the evidence spine they require.

What's next

Chapter 6 turns approved requirements into user stories and functional design.

Chapter 6: User Stories and Functional Design for Vehicle Software

After this chapter you can take approved SDV requirements through AI-DLC’s user-story and functional-design steps, decomposing a feature into units of work the Construction phase can build.

6.1 From requirements to user stories for in-vehicle and fleet features

The approved requirements from Chapter 5 say *what* the feature must do and *how safely*. User stories restate that work from the point of view of whoever benefits, which keeps the team building for an actor rather than for a specification. In an SDV, the actors are unusually varied: a driver, the vehicle itself, a background service, and a fleet operator can all be the “user” of one feature.

For EV charge scheduling, the same feature produces stories at three different altitudes:

- **Driver-facing.** *As a driver, I want to set a departure time from my app so that my car is charged and warm when I leave.* This story traces to the functional requirement that the battery reach its target SoC by departure (CS-FR-014).
- **Vehicle-internal.** *As the charge-management service, I need to execute the most recent schedule even with no connectivity so that charging is reliable off-grid.* This traces to CS-FR-021 and inherits the safety constraint that the BMS limits are authoritative (CS-SR-002).
- **Backend/fleet.** *As a fleet operator, I want vehicles to charge during off-peak windows so that energy cost across the fleet is minimized.* This traces to the cost-optimization intent and the OTA delivery of the in-vehicle component.

Writing the actor explicitly matters in an SDV because it exposes which stories cross the safety boundary. The driver-facing story is comfort and convenience; the vehicle-internal story touches the BMS interface and therefore carries inherited safety constraints into design. Each story keeps a link back to its source requirement so the traceability chain from Chapter 5 extends rather than restarts.

Good SDV stories also carry **acceptance criteria** in testable form. For the driver story: the car reaches target SoC no later than departure when plugged in and grid power is available; if it cannot (short plug-in time, grid fault), the driver is told why. Acceptance criteria are where timing and degraded-mode behavior become concrete enough for the test engineer to design against.

The user-stories workflow emits `stories.md`[†] (INVEST-structured) alongside `personas.md`. One story for our feature:

```
# User Stories - EV Charge Scheduling
↪ (aidlc-docs/inception/user-stories/stories.md)

## US-03 - Departure-time charging (driver)
As a driver, I want to set a departure time from my app so that my car is
charged to my target SoC and pre-conditioned when I leave.

Acceptance criteria:
- Given the car is plugged in with grid power, when a departure time is set,
  then the battery reaches the target SoC no later than that time.
- Given the target cannot be met (short plug-in time / grid fault),
  then the driver is notified with the reason.

Traces to: CS-FR-014, CS-NFR-007 | Persona: Commuter EV owner
```

Before these stories are generated, the User Stories step runs a short planning pass that, like elaboration, is driven by an editable questionnaire (`story-planning-questions.md`). It asks which actors deserve first-class stories, whether acceptance criteria must cover degraded modes, and how stories trace back to requirements:

```
# Story Planning Questions - EV Charge Scheduling
# (aidlc-docs/inception/user-stories/story-planning-questions.md)

## Question 1
Which actors should get first-class user stories for this feature?
A) Driver, the vehicle (autonomous actor), and the fleet operator
B) Driver only
C) Driver and fleet operator only
D) Other (please describe after [Answer]: tag below)
[Answer]: A

## Question 2
Should acceptance criteria include degraded-mode behavior?
A) Yes - notify the driver with a reason when the target cannot be met
B) No - only the happy path
C) Other (please describe after [Answer]: tag below)
[Answer]: A
```

Answering Q1 = A is what produced stories at all three altitudes above; Q2 = A is why US-03 carries its “tell the driver why” criterion. The team can add a question (say, a fleet-operator reporting need) or remove one that does not apply, and the plan regenerates from the edited answers.

The companion `personas.md` names who each altitude serves, including the one that is unusual outside automotive, the vehicle as an autonomous actor:

```
# Personas - EV Charge Scheduling (aidlc-docs/inception/user-stories/personas.md)

Commuter EV owner    goal: charged, warm car by departure at lowest cost
↪ (US-03, US-15)
```

[†]Story identifiers take the form US-NUMBER (user story). Each is a stable handle linked to the requirements it satisfies and, later, to the units of work and tests that implement and verify it.

```
Vehicle (autonomous)  goal: execute the plan safely, with or without a link
↳ (US-11)
Fleet operations mgr  goal: minimize aggregate energy cost across vehicles
↳ (US-07)
```

Naming the vehicle as a persona is what forces the offline, degraded-mode, and safety-inheriting behavior into the stories, rather than leaving it implicit behind the driver's convenience.

6.2 Technology-agnostic functional design

Functional design in AI-DLC captures *behavior* before *technology*. It answers “what does the system do, in what order, under what conditions” without yet deciding whether a component runs on AUTOSAR Adaptive, a particular system-on-chip, or a cloud service. Holding off on those commitments keeps the design reviewable by the whole mob, safety and systems engineers included, and avoids baking in platform choices the team has not yet made.

For charge scheduling, the technology-agnostic functional flow looks like this:

1. The driver submits a target SoC and departure time.
2. The scheduler computes a charging plan from current SoC, tariff windows, and estimated charge time, leaving margin for pre-conditioning.
3. The plan is delivered to the vehicle and cached locally.
4. At the planned start, the charge-management service requests charging within the BMS-permitted envelope.
5. The service monitors progress and adjusts, but never overrides a BMS inhibit.
6. Status (on track, delayed, complete, fault) flows back to the driver.

Notice what this design does *not* say. It does not name a messaging protocol, a cloud provider, or an in-vehicle operating system. It does not decide whether the scheduler runs in the cloud or partly on the vehicle. Those are Construction decisions (Chapter 7). What it does fix is the behavioral contract, including the non-negotiable one: step 5 encodes the safety constraint as a behavior, so that no later technology choice can quietly discard it.

This separation is especially valuable in automotive, where the same functional behavior may be realized very differently across vehicle lines (different ECUs, different suppliers). A technology-agnostic functional design is reusable across those variants; a design fused to one platform is not.

Functional design also fixes the vocabulary the feature reasons about, still without naming a technology. For the charge-management service it records the domain entities and the invariants that bind them:

```
# Domain Entities - charge-management-service
↳ (functional-design/domain-entities.md)

ChargePlan      target_soc_pct, departure_time, charge_windows[], source
↳ (APP|RECURRING|DEMAND_RESPONSE)
BmsEnvelope     max_charge_current_a, inhibit (authoritative), valid (false on
↳ interface loss)
ChargeSession   state: IDLE | REQUESTING | CHARGING | COMPLETE | SAFE_IDLE | FAULT

Invariant: a session may be CHARGING only while inhibit == false and valid == true.
Invariant: cost optimization may reorder charge_windows but never violates the
          BMS envelope or the departure deadline.
```

Writing the invariants at design time is what makes the safety constraint testable later: the

CHARGING-only-when-not-inhibited rule becomes an assertion in the generated tests (Chapter 8), and the `SAFE_IDLE` state gives the degraded-mode behavior a name the test engineer can target rather than a vague expectation.

6.3 Decomposing vehicle features into units of work

A feature the size of charge scheduling is too large for a single pass of Construction. AI-DLC decomposes it into **units of work**, coherent pieces that can be designed, built, and verified largely independently, each mapping to a service or component. Decomposition is where the cloud/in-vehicle split and the safety boundary become structural.

For our feature, a natural decomposition:

Unit	Where it runs	Traces to	Safety posture
Schedule API & app integration	Cloud	Driver story, CS-SEC-005	QM; security-sensitive
Tariff-optimization & planning	Cloud	Cost intent, CS-FR-014	QM
In-vehicle charge-management service	Vehicle	CS-FR-014, CS-FR-021	QM, but interfaces with BMS
BMS interface adapter	Vehicle	CS-SR-002, CS-SR-003	Safety-relevant boundary
OTA packaging & delivery	Cloud + vehicle	OTA intent, R156 baseline	QM; release-governed
Status & telemetry feedback	Vehicle → Cloud	Acceptance criteria	QM; privacy-sensitive

Two decomposition principles earn their keep here. First, **isolate the safety boundary in its own unit**. The BMS interface adapter is split out precisely so the safety-relevant code is small, reviewable, and held to a higher verification bar (HIL testing), while the rest of the feature proceeds at QM pace. Second, **align units with verification**. A unit that can be fully exercised by SIL tests and mocks (the cloud planner) is a strong candidate for aggressive AI-DLC generation; a unit gated on physical hardware (the BMS adapter) is not.

Each unit becomes a candidate Bolt: a focused pass that carries that unit from its (already approved) requirements through design, code, and verification. Sequencing matters: the BMS interface contract should stabilize before the in-vehicle service that depends on it.

Units Generation records this as `unit-of-work.md`[‡], plus a dependency matrix and a story map:

```
# Units of Work - EV Charge Scheduling
↪ (aidlc-docs/inception/application-design/unit-of-work.md)

U1 cloud-schedule-api      stories: US-03      deploy: cloud service
U2 tariff-planner          stories: US-07      deploy: cloud service
U3 charge-management-svc   stories: US-03, US-11 deploy: in-vehicle (AUTOSAR
↪ Adaptive)
U4 bms-interface-adapter   stories: US-11      safety-relevant boundary
U5 ota-delivery            R156 baseline      deploy: cloud + vehicle
U6 telemetry-feedback      privacy-sensitive   vehicle -> cloud
```

[‡]Unit identifiers U1 through U6 label the units of work the feature decomposes into; each becomes a candidate Bolt and maps to the stories it delivers. U4, the BMS interface adapter, is the one safety-relevant unit.

Dependencies: U3 -> U4 (stabilize first) ; U1 -> U2 ; U6 -> U1

The figure traces the feature across the three altitudes, from persona to story to unit, with the safety boundary visible on the vehicle-internal lane.

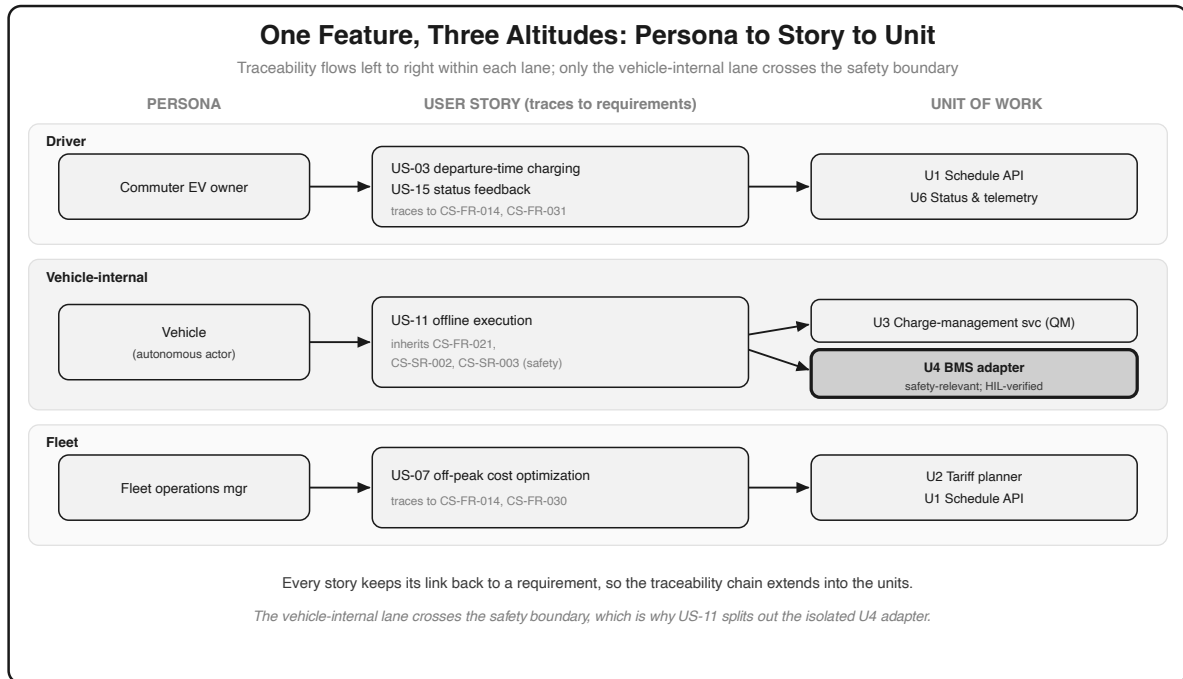


Figure 4: A three-lane traceability map for EV charge scheduling, one lane per story altitude, with arrows flowing left to right within each lane and never crossing between lanes. The top lane, Driver, runs from the persona Commuter EV owner to its stories US-03 departure-time charging and US-15 status feedback (tracing to CS-FR-014 and CS-FR-031) to the units U1 schedule API and U6 status and telemetry. The middle lane, Vehicle-internal, runs from the persona Vehicle as autonomous actor to the story US-11 offline execution (inheriting CS-FR-021, CS-SR-002, and CS-SR-003) to two units: U3 charge-management service in light QM shading and U4 BMS interface adapter in dark, thick-bordered safety shading. The bottom lane, Fleet, runs from the persona Fleet operations manager to the story US-07 off-peak cost optimization (CS-FR-014, CS-FR-030) to the units U2 tariff planner and U1 schedule API. A footer notes that only the vehicle-internal lane crosses the safety boundary, which is why US-11 decomposes into an isolated safety-relevant unit, U4.

6.4 A worked thread: one story from altitude to units

It is worth following a single story all the way through, because the vehicle-internal altitude is where an SDV feature differs most from ordinary software. Take US-11: *as the charge-management service, I need to execute the most recent schedule even with no connectivity.*

Its acceptance criteria already carry three behaviors: run the cached plan with no cloud dependency, fall back to an immediate charge within the BMS-permitted envelope when no plan exists, and stop immediately on a BMS inhibit, never overriding it. That last criterion is inherited, not invented: the story traces to CS-FR-021 for the offline behavior and to CS-SR-002 and CS-SR-003 for the safety constraints it must honor.

Because the story crosses the safety boundary, it decomposes into two units, not one. The QM behavior (caching, plan execution, status reporting) becomes the in-vehicle charge-management

service (U3); everything that actually touches the battery is pushed into the BMS interface adapter (U4), the single safety-relevant unit. The story's inherited **CS-SR-*** constraints attach to U4, which is held to the higher verification bar (hardware-in-the-loop), while U3 proceeds at QM pace. One driver-invisible story, followed honestly, is what forces the safety boundary to appear in the decomposition; a team that wrote only the driver's convenience story would have missed it.

6.5 Anti-patterns in stories and functional design

A few habits quietly undermine this step, and naming them helps the mob catch them before the gate:

- **Solution-shaped stories.** Writing “as a driver, I want a message sent to the vehicle over a particular protocol” bakes a technology choice into a story. Keep stories about the actor's goal; the protocol is a Construction decision.
- **Dropping the degraded modes.** Stories that cover only the happy path leave offline behavior, faults, and safe states undefined, which is exactly where SDV features fail. Require degraded-mode acceptance criteria (the US-03 “tell the driver why” and the US-11 fallback).
- **Platform-fused functional design.** Deciding the operating system, middleware, or cloud provider inside functional design forfeits reuse across vehicle lines and hides the behavior behind plumbing. Keep behavior and technology separate.
- **A shared safety unit.** Letting several units each call the BMS spreads the safety surface. Isolate it in one adapter unit (U4) so the higher review bar applies to the smallest possible amount of code.
- **Stories without traceability.** A story that does not link back to a requirement cannot be verified or defended later. Every story cites its **CS-*** IDs.

6.6 Approval gates: what the team signs off in Inception

Inception ends at a gate, not a deadline. Before any Construction begins, the mob signs off on a defined set of artifacts. Until that approval, the feature does not advance; this is the human-decision discipline from Chapter 3 made concrete.

What the team approves to close Inception for charge scheduling:

- **Requirements:** the structured set is complete, each item verifiable and classified for safety and timing; the safety constraints are agreed and owned.
- **User stories with acceptance criteria:** every story traces to a requirement and has testable criteria, including degraded-mode behavior.
- **Functional design:** the technology-agnostic behavior is agreed, with the safety constraint expressed as a behavior, not an aspiration.
- **Unit decomposition:** the split into units is agreed, the safety boundary is isolated, and unit sequencing is set.
- **Open items:** anything unresolved is explicitly logged with an owner, not silently carried into design.

Specific sign-offs carry specific accountability. The functional safety engineer approves the safety classification and the boundary; the systems and supplier representatives approve the BMS interface assumptions; the product owner approves scope; the test engineer confirms each requirement and story is verifiable. These named approvals are not bureaucracy; they are the first entries of the human-accountable evidence that automotive standards such as ISO 26262 and ASPICE require, captured at the moment the decisions are actually made.

With the gate passed, the feature carries a complete, traceable Inception record into Construction, exactly the input Part III assumes.

Key takeaways

- User stories restate approved requirements from an actor's view; in an SDV the actors span driver, vehicle, background service, and fleet, and each story keeps its link back to a requirement.
- Functional design captures **behavior before technology**, keeping the design reviewable and reusable across vehicle variants, and encoding the safety constraint as a behavior no platform choice can discard.
- Decomposition into **units of work** makes the cloud/in-vehicle split and the safety boundary structural; isolate the safety-relevant unit and align units with how they will be verified.
- Following a vehicle-internal story to its units reveals the safety boundary: a story that crosses it (US-11) decomposes into a QM unit and an isolated safety-relevant unit, with the inherited CS-SR constraints attached to the latter.
- Watch for the recurring anti-patterns: solution-shaped stories, dropped degraded modes, platform-fused functional design, a shared safety unit, and stories without traceability.
- Inception closes at an **approval gate** with named, accountable sign-offs, the first human-accountable evidence the compliance trail needs.

What's next

Part III enters Construction, beginning with architecture and infrastructure design.

Chapter 7: Application and Infrastructure Design via Mob Construction

After this chapter you can run AI-DLC’s design steps for an SDV feature, using Mob Construction to let AI propose in-vehicle, edge, and cloud architecture while the team decides.

7.1 Architecture design for SDV: in-vehicle, edge, and cloud

Construction opens where Inception left off: with an approved set of requirements, user stories, and a unit decomposition. The first design step is architecture, deciding how the units from Chapter 6 are realized as components and where each one runs. In an SDV this is inherently a three-tier question, because a single feature lives across the vehicle, the edge, and the cloud at once.

For EV charge scheduling, the units map onto the tiers like this:

Tier	Components (from Chapter 6 units)	Responsibility
Cloud	Schedule API & app integration; tariff-optimization & planning; OTA packaging; telemetry ingestion	Driver-facing API, planning, fleet-wide data, software delivery
Edge (optional)	Tariff/grid-signal aggregation	Regional demand-response and tariff feeds, where latency or scale warrants it
In-vehicle	Charge-management service; BMS interface adapter; status feedback	Executing the plan within safe limits; reporting state

In Mob Construction (Section 7.4) the AI proposes this mapping along with options and trade-offs; the team decides. Two architectural decisions dominate the rest of the feature:

- **Where the scheduling intelligence lives.** Option A keeps planning in the cloud and ships only a resolved plan to the vehicle; Option B pushes more logic on-vehicle for autonomy. The team’s earlier requirement that the vehicle execute the most recent cached plan with no connectivity (CS-FR-021) tips the balance: planning stays in the cloud, but the resolved plan and a safe local fallback live on the vehicle.
- **How the tiers communicate.** Cloud-to-vehicle commands need authentication and integrity (CS-SEC-005), and the in-vehicle path to the battery must honor the safety boundary. These constraints, set in Inception, are now architectural inputs rather than afterthoughts.

The output of this step is a component architecture with the safety boundary drawn explicitly: everything from the app to the in-vehicle charge-management service is QM, and the line to the battery is mediated by the BMS interface adapter. The figure shows that three-tier architecture with the boundary marked.

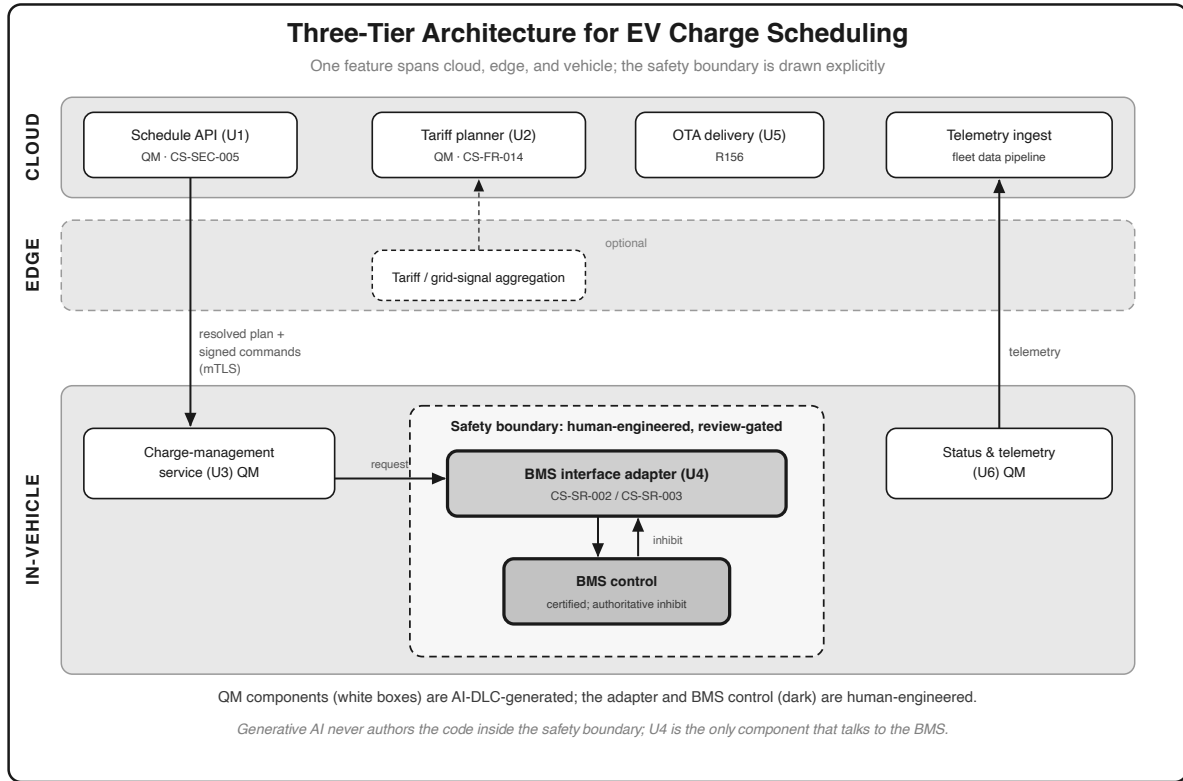


Figure 5: Three-tier component architecture for EV charge scheduling, drawn as three stacked, labeled horizontal layer bands. The top band, CLOUD, holds the schedule API and app integration (U1), the tariff-optimization and planning service (U2), OTA packaging and delivery (U5), and telemetry ingestion. The middle band, EDGE, is optional and holds tariff and grid-signal aggregation. The bottom band, IN-VEHICLE, holds the charge-management service (U3) and the status and telemetry feedback (U6), both marked QM as white boxes, and nested within it a dashed box labeled the safety boundary that contains a darker, thick-bordered BMS interface adapter (U4) and the certified BMS control. Arrows traverse the layers: a resolved plan and signed commands flow cloud to vehicle over mutual TLS (CS-SEC-005), telemetry flows vehicle to cloud, and the charge-management service reaches the battery only through the BMS interface adapter, which forwards an authoritative inhibit. A footer states that generative AI never authors the code inside the safety boundary and that U4 is the only component that talks to the BMS.

7.2 Designing within AUTOSAR Adaptive and service-oriented middleware

The in-vehicle components cannot be designed in a vacuum; they must fit the platform that will run them. For a modern high-performance controller that means AUTOSAR Adaptive and service-oriented middleware, and the design AI proposes has to be constrained to those realities, or it will generate plausible code that cannot run on the target.

For the charge-management service, the platform constraints shape the design:

- **Service interfaces.** The service is modeled as an AUTOSAR Adaptive application communicating over `ara::com`, with interfaces expressed in a formal interface definition so they can be generated and verified rather than hand-described. Service discovery and event/method semantics follow the middleware’s model (for example, SOME/IP or a Data Distribution Service [DDS] binding), not a generic cloud-style request/response assumption.
- **The BMS boundary as an interface contract.** The BMS interface adapter is designed as a thin, explicit contract: the charge-management service may *request* charging within an envelope and must *honor* a BMS inhibit immediately (CS-SR-002, CS-SR-003). The adapter is the only component that talks to the safety-relevant BMS, keeping the safety surface small and reviewable.
- **Timing and execution semantics.** The soft-real-time scheduling behavior and the hard requirement to honor an inhibit are expressed as explicit timing properties of the design, so they survive into code generation and test (Chapters 8 and 9) rather than being discovered on the bench.

Making the BMS contract explicit at design time is what keeps the safety surface small. The adapter’s interface is specified, not described in prose, so it can be generated and verified against rather than reinterpreted:

```
# BMS interface contract (design) - bms-interface-adapter (U4)
# (aidlc-docs/construction/.../functional-design/domain-entities.md)

service BmsChargeControl:
  getPermittedEnvelope() -> {maxCurrent, maxVoltage, tempOk} # read
  isInhibited() -> bool # read;
  ↪ authoritative
  event InhibitChanged # pushed on state
  ↪ change
  # the adapter MAY request charging within the envelope
  # it MUST honor an inhibit immediately (hard deadline, CS-SR-002)
  # loss of the interface MUST drive a safe idle state (CS-SR-003)
  # no override path exists
```

The charge-management service (U3) is designed against this contract, and the dependency recorded in Inception (U3 depends on U4) means the contract stabilizes first. The QM service and the safety-relevant adapter can then be built at the same time without the service ever reaching around the boundary to the battery.

This is where the maturity gradient from Chapter 4 becomes concrete. AI design assistance is genuinely useful for the service skeleton, interface stubs, and the QM-level logic, but the design deliberately isolates the safety-relevant adapter so that the higher-rigor work, and the human review it demands, is contained. The team approves the platform-constrained design before any code is written.

Functional Design is per-unit and technology-agnostic; for the charge-management service it writes `business-logic-model.md`, `business-rules.md`, and `domain-entities.md`. The rules that encode the safety boundary:

```
# Business Rules - charge-management-service
# (aidlc-docs/construction/charge-management-service/functional-design/business-rules.md)
↪ les.md)

BR-01 Charging MAY be commanded only within the BMS-permitted envelope.
```

```
BR-02 A BMS inhibit MUST stop charge initiation immediately (hard,
↳ non-overridable).
BR-03 Loss of the BMS interface MUST drive a safe idle state.
BR-04 Safety/charge limits ALWAYS take precedence over cost optimization.
BR-05 With no connectivity, the most recent cached plan executes locally.
```

7.3 Infrastructure design: cloud backend, OTA, data pipelines

The cloud tier is where AI-DLC is at its most effective (Chapter 4's Accelerate band), because the patterns are well-known and verification can be fully automated. The infrastructure design for charge scheduling covers three backends, each provisioned as infrastructure as code so the design is reviewable and reproducible:

- **Service backend.** The schedule API and tariff-optimization planner, exposed through an authenticated API. Cloud-to-vehicle commands are protected with mutual Transport Layer Security (mTLS) and scoped credentials via identity and access management (IAM), satisfying CS-SEC-005. This backend is a strong candidate for end-to-end AI-DLC generation.
- **OTA backend.** The packaging, signing, and staged rollout of the in-vehicle charge-management component. The design carries the R156 baseline forward: every delivered version is traceable to the requirement set it satisfies, and rollback is a first-class capability, not an afterthought.
- **Data pipeline.** Telemetry from the status-and-feedback unit flows back over a lightweight transport (for example, Message Queuing Telemetry Transport, MQTT) into a fleet data pipeline for readiness-accuracy and cost-savings metrics. Because this carries location and usage data flagged as privacy-sensitive in Inception, the design specifies what is collected, minimized, and retained.

Infrastructure Design maps these logical components to real services, writing `infrastructure-design.md` and `deployment-architecture.md` per unit:

```
# Infrastructure Design - cloud-schedule-api
# (aidlc-docs/construction/cloud-schedule-api/infrastructure-design/infrastructure_
↳ -design.md)

Compute:    container service (REST API), autoscaled
Datastore:  managed DB for schedules; object store for OTA packages
Messaging:  MQTT broker for vehicle telemetry -> stream -> fleet data pipeline
Security:   mTLS for cloud-to-vehicle commands; IAM-scoped credentials (CS-SEC-005)
Delivery:   OTA service - signed packages, staged rollout, rollback (R156)
IaC:        Terraform modules; CI validates the plan before apply
```

The companion deployment view records how those services reach the fleet and how a bad release is undone, so rollout and rollback are designed rather than improvised:

```
# Deployment Architecture - cloud-schedule-api
# (aidlc-docs/construction/cloud-schedule-api/infrastructure-design/deployment-arc_
↳ hitecture.md)

Environments: dev -> staging -> production (fleet)
Rollout:      OTA campaign, canary 1% -> phased 25% -> fleet 100%
Rollback:     previous signed version retained on-vehicle; one-step revert
Evidence:     release-record.md per version, per VIN (R156)
Gate:         human release approval (Quality Steward) before each stage
```

A security note worth making explicit: the schedule API is an internet-facing, network-exposed endpoint. Its authentication and authorization are not optional hardening to add later; they are design requirements (CS-SEC-005) that the team confirms now, because an unauthenticated path that can influence vehicle charging is a cybersecurity exposure under the R155 scope.

7.4 Mob Construction: AI proposes, the team decides

Mob Construction is the working mode for all of the above. As in Mob Elaboration, the cross-functional team works *with* the AI rather than reviewing after the fact: the AI proposes architecture and infrastructure options with their trade-offs, and the team steers and decides. The difference from Inception is the subject matter, designs and components rather than requirements, but the human-decision discipline is identical.

The AI presents these options the same way elaboration presented open questions: as an editable, multiple-choice questionnaire (`design-questions.md`), where each answer becomes an architecture decision record. An excerpt:

```
# Design Questions - EV Charge Scheduling (Mob Construction)
# (aidlc-docs/construction/design-questions.md)

## Question 1
Where should the scheduling intelligence live?
A) Plan in the cloud; ship a resolved plan with an on-vehicle cache and safe
  ↪ fallback
B) Push full planning logic onto the vehicle for maximum autonomy
C) Other (please describe after [Answer]: tag below)
[Answer]: A

## Question 2
How should the BMS boundary be structured?
A) A single, isolated BMS interface adapter unit (the only component that talks to
  ↪ the BMS)
B) Inline the BMS calls inside the charge-management service
C) Other (please describe after [Answer]: tag below)
[Answer]: A
```

What makes this more than a conversation is that decisions are captured as artifacts. Each significant choice becomes an architecture decision record (ADR): the options considered, the decision, and the rationale. For charge scheduling the mob records, among others:

- Planning in the cloud with an on-vehicle resolved-plan cache and safe fallback (over full on-vehicle planning). Rationale: the no-connectivity requirement and a smaller in-vehicle safety surface.
- An isolated BMS interface adapter as the sole safety-relevant component. Rationale: containment of the higher-rigor review and verification.
- mTLS + IAM-scoped credentials for cloud-to-vehicle commands. Rationale: CS-SEC-005 and the R155 cybersecurity scope.

These records do double duty. They steer Construction, and they extend the traceability chain from Chapter 5 into design, so that the line from intent to requirement now continues into the architecture that satisfies it, ready for code generation in Chapter 8. The team's sign-off on the design, with the safety boundary and security decisions explicit, closes the design step and gates the move to code.

7.5 A worked design pass: from units to approved architecture

It helps to see the whole design step run once, end to end, for the charge-scheduling feature. The mob starts from the six approved units (U1 to U6) and the requirements they trace to, and works through the design in a fixed order.

First, the **architecture mapping**. The AI proposes the three-tier placement of Section 7.1; the mob confirms that planning stays in the cloud (U1, U2) and execution on the vehicle (U3), driven by the no-connectivity requirement CS-FR-021. That single requirement, set in Inception, decides the shape of the whole feature.

Second, the **boundary**. Because U3 interfaces with the battery, the mob confirms the decision to isolate all safety-relevant behavior in U4, the BMS interface adapter, rather than inline it in U3. This keeps the QM service, which AI-DLC generates freely, cleanly separated from the safety-relevant adapter, which stays human-engineered and review-gated. The interface contract from Section 7.2 is agreed here.

Third, the **platform fit**. The AI constrains U3's design to AUTOSAR Adaptive and `ara::com`, and the mob checks that the timing properties (soft-real-time scheduling and the hard inhibit deadline) are expressed in the design rather than left implicit.

Fourth, the **cloud infrastructure**. U1, U2, and U5 are mapped to concrete services with mTLS and IAM-scoped credentials (CS-SEC-005), and U6's telemetry pipeline is designed with data minimization.

Each of these is a recorded decision, and the sequence ends at the design gate: the mob signs off with the safety boundary and the security posture explicit, and only then does the feature move to code generation. The order matters, because the boundary and the platform constraints are decided before code, so they shape the structure rather than being retrofitted onto it.

7.6 Design anti-patterns to avoid

A few failure modes recur in SDV design, and naming them helps the mob catch them at the gate rather than on the bench:

- **Platform-ignorant design.** Letting the AI propose a generic cloud-style request/response service for an in-vehicle component yields plausible design that cannot run on AUTOSAR Adaptive. Constrain the design to the target platform and its middleware first.
- **A smeared safety boundary.** Inlining BMS calls across the charge-management service spreads safety-relevant logic through QM code, so every later change risks touching safety. Keep U4 the only component that talks to the BMS.
- **Authentication as later hardening.** Treating the schedule API's authentication as something to add before launch, rather than a design requirement (CS-SEC-005), leaves a network-exposed path to vehicle charging. Decide it now.
- **Over-pushing logic on-vehicle.** Moving full planning onto the vehicle for autonomy enlarges the in-vehicle safety surface and the code that must be verified to a higher bar. Keep the vehicle side as small as the requirements allow.
- **Undocumented decisions.** A design agreed only in conversation cannot be traced or defended later. Every significant choice becomes an architecture decision record, or it did not happen.

7.7 How the design maps to the code

The design is not an abstraction that floats away from the implementation. Each unit becomes a directory in the runnable example (the companion repository is linked in the Preface), so the architecture and the code stay in step:

```

U1 cloud-schedule-api      -> code/cloud/schedule-api/
U2 tariff-planner          -> code/cloud/tariff-planner/
U3 charge-management-service -> code/in-vehicle/charge-management-service/
U4 bms-interface-adapter   -> code/in-vehicle/bms-interface-adapter/
  ↪ (safety-relevant)
U5 ota-delivery            -> code/ota/
U6 telemetry-feedback      -> code/telemetry/

```

The safety boundary drawn in this chapter is visible in that layout: U4 is its own directory, human-engineered and review-gated, while everything around it is QM code that AI-DLC generates against the approved design.

Key takeaways

- SDV architecture is a three-tier problem: one feature spans cloud, edge, and in-vehicle, and the unit decomposition from Inception maps directly onto those tiers.
- In-vehicle design must be constrained to platform realities (AUTOSAR Adaptive, service-oriented middleware), and the safety-relevant boundary should be isolated in a small, explicit interface contract.
- The cloud tier is where AI-DLC accelerates most; infrastructure as code keeps the design reviewable, and network-exposed endpoints carry authentication and authorization as design requirements, not later hardening.
- Mob Construction keeps humans deciding while AI proposes, and records choices as architecture decision records that extend the traceability chain into design.
- A worked design pass runs in order: map units to tiers, isolate the safety boundary, fit the platform, then design the cloud infrastructure, ending at a human design gate before any code is written.
- Watch for the recurring anti-patterns: platform-ignorant design, a smeared safety boundary, authentication as later hardening, over-pushing logic on-vehicle, and undocumented decisions.

What's next

Chapter 8 moves from design to deterministic code generation.

Chapter 8: Code Generation for SDV

After this chapter you can apply AI-DLC’s deterministic code generation to SDV software, embedded, middleware, and backend, with verification, tests, and a path for existing code.

8.1 Deterministic code generation with verification

Code generation in AI-DLC is not a single prompt that emits a service. It is a planned, step-by-step pass in which AI generates code and tests one layer at a time, a deterministic process checker confirms each step’s outputs before the next begins, and a build-and-test gate must pass before the Bolt advances. That discipline matters everywhere, but it matters most in automotive, where the cost of a defect is measured in recalls, not hotfixes.

The word *deterministic* needs care, because the generator itself is not. Ask a language model to write the same service twice and you get two phrasings. What AI-DLC makes deterministic is the *process wrapped around* that generation: the plan is fixed and approved before generation starts, each step has defined outputs the checker verifies, and nothing advances until the gate is green. So while any single generation is variable, the pipeline’s behavior is repeatable: the same plan produces the same sequence of verified steps, and the same failing check always halts the same way. This is the reconciliation the book returns to in Chapter 12: you do not need a deterministic generator if you have a deterministic process that only lets verified output through.

The loop is the same regardless of what is being built.

For EV charge scheduling, this means the cloud schedule API, the in-vehicle charge-management service, and the BMS interface adapter each move through the same gated loop, but, as the next section shows, with very different amounts of AI generation and human review.

8.2 Generating embedded, middleware, and backend code

The same workflow targets all three tiers of the feature, yet the maturity gradient from Chapter 4 reasserts itself: generation confidence is highest in the cloud and falls toward the safety-critical edge, while verification difficulty and review rigor rise in the opposite direction.

Code generation begins from a per-unit plan the team approves. For the in-vehicle charge-management service, the workflow writes a numbered, story-traced plan:

```
# Code-Generation Plan - charge-management-service
# (aidlc-docs/construction/plans/charge-management-service-code-generation-plan.md)
```

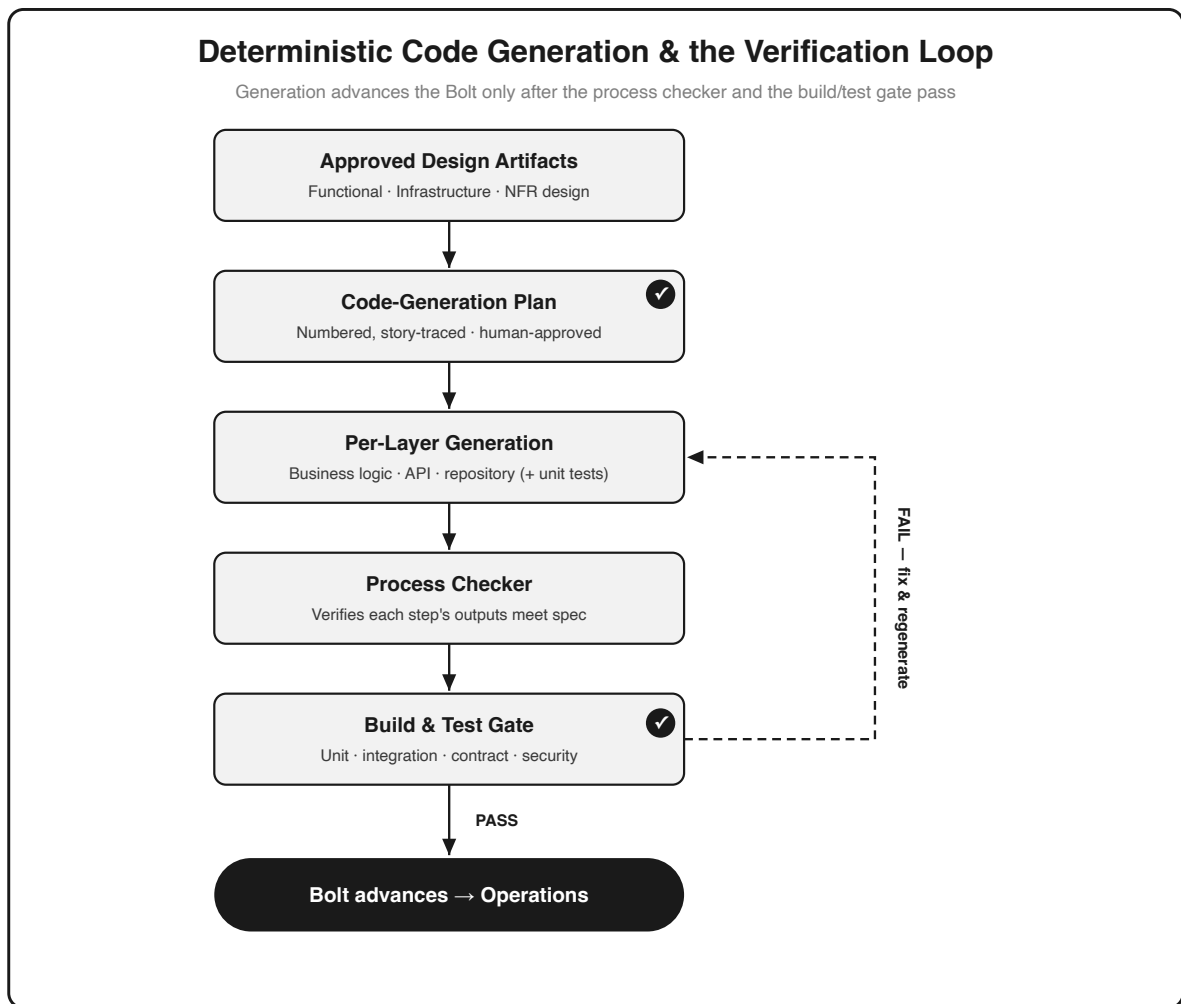


Figure 6: Vertical flow showing code generation advancing only when checks pass: approved design artifacts feed a human-approved code-generation plan, then per-layer generation with unit tests, a process checker that verifies each step's outputs, and a build-and-test gate (unit, integration, contract, security); on success the Bolt advances to Operations, and on failure a dashed loop returns to per-layer generation to fix and regenerate.

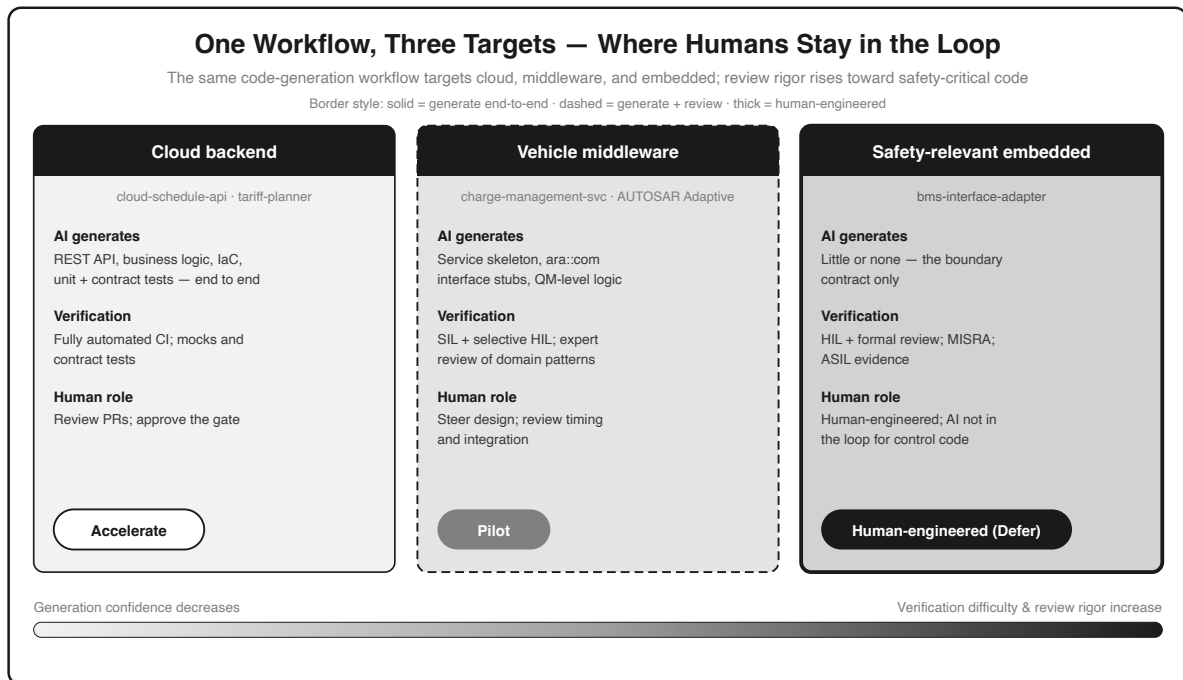


Figure 7: Three side-by-side cards for the same workflow’s targets, distinguished by shade and border style rather than color. Cloud backend (lightest fill, solid border, Accelerate): AI generates REST API, business logic, IaC, and tests end to end, verified by fully automated CI, humans review pull requests. Vehicle middleware (mid-gray fill, dashed border, Pilot): AI generates service skeleton, ara::com stubs, and QM-level logic, verified by SIL plus selective HIL with expert review. Safety-relevant embedded (darkest fill, thick border, Human-engineered/Defer): AI generates little or none beyond the boundary contract, verified by HIL and formal review with MISRA and ASIL evidence, kept out of the control loop. A grayscale gradient bar shows generation confidence decreasing and review rigor increasing left to right.

```

Target: in-vehicle/charge-management-service/    (AUTOSAR Adaptive app)
Stories: US-03, US-11    Depends on: bms-interface-adapter (U4)

[x] Step 1  Business logic - schedule execution & SoC tracking
[x] Step 2  Business logic unit tests
[x] Step 3  ara::com service interface (from IDL)
[ ] Step 4  BMS adapter client (honor inhibit; safe idle)    <- review-gated
[ ] Step 5  Telemetry/status publisher
[ ] Step 6  Deployment artifacts + manifest

```

Application code lands in the workspace, never in the documentation tree; the workflow keeps a markdown summary alongside it (the generated source for every unit is in the companion repository, linked in the Preface):

```

# Code Generation Complete - charge-management-service
Created:
- in-vehicle/charge-management-service/src/schedule_executor.cpp
- in-vehicle/charge-management-service/src/soc_tracker.cpp
- in-vehicle/charge-management-service/src/bms_client.cpp    (boundary;
  ↪ review-gated)
- in-vehicle/charge-management-service/tests/test_schedule_executor.cpp
Docs:
- aidlcc-docs/construction/charge-management-service/code/    (summaries)

```

Notice `bms_client.cpp` is flagged review-gated. The workflow can scaffold the adapter's structure against the agreed interface contract, but the logic that honors a BMS inhibit is safety-relevant and is human-engineered and reviewed, the red column of the figure, not the green one.

The three tiers are worth contrasting directly, because they show the same workflow behaving very differently. The **cloud schedule API** is generated end to end (request handling, validation, business logic, infrastructure-as-code, and tests) and validated by a fully automated continuous-integration pipeline; a reviewer reads a pull request rather than authoring the code. The **in-vehicle charge-management service** is generated for its QM-rated parts (the scheduling and state-tracking logic, the `ara::com` service skeleton) but its output is verified by software-in-the-loop (SIL) tests plus selective hardware-in-the-loop (HIL) runs, and an engineer reviews anything that touches timing or the battery interface. The **BMS boundary** is barely generated at all: the workflow may produce the interface stub from the contract, but the inhibit-handling logic is written and owned by engineers and proven on hardware. One plan, one loop, three sharply different balances of generation and review, dictated by where each unit sits on the maturity gradient, not by convenience.

8.3 Working with C/C++, config-heavy toolchains, and existing code

Little SDV work is greenfield. The charge-management service must integrate with existing vehicle software, configuration-heavy AUTOSAR toolchains, and hand-written C/C++ that predates the feature. AI-DLC handles this through its reverse-engineering step: before changing existing code, the workflow builds an understanding of it (an architecture summary, a component inventory, and a technology-stack note) and uses that context during generation.

Two automotive realities shape this step. First, **modify in place, never duplicate**: the workflow edits existing files rather than emitting `module_new.cpp` beside `module.cpp`, which

keeps the codebase and its traceability intact. Second, **configuration is code**: much of an AUTOSAR integration lives in ARXML and toolchain configuration rather than in `.cpp` files, and the design must respect that the generated service is one participant in a larger, vendor-shaped build. Where context windows cannot hold a very large legacy codebase, the work is scoped to the unit and its interfaces rather than attempted wholesale, the honest boundary from Chapter 4.

8.4 Unit and test generation

Tests are generated alongside code, not after it. Each layer’s generation step is paired with a test step in the plan (Steps 1–2 above), so the unit arrives with its unit tests already written. This is where AI-DLC pays off even on work that is otherwise hard to accelerate: test generation is high-value across every tier (Chapter 4), so even the middleware and boundary units benefit from generated unit, integration, and contract tests even when their production code is human-written.

For charge scheduling the generated tests target the requirements directly: that the service reaches target SoC by departure (CS-FR-014), executes a cached plan with no connectivity (CS-FR-021), and, critically, that it never commands charging the BMS has inhibited (CS-SR-002). The safety-constraint tests are written and reviewed by engineers; the AI assists but does not own them.

Generating tests with the code has a subtler benefit than speed: it forces the requirement to be stated precisely enough to test. A vague requirement, “charge efficiently”, cannot be turned into an assertion; a precise one, “reach the target state of charge by the set departure time in at least 95% of sessions”, becomes a concrete check. In this way the test-generation step doubles as a requirements-quality check, surfacing ambiguity that survived Inception before it reaches the field. And because each test names the requirement it exercises, the test suite itself becomes part of the traceability chain rather than a separate artifact a team must maintain by hand.

8.5 Build, test, and the verification loop

The Bolt does not advance on generated code alone; it advances on a passing build-and-test gate. The build-and-test step compiles all units and runs the testing strategy (unit, integration, contract, and security tests) then records a summary that becomes part of the evidence trail:

```
# Build and Test Summary
# (aidlc-docs/construction/build-and-test/build-and-test-summary.md)

Build: Success
Unit tests:      142 passed / 0 failed    coverage 88%
Integration tests: 18 passed / 0 failed    (schedule-api <-> charge-mgmt)
Contract tests:   6 passed / 0 failed     (ara::com, REST)
Security tests:   mTLS + auth verified; no high/critical findings
BMS boundary (HIL): inhibit honored; safe-idle on link loss [human-reviewed]
Ready for Operations: Yes
```

If any check fails, the loop in Figure 7.1 routes back to generation: fix and regenerate, then re-run the gate. Only a green summary, including the human-reviewed BMS boundary result on hardware, lets the feature proceed. The result carries forward to Operations (Chapter 10) as both working software and the verification evidence the compliance trail needs.

8.6 Traceability that survives into the code

The point of generating code inside a documentation-first workflow is that the line from intent to implementation does not break at the source-file boundary. The requirement IDs that appeared in Inception (Chapter 5) and design (Chapter 7) carry into the generated code and its tests as first-class references, so the chain is navigable in both directions, from a requirement down to the code and tests that satisfy it, and from a line of code back to the requirement that justifies it.

Concretely, the generated units annotate their behavior with the requirements they realize, and the tests name the same IDs in their assertions:

```
// schedule_executor.cpp (generated, QM)
// Realizes: CS-FR-014 (reach target SoC by departure),
//           CS-FR-021 (execute cached plan with no connectivity)
// Boundary: all charge commands pass through bms_client (CS-SR-002)

// test_schedule_executor.cpp (generated)
TEST(ScheduleExecutor, ReachesTargetSocByDeparture) // CS-FR-014
TEST(ScheduleExecutor, ExecutesCachedPlanOffline) // CS-FR-021
TEST(ScheduleExecutor, NeverChargesWhenBmsInhibited) // CS-SR-002
↪ [human-reviewed]
```

This is what makes the compliance argument of Part IV mechanical rather than archaeological. When an assessor asks “show me that requirement CS-SR-002 is implemented and verified,” the answer is a direct link, to the boundary code that enforces it and the test that proves it, not a manual reconstruction after the fact. The same annotations let change-impact analysis work: touch the requirement, and the code and tests that reference it are immediately identifiable. Generation speed is the visible benefit of this step; the durable one is that the code ships already wired into the evidence chain the vehicle’s regulators will ask to see.

Key takeaways

- AI-DLC code generation is a gated loop: per-layer generation, a deterministic process checker, and a build/test gate that must pass before the Bolt advances.
- One workflow serves all three SDV tiers, but generation confidence falls and review rigor rises from cloud to middleware to safety-relevant embedded code.
- Existing code is reverse-engineered first and modified in place; AUTOSAR configuration is treated as code, and oversized legacy migrations are scoped, not attempted wholesale.
- Tests are generated alongside code and trace to requirements; safety-constraint tests stay human-owned, and only a passing gate advances the feature.
- Requirement IDs carry into the generated code and tests, so traceability survives into the source and the compliance argument becomes a direct link rather than an after-the-fact reconstruction.

What’s next

Chapter 9 treats non-functional requirements, safety, timing, security, as first-class.

Chapter 9: Non-Functional Requirements as First-Class Citizens

After this chapter you can use AI-DLC's NFR assessment and design steps to make SDV-critical constraints, safety, real-time, reliability, security, explicit inputs rather than afterthoughts.

9.1 NFR assessment for SDV: safety, real-time, reliability, security

In most software, non-functional requirements (NFRs) are quality attributes you tune after the feature works. In an SDV they often *are* the feature's hardest part: a charging feature that ignores battery safety, misses a timing deadline, or ships an unauthenticated command is not a lower-quality feature; it is an unacceptable one. AI-DLC treats NFRs as a first-class step, assessing them per unit after functional design and before code, so the constraints shape the design rather than being retrofitted onto it.

The distinction is not academic. NFRs like timing determinism, freedom from interference, and fail-safe behavior are *architectural* properties; they are decided by how components are separated, how control flows, and where boundaries sit. You cannot sprinkle them on after the code works, because by then the structure that would have carried them is already set. A service written without a hard-real-time inhibit path in mind cannot be made hard-real-time by a later tweak; a design that lets a QM component reach the battery directly cannot be made interference-free without being re-drawn. Treating NFRs as an afterthought in an SDV is therefore not a matter of polish deferred; it is a matter of building the wrong structure and discovering it on the test bench, when the fix is a redesign. Placing the NFR step *before* code is how AI-DLC ensures the constraints are inputs to the structure rather than complaints about it.

For the charge-management service, the NFR assessment surfaces the four constraint families that dominate vehicle software. The workflow records them as a documentation-first artifact:

```
# NFR Requirements - charge-management-service
# (aidlc-docs/construction/charge-management-service/nfr-requirements/nfr-requirements.md)

Safety:      BMS limits authoritative; freedom from interference (CS-SR-002/003)
Real-time:   soft RT for readiness-by-departure; HARD inhibit response (CS-FR-014)
Reliability: operate with no connectivity; safe degraded modes (CS-FR-021)
Security:    authenticated, integrity-protected commands; R155 scope (CS-SEC-005)
Tech stack:  AUTOSAR Adaptive (C++), ara::com; cloud planner separate unit
```

The assessment is driven by the same editable questionnaire mechanism. For this unit the

workflow writes `nfr-questions.md`, typing each constraint family explicitly before it binds the design:

```
# NFR Assessment Questions - charge-management-service
# (aidlc-docs/construction/charge-management-service/nfr-requirements/nfr-questions.md)

## Question 1
What timing class applies to honoring a BMS inhibit?
A) Hard real-time, non-overridable, within a worst-case execution time budget
B) Soft real-time (best effort)
C) Other (please describe after [Answer]: tag below)
[Answer]: A

## Question 2
What reliability behavior is required when connectivity is lost?
A) Execute the cached plan with bounded retries; safe idle on BMS interface loss
B) Require connectivity to operate
C) Other (please describe after [Answer]: tag below)
[Answer]: A
```

A unit with no security surface could simply delete the security question; here every family applies, so none is removed. Q1 = A is what makes the inhibit path hard real-time in the `safety-monitor`, and Q2 = A is what puts the cached plan fallback and safe-idle behavior into the service.

Each line traces back to a requirement from Chapter 5. Crucially, the assessment does not invent these; it makes the constraints the team already flagged in Inception explicit and reviewable before they bind the design.

9.2 NFR design: turning constraints into logical components

The NFR design step turns each constraint into a concrete pattern or logical component, so the design carries a visible answer to every non-functional demand. This is where “safety” stops being an adjective and becomes an isolated adapter, a monitor, and an inhibit path.

The workflow captures these as design patterns and logical components:

```
# NFR Design - charge-management-service
# (aidlc-docs/construction/charge-management-service/nfr-design/logical-components.md)

- bms-adapter          isolates the safety boundary; only path to the BMS
- safety-monitor       watches inhibit signal; forces safe idle on loss
- plan-cache           last-known-good schedule for offline execution
- command-verifier     validates mTLS identity + message integrity
Patterns: bulkhead (isolate adapter) · fail-safe default (safe idle) ·
↳ timeout/retry
```

The same design records which of those components are generated and which are not, so the safety boundary is explicit in the build plan rather than implied:

```
# Component placement (nfr-design/logical-components.md)

Component          Unit    Generated?
```

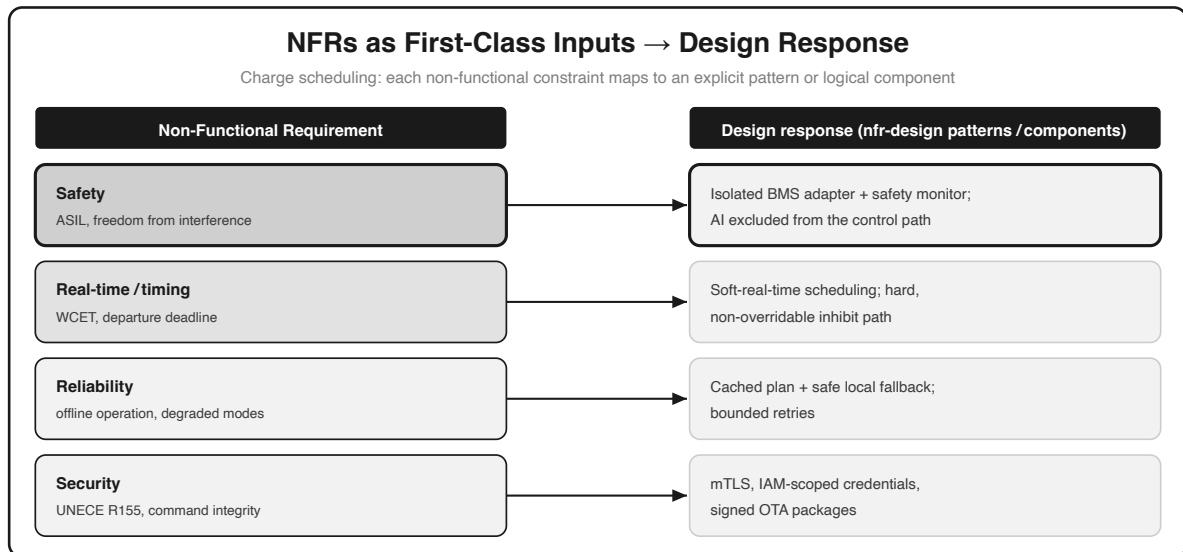


Figure 8: Mapping diagram with two columns. Left, four non-functional requirements for charge scheduling: Safety (ASIL, freedom from interference); Real-time/timing (WCET, departure deadline); Reliability (offline operation, degraded modes); Security (UNECE R155, command integrity). Each maps by an arrow to a design response on the right: an isolated BMS adapter plus safety monitor with AI excluded from the control path; soft-real-time scheduling with a hard, non-overridable inhibit path; a cached plan with safe local fallback and bounded retries; and mTLS with IAM-scoped credentials and signed OTA packages.

bms-adapter	U4	No	(human-engineered, review-gated)
safety-monitor	U4	No	(human-engineered)
plan-cache	U3	Yes	(QM)
command-verifier	U3/U1	Yes	(QM, security-reviewed)
schedule-executor, SoC-track	U3	Yes	(QM)

Two of these, the BMS adapter and the safety monitor, exist specifically to contain the safety-critical behavior so the rest of the service can be built and verified at QM pace. The NFR design is, in effect, where the safety boundary becomes architecture.

9.3 Functional safety and timing as explicit AI-DLC inputs

Safety and timing are not advisory notes for these units; they are explicit inputs the workflow must honor. Three in particular drive the design:

- **ASIL and freedom from interference.** The safety-relevant behavior is confined to the BMS adapter and monitor, and the QM-rated service must not be able to interfere with the BMS's authority. This is an architectural constraint, enforced by isolation, not a coding convention.
- **Timing budgets, typed.** The readiness-by-departure behavior is soft real-time, but honoring a BMS inhibit is hard real-time with a worst-case execution time (WCET) the design must respect. Typing the timing, soft vs. hard, carries the constraint into both code generation and the test gate.
- **Freedom-from-interference for security.** Authenticated, integrity-protected commands (R155) are treated as a safety-adjacent property, since a forged command must never reach the control path.

Typing the timing turns a vague “fast” into a budget the design and the test gate can both hold to. For charge scheduling the classes are recorded explicitly:

```
# Timing budget (nfr-design)

readiness-by-departure    soft real-time    plan may adjust; miss => notify driver
↪ (CS-FR-014)
honor BMS inhibit         HARD real-time    inhibit-to-stop within WCET budget; no
↪ override (CS-SR-002)
loss of BMS interface     HARD                detect and enter safe idle within
↪ bounded time (CS-SR-003)
```

Freedom from interference is then enforced structurally, not by convention: the QM service reaches the battery only through the isolated adapter, so no scheduling bug, retry storm, or cost-optimization path in the generated code can delay or override the inhibit. The adapter and monitor own the hard-real-time behavior; the generated service lives entirely in the soft-real-time world around them, and the partition between the two is what makes the freedom-from-interference claim provable rather than asserted.

Because these are recorded as NFR artifacts with requirement traceability, they become verifiable gate criteria in Chapter 8’s build-and-test step rather than hopes.

9.4 Where GenAI must not be in the loop

This is the hardest and most important boundary in the book. Everything so far has shown AI-DLC accelerating SDV work; this section draws the bright line it must not cross. Generative AI does not author the safety-critical control path, the logic that can directly cause or prevent a hazard. For charge scheduling that means the BMS’s thermal and cell protection control and its authoritative inhibit are human-engineered, reviewed, and verified on hardware; AI is not in that loop.

The diagram makes the rule concrete. Plan and command flow left to right and are generated then reviewed; but the BMS inhibit flows back across the boundary and always wins, and it is never AI-generated. The principle generalizes beyond this feature: braking, steering, powertrain, and any ASIL-C/D control path sit on the human-engineered side of the line. AI-DLC still contributes around them (tests, documentation, the QM-rated orchestration, the interface contracts) but the control logic itself, and the accountability for it, stays with engineers.

Drawing this line explicitly is not a limitation of the methodology; it is what makes the methodology safe to adopt in automotive at all.

9.5 The boundary as an enabler, not just a restriction

It is tempting to read Section 9.4 as purely a “no”, a list of what AI must not touch. In practice the boundary does more than forbid; it *enables* everything on the other side of the line to move fast. This is the quiet payoff of isolating the safety-relevant behavior into a small adapter and monitor.

Consider what the isolation buys. Because the BMS adapter and safety monitor contain all the safety-relevant logic, the rest of the charge-management service is provably unable to influence the battery except through that reviewed, human-engineered contract. That containment means the QM-rated scheduling logic, the bulk of the feature, can be generated, changed, and re-generated at full AI-DLC pace without re-opening the safety argument each time, because no

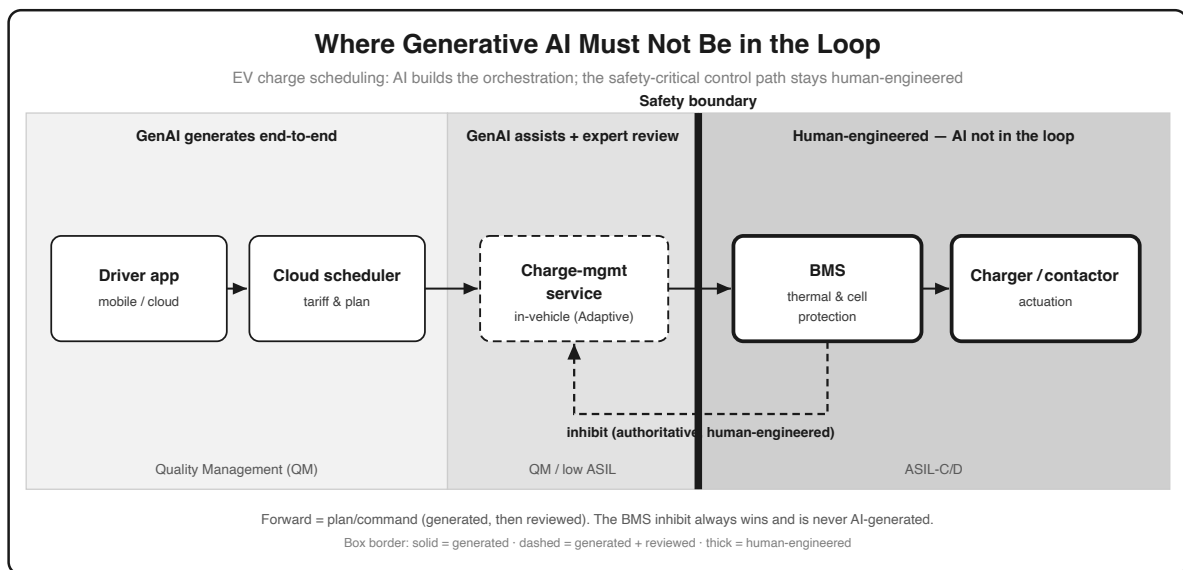


Figure 9: Left-to-right control path for EV charge scheduling across three zones shown as increasing gray shades. Lightest zone (Quality Management): driver app and cloud scheduler, with solid-bordered boxes, where GenAI generates end-to-end. Mid-shade zone (QM/low ASIL): the in-vehicle charge-management service, in a dashed-bordered box, where GenAI assists with expert review. A bold black vertical “safety boundary” line separates these from the darkest zone (ASIL-C/D): the BMS thermal and cell protection and the charger/contacter, in thick-bordered boxes, which are human-engineered with AI not in the loop. Forward arrows carry the plan/command; a dashed return arrow from the BMS carries an authoritative, human-engineered inhibit that always wins.

change to it can cross the boundary. A small, stable, human-owned core protects a large, fast-moving, AI-accelerated periphery. Without the boundary, the safety-relevant behavior would be smeared across the whole service, and *every* change, even a cosmetic scheduling tweak, would potentially touch safety and demand the full rigor. The line is what keeps the expensive rigor confined to a small surface.

This reframes the maturity gradient of Chapter 4 as an architectural strategy rather than a lament. You do not accept “AI can’t do the safety-critical part” as a ceiling on productivity; you *design* so that the safety-critical part is as small and well-contained as it can be, and then let AI-DLC accelerate everything around it. Good NFR design is, in this sense, what converts the safety boundary from a brake on the whole feature into a brake on only the small part that truly needs one.

9.6 NFR anti-patterns in SDV

The failure modes here are consistent enough to name. Each is a way of letting a non-functional constraint slip from an input into an afterthought, and each has a concrete cost on the bench or in the field.

- **Post-hoc tuning.** Treating safety, timing, and interference as qualities to optimize once the feature works. In an SDV these are architectural; deferring them means discovering a wrong structure on the test bench, when the fix is a redesign rather than a tweak.
- **Untyped timing.** Writing “fast enough” instead of a class. A requirement that does not say soft versus hard, and for hard paths does not name a worst-case execution time (WCET) budget, cannot be verified, so the test gate has nothing to check the inhibit path against.
- **A smeared safety surface.** Letting more than one component reach the battery, or scattering safety-relevant logic across the service. Freedom from interference becomes unprovable, and every later change potentially touches safety; keep a single isolated adapter as the only path.
- **Security as later hardening.** Postponing command authentication and integrity to a “security pass.” For an SDV, command integrity (R155, CS-SEC-005) is safety-adjacent, since a forged command must never reach the control path, so it is designed into the `command-verifier`, not bolted on.
- **Generating the safety monitor.** Letting the model author the adapter or monitor because it can read the interface. Reading is not licence to generate; putting the generator on the control path is exactly the boundary Section 9.4 forbids, and it quietly moves accountability off the engineers who own it.

Each anti-pattern is avoided by the same move the chapter has argued for: type the constraint, record it as a traceable NFR artifact before code, and let it shape the structure rather than critique it afterward.

Key takeaways

- AI-DLC assesses NFRs as a first-class, per-unit step; in SDVs safety, timing, reliability, and security are often the hardest part of the feature, not after-the-fact tuning.
- NFR design turns each constraint into a concrete pattern or logical component: the safety boundary becomes an isolated adapter, a monitor, and an inhibit path.
- Functional safety and timing are explicit, typed inputs (ASIL, freedom from interference, hard vs. soft timing) that become verifiable gate criteria.

- Generative AI must not author the safety-critical control path; it works around it while the control logic and its accountability stay human-engineered.
- The boundary is an enabler, not just a restriction: isolating a small, stable safety core lets the large QM periphery be generated and re-generated at full pace without re-opening the safety argument, so good NFR design confines the expensive rigor to the smallest possible surface.
- Type the timing and enforce freedom from interference structurally: hard paths (the inhibit and safe-idle) get a WCET budget and are reachable only through the isolated adapter, so generated code cannot delay or override them.
- Avoid the recurring NFR anti-patterns: post-hoc tuning, untyped timing, a smeared safety surface, security as later hardening, and generating the safety monitor.

What's next

Part IV moves to Operations and governance, starting with deployment and monitoring.

Chapter 10: Operations — Deploy, Monitor, Document

After this chapter you can run AI-DLC's Operations phase for an SDV feature: OTA deployment, monitoring, documentation from accumulated context, and feeding learnings into the next Bolt.

10.1 OTA deployment in the Operations phase

A note on scope first. In the open-source AI-DLC workflow today, Operations is deliberately a placeholder: the deterministic, gated stages end after Build & Test in Construction. That is an honest reflection of where the tooling is, and it matters for the maturity argument in Chapter 4: the further a workload moves from code generation toward field operation, the less the workflow automates and the more humans own. This chapter therefore describes how the Operations phase *applies to SDV* and what its documentation-first artifacts look like, extending the same principle the earlier phases established rather than claiming automation that does not yet exist.

Operations is also where AI-DLC's three phases become a loop rather than a line. A feature does not end at deployment; it is deployed, observed, documented, and fed back as new intent.

For EV charge scheduling, deployment is an over-the-air rollout of the in-vehicle charge-management component, gated by a human sign-off and staged rather than flipped fleet-wide. The phase records what was deployed, to whom, and on what evidence:

```
# Release Record - charge-management-service v1.3.0
# (aidlc-docs/operations/release-record.md)

Gate:      Quality Steward sign-off [human]   build-and-test-summary: PASS
Rollout:    canary 1% -> phased 25% -> fleet 100%   (halt on KPI breach)
Rollback:   proven; previous version v1.2.4 retained on-vehicle
Traces to:  requirements CS-FR-014/021, CS-SR-002/003, CS-SEC-005
R156:       every VIN's deployed version recorded with this requirement baseline
```

The release gate is the human decision point Chapter 3 named: the Quality Steward confirms operational acceptance and approves deployment. AI proposes the rollout strategy and assembles the evidence; a person signs.

The staging itself is a safety mechanism, not just a deployment convenience. Shipping a change to a whole fleet at once means a latent defect reaches every vehicle before anyone can react; staging bounds the blast radius. The canary phase exposes the update to a small, monitored population where a regression shows up in the metrics before it matters at scale; each subsequent phase widens exposure only if the previous one held its key performance indicators (KPIs). Underpinning all of it is a proven rollback: the previous version stays resident on the vehicle so a bad update can be reverted without a workshop visit, and R156 requires that every vehicle's

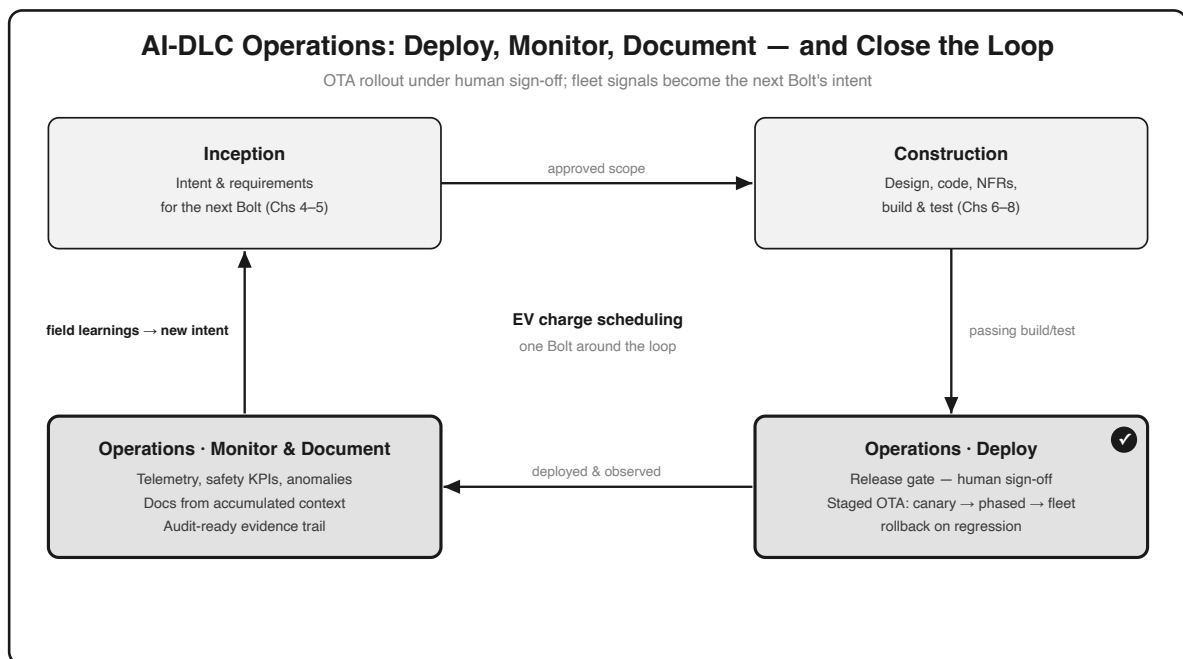


Figure 10: A clockwise four-corner cycle for one EV charge scheduling Bolt. Top-left Inception (intent and requirements) leads to top-right Construction (design, code, NFRs, build and test), which leads down to bottom-right Operations Deploy — a release gate with human sign-off and a staged OTA rollout from canary to phased to full fleet, with rollback on regression. That leads left to bottom-left Operations Monitor and Document — telemetry, safety KPIs, anomaly detection, documentation from accumulated context, and an audit-ready evidence trail — which loops back up to Inception labeled field learnings to new intent. The two Operations corners are emphasized with darker fill and thicker borders.

version history is known at each step. In an SDV the deployment strategy is therefore part of the safety argument (a way of catching, containing, and undoing a fault in the field) rather than an operational afterthought. AI can propose the staging plan and watch the KPIs; the decision to advance or halt each phase remains a human one.

10.2 Monitoring, telemetry, and feedback into the next Bolt

Once deployed, the feature is observed. Telemetry from the status-and-feedback unit flows back into the fleet data pipeline, and monitoring tracks both product metrics (did charging finish by departure?) and safety-relevant signals (did the BMS inhibit fire more often than expected?). AI is genuinely useful here for anomaly detection across a fleet's worth of data; humans judge severity and response.

```
# Monitoring Summary - charge-management-service v1.3.0 (first 30 days)
# (aidlc-docs/operations/monitoring-summary.md)

Readiness accuracy: 97.4% reached target SoC by departure (goal >= 95%)
Cost savings:       median 18% vs. unscheduled charging
Anomaly:            0.3% of sessions saw early BMS inhibit in cold ambient
Action proposed:    refine pre-conditioning window -> candidate next-Bolt intent
Safety:             no inhibit-handling regressions; all safe-idle on link loss
```

That last anomaly is the important one: it is not a defect in the safety path (the inhibit worked exactly as designed), but a signal that the *orchestration* could be smarter in cold weather. It becomes a candidate intent for the next Bolt, the loop closing.

The separation of signals is deliberate and matters for how the fleet is run. **Product signals** (did charging finish by departure, how much did drivers save) answer whether the feature is delivering value and guide the product roadmap. **Safety-relevant signals** (how often the BMS inhibit fired, whether any vehicle failed to reach safe idle on link loss) answer whether the safety argument still holds in the field, and they are watched against the assumptions made during design and validation. AI earns its place here precisely because a fleet produces more telemetry than any team can read: it clusters, ranks, and flags the handful of patterns worth human attention out of millions of ordinary sessions. But the judgment that turns a flag into an action (a tolerable quirk, a roadmap item, or a safety concern warranting a halt) stays human, and a safety-relevant flag is escalated to the people accountable for the safety case, not folded into the product backlog.

10.3 Documentation-first operations and accumulated context

Because every earlier phase emitted artifacts, Operations does not start from a blank page. The deployment, monitoring, and incident documentation is generated from the context the Bolt accumulated (requirements, design, NFRs, test results, and the decision audit trail) rather than written by hand after the fact. The audit log is the spine:

```
# Audit Trail (excerpt) (aidlc-docs/audit.md)

2026-06-12T09:14Z Requirements approved CS-FR/NFR/SR set v1
2026-06-18T15:02Z Design + NFR design approved BMS adapter isolated
2026-06-24T11:47Z Build & Test PASS 142 unit / 18 integ / 6 contract
2026-06-27T08:30Z Release v1.3.0 signed off Quality Steward; staged OTA
```

For an automotive program this accumulated record is not just convenience; it is the evidence base. The same trail that documents operations is what Chapter 12 draws on to satisfy ISO

26262, ISO 21434, and UNECE R156, the deployed version of every VIN traceable back through tests, design, and requirements to the original intent.

10.4 Incident response and the difference between a fix and a rollback

Not every field signal is a candidate for the next Bolt; some demand action now. The Operations phase therefore needs a clear decision discipline for when something goes wrong in the field, and it is a distinctly automotive one because the options carry different safety and regulatory weight.

The first question is severity and containment. A monitored regression caught in a canary phase is contained by definition: halt the rollout, and the fault never widens. A problem discovered after broader deployment forces a sharper choice between **rollback** and **fix-forward**. Rollback reverts affected vehicles to the previous known-good version, which is fast and low-risk precisely because that version already passed every gate and is retained on the vehicle; it is the default for anything touching safety. Fix-forward, patching and rolling a new version, is appropriate for a contained, non-safety product issue, but it means running a fresh change through the same Construction gates rather than shipping an unvetted hotfix. The one option that does not exist is an ungated emergency patch to the field: the whole point of the methodology is that nothing reaches a vehicle without passing verification a human approved.

Two automotive constraints frame the decision. A change significant enough to alter the vehicle “type” can carry type-approval implications, so the incident call is not purely technical. And whatever is decided, it is recorded: severity, action, and rationale enter the same audit trail, because an incident and its handling are part of the vehicle’s lifetime evidence just as much as the original release. Crucially, none of this touches the control path: because the safety-critical logic was human-engineered and the inhibit is authoritative, a field incident in the orchestration layer is handled by staging, rollback, and gated fixes, never by putting generative AI on the safety loop under time pressure.

10.5 Closing the loop back to Inception

The loop closes when a field signal becomes new intent. The cold-weather anomaly from monitoring does not get hot-fixed in production; it enters the next Bolt as a fresh, captured intent, and runs the same Inception → Construction → Operations path, with the same gates and the same safety boundary.

```
# Captured Intent - next Bolt
# (aidlc-docs/inception/requirements/requirements.md [draft - next Bolt])

Intent:    Improve cold-ambient pre-conditioning so target SoC and cabin
           readiness hold below 0 C, without changing the BMS safety path.
Source:    Operations monitoring v1.3.0 (early-inhibit anomaly, 0.3% sessions)
In scope:  scheduler/pre-conditioning logic (QM)
Out of scope: BMS thermal control (safety-relevant; unchanged)
```

This is the payoff of treating governance and traceability as the product (Chapter 3): a field observation links cleanly to a new requirement, the new work inherits the existing safety boundary, and the vehicle improves over its lifetime without ever putting generative AI on the control path.

Key takeaways

- In the current open-source workflow, Operations is a placeholder that ends after Build & Test; this chapter shows how its documentation-first principle extends to SDV deployment and field feedback.
- OTA deployment is a human-gated, staged rollout (canary → phased → fleet) with proven rollback, recorded with full requirement and R156 traceability.
- Monitoring separates product signals from safety signals; AI flags anomalies across the fleet, and humans judge severity and response, escalating safety-relevant flags to those accountable for the safety case.
- Incident response is a disciplined choice between staged-rollout halt, rollback to a retained known-good version, and gated fix-forward, never an ungated emergency patch, and never AI on the control path under time pressure.
- Operational documentation is generated from accumulated context, and field learnings close the loop back to Inception as new intent, improving the feature over the vehicle's life without AI entering the control path.

What's next

Chapter 11 turns to measurement: understanding the productivity gains from AI-DLC using DORA metrics.

Chapter 11: Understanding Productivity Gains — AI-DLC and DORA Metrics

After this chapter you can measure whether AI-DLC actually improves delivery performance for SDV work, using the four DORA metrics, map its effect across the vehicle software stack, and avoid mistaking raw speed for productivity.

11.1 Why measure the gain, not just feel it

Chapter 4 named the productivity paradox: AI generates code several times faster than teams can review it, adoption is high while trust is low, and the bottleneck shifts from writing to validating. That paradox is exactly why productivity claims must be *measured* rather than felt. “It feels faster” is not evidence an automotive organization can act on, and, worse, generating more code faster can look like progress while quality and safety quietly degrade.

The Value pillar of the VRF framework (Chapter 4) asked whether a workload’s gains justify adoption. This chapter supplies the instrument to answer that with data after the fact. The DORA metrics, from the DevOps Research and Assessment program, are the industry-standard way to quantify software *delivery performance*, and they have a property that suits the productivity paradox perfectly: they measure speed and stability together, so a gain in one that costs the other shows up rather than hiding.

A word on why *delivery* performance is the right thing to measure, rather than the tempting alternatives. Lines of code generated, prompts accepted, or features “AI-assisted” all measure activity, not outcome, and in an AI-led workflow they are especially misleading, because generation volume is exactly the thing that has become cheap. Measuring output volume in an AI-DLC program is like measuring a factory by how much raw material it consumes. DORA instead measures what the organization actually cares about: how quickly a *validated* change reaches the user, and how safely it lands there. That framing keeps attention on the flow of finished, trustworthy change through the whole pipeline, including the human gates, rather than on the generator’s raw throughput, which is precisely the number the productivity paradox warns you not to celebrate.

11.2 The four DORA metrics

DORA tracks four metrics in two pairs. Two measure **throughput** (how quickly change flows to users) and two measure **stability** (how safely it lands). The point is to read all four together.

For an SDV the definitions need slight translation. “Deployment” spans two very different acts: deploying a cloud service (continuous, low-ceremony) and deploying to vehicles via a staged

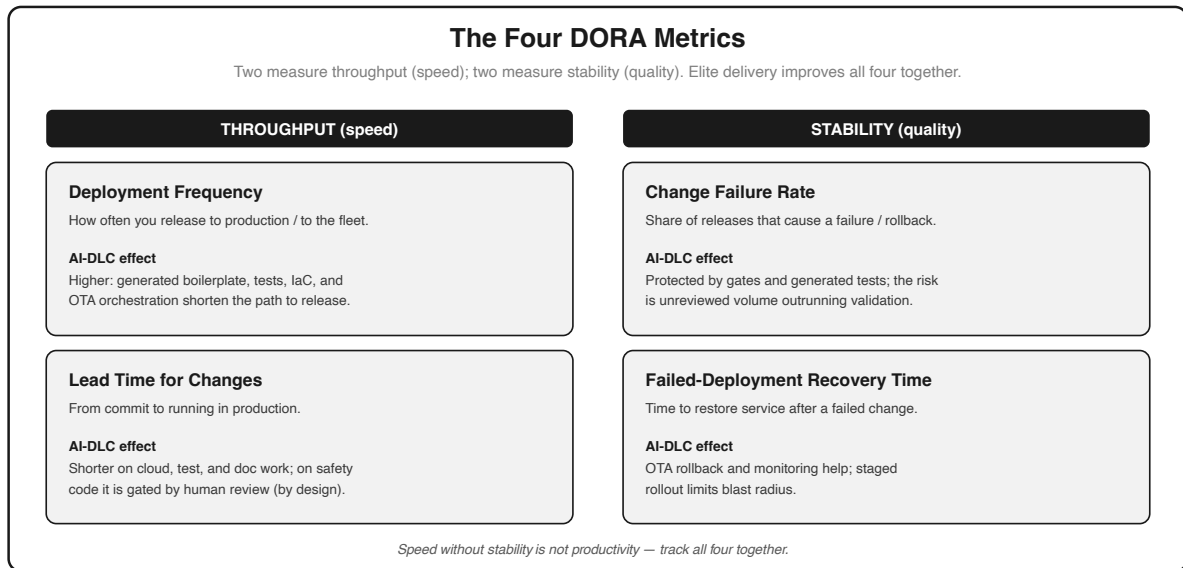


Figure 11: Four DORA metric cards in two columns. Throughput (speed): Deployment Frequency — how often you release to production or fleet, raised by AI-DLC’s generated boilerplate, tests, IaC, and OTA orchestration; and Lead Time for Changes — commit to running in production, shorter on cloud/test/doc work but gated by human review on safety code. Stability (quality): Change Failure Rate — share of releases that cause a failure or rollback, protected by gates and generated tests with the risk being unreviewed volume; and Failed-Deployment Recovery Time — time to restore after a failed change, helped by OTA rollback, monitoring, and staged rollout. A footer notes that speed without stability is not productivity.

OTA campaign (periodic, governed). A healthy SDV program therefore tracks DORA *per layer*, not as a single fleet-wide number: a cloud service can deploy many times a week while the in-vehicle component it talks to ships on an OTA cadence measured in weeks. Conflating them hides both signals.

11.3 What AI-DLC moves — and what it doesn't

AI-DLC does not move all four metrics equally, and it should not. Its throughput effect is strongest exactly where Chapter 4 said it would be, the cloud, and it deliberately yields to human gates where safety is at stake, which protects the stability metrics rather than sacrificing them.

Derived from the same documentation-first artifacts the earlier phases produced (`audit.md`, `build-and-test-summary.md`, `release-record.md`, and `monitoring-summary.md`), a DORA snapshot for the charge-scheduling units makes the per-layer split concrete:

```
# DORA Snapshot - EV charge scheduling (derived from aidlc-docs)
# sources: audit.md, build-and-test-summary.md, release-record.md,
↪ monitoring-summary.md

Unit: cloud-schedule-api          (Accelerate)
Deployment frequency ... on demand (multiple/week)
Lead time for changes ... hours (commit -> cloud deploy)
Change failure rate ..... low; contract tests + canary
Recovery time ..... minutes (automated rollback)
```

Where AI-DLC Moves the DORA Needle in SDV			
Impact varies by layer; stability is held by gates, not traded for speed			
DORA metric	Cloud	Middleware	Safety-critical control
Deployment Frequency	Strong ▲	Moderate	Minimal
Lead Time for Changes	Strong ▲	Moderate	Minimal
Change Failure Rate	Improved (gated)	Guarded by gates	Human-owned
Failed-Deployment Recovery Time	Strong (rollback)	Moderate	Guarded by gates

Darker = stronger throughput gain. Dashed / thick border = stability held by human gates, never traded for speed.

Figure 12: A matrix of the four DORA metrics (rows) against three SDV layers (columns: Cloud, Middleware, Safety-critical control). Deployment Frequency and Lead Time show Strong improvement in cloud, Moderate in middleware, and Minimal in safety-critical. Change Failure Rate is Improved-but-gated in cloud, Guarded by gates in middleware, and Human-owned in safety-critical. Failed-Deployment Recovery Time is Strong via rollback in cloud, Moderate in middleware, and Guarded by gates in safety-critical. Darker cells mark stronger throughput gains; dashed or thick borders mark stability held by human gates rather than traded for speed.

```
Unit: charge-management-service (Pilot; in-vehicle)
Deployment frequency ... per OTA campaign (weeks) <- bounded by fleet rollout
Lead time for changes ... days; HIL gate + human review of BMS boundary
Change failure rate ..... low; staged canary -> phased -> fleet
Recovery time ..... staged rollback; per-VIN version pinning (R156)
```

The cloud unit shows the headline gains; the in-vehicle unit shows modest throughput improvement with stability deliberately protected by the HIL gate and staged rollout. That contrast is the honest productivity story: large, real gains on the cloud side; smaller, guarded gains in the vehicle; and no throughput claim at all on the safety-critical control path.

11.4 Measuring without gaming: guardrail metrics and SDV caveats

DORA is powerful but gameable. You can inflate deployment frequency with trivial releases, or shorten lead time by thinning review, both of which are actively harmful in automotive. Two safeguards keep the measurement honest.

First, **read throughput and stability as a pair**. A rising deployment frequency with a rising change failure rate is not a win; it is the productivity paradox showing up in the numbers. The validation bottleneck from Chapter 4 will appear here first, as failures and slow recovery, if generation is allowed to outrun review.

Second, **add SDV guardrail metrics that DORA does not cover**. Safety is not a DORA metric, and must never be treated as one. Useful guardrails alongside DORA include:

- Review latency and review backlog (is validation keeping up with generation?).
- Escaped-defect and field-anomaly rate (Chapter 10's monitoring).

- Safety-gate and HIL pass rates, and any safety-relevant rollback.
- Security-finding rate from the Security Baseline checks (Chapter 13).
- Cost efficiency: inference cost per Bolt and per merged change, tracked per layer, the runaway-cost guardrail from Chapter 4, read alongside DORA so a throughput gain bought with an exploding model bill does not look like a win.

The goal of measurement is not a higher DORA score; it is evidence that AI-DLC delivers real value at acceptable risk. Used that way, DORA turns the productivity-paradox debate from opinion into something an SDV organization can manage, while the safety boundary, which no metric is allowed to move, stays exactly where Chapter 9 put it.

11.5 Baselining honestly: attributing the gain

A number only means something against a baseline, and here the baselining is easy to get wrong in ways that flatter the tool. Two disciplines keep the attribution honest.

First, **measure the same kind of work before and after**. AI-DLC's gains concentrate in the cloud and cross-cutting layers, so a program that also happens to shift its mix toward cloud work during a pilot will show a DORA improvement that is partly composition, not method. Compare like with like (the same layer, similar change sizes) or the headline number conflates “we did easier work” with “the method helped.” The per-layer split from Section 11.3 is the guard against this: report the cloud and in-vehicle units separately so a shift in mix cannot masquerade as a productivity jump.

Second, **give the change time and expect an initial dip**. Teams adopting AI-DLC reorganize how they work (judgment moves upstream, review becomes the constraint) and the first Bolts often run slower, not faster, as the team learns the gates and builds review capacity. A snapshot taken during that transition understates the steady-state gain; a snapshot taken only where the tool shines overstates it. The trustworthy signal is a sustained trend across comparable work, read together with the guardrail metrics, not a single before/after pair chosen to make a case.

The reason to be this careful is not academic rigor for its own sake. In an automotive organization the DORA numbers will be used to decide where to widen AI-DLC adoption and where to hold back, and a gain misattributed to the method, when it was really easier work or a favorable snapshot, leads to pushing the tool into a layer where the honest maturity assessment of Chapter 4 said it does not yet belong. Good measurement protects the maturity boundary as much as the safety gates do.

Key takeaways

- Productivity claims must be measured, not felt; DORA quantifies delivery performance and exposes speed-for-stability trade-offs rather than hiding them.
- The four metrics split into throughput (deployment frequency, lead time) and stability (change failure rate, recovery time): track all four, and track them per layer in an SDV (cloud vs. OTA-delivered vehicle software).
- AI-DLC's throughput gains are strongest in the cloud, moderate in middleware, and minimal on safety-critical code; its gates protect the stability metrics by design.
- Guard against gaming by reading throughput and stability together and adding SDV guardrail metrics (review latency, escaped defects, safety-gate pass rate); safety is never a DORA metric.
- Baseline honestly: compare like-for-like work per layer and read a sustained trend (expecting an early adoption dip), so a gain from easier work or a favorable snapshot is not

misattributed to the method.

What's next

Chapter 12 examines safety, validation, and compliance inside the AI-DLC gates.

Chapter 12: Safety, Validation, and Compliance Inside AI-DLC

After this chapter you can place automotive safety and compliance correctly within AI-DLC’s phases and gates, and use its documentation-first artifacts as compliance evidence.

12.1 The hard problem: trusting AI-orchestrated, non-deterministic work

Automotive safety rests on a deterministic argument: given this hazard, this requirement controls it; given this requirement, this code implements it and this test verifies it. Generative AI is non-deterministic: ask twice, get two phrasings. The central tension of this chapter is reconciling those two facts.

The resolution is not to make the AI deterministic; it is to make the *process* deterministic around it. AI-DLC’s gates, process checker, and documentation-first artifacts mean that what advances is never “whatever the model produced” but “output a human reviewed, a checker validated, and a test confirmed.” The safety argument is built on the verified artifacts and the human decisions, not on trust in the generator. Two principles make this work in practice:

- **The generator is untrusted; the gates are trusted.** Nothing reaches a vehicle because AI wrote it. It reaches a vehicle because it passed verification a human signed off on.
- **The control path stays human-engineered.** As Chapter 9 established, generative AI never authors the safety-critical control logic. The hardest safety argument is therefore made about human-written code, exactly as today.

It helps to be precise about what the non-determinism actually threatens. It does *not* threaten the final artifact: a specific generated file is fixed, readable, testable, and version-controlled like any other, and once it passes review it is no more “non-deterministic” than code a contractor wrote. What varies is the *path* to that artifact: regenerate and you may get a different implementation of the same requirement. For safety, that is tolerable so long as the requirement and its verification are fixed: the argument is anchored to the requirement the code must satisfy and the test that proves it did, not to any particular way of getting there. The generator’s variability is thus quarantined behind the same gates that would catch a human’s mistake, which is the only sense in which “trusting AI” ever enters a safety case; it does not.

12.2 ISO 26262 and ISO 21448 (SOTIF) within AI-DLC gates

Functional safety work does not run alongside AI-DLC; it attaches to its gates. Each phase carries the safety activity that belongs there, so the safety lifecycle and the development lifecycle

advance together rather than as parallel paperwork.

AI-DLC point	ISO 26262 / SOTIF activity	Artifact it attaches to
Inception (requirements gate)	Hazard analysis and risk assessment (HARA); safety goals; ASIL assignment	requirements.md (safety requirements CS-SR-*)
Functional / NFR design gate	Technical safety concept; freedom from interference	nfr-design/logical-components.md (BMS adapter, monitor)
Build & test gate	Safety-requirement verification; SOTIF scenario coverage	build-and-test-summary.md (incl. BMS-boundary HIL)
Release gate	Confirmation that safety goals are met for the release	release-record.md (human sign-off)

For EV charge scheduling this is concrete: the CS-SR-002/003 constraints trace from a hazard (uncontrolled charging in a thermal fault) through the isolated BMS adapter design to the hardware-in-the-loop test that proves the inhibit is honored. SOTIF concerns (cold-ambient behavior, stale tariff data) are logged as scenarios and verified, not left implicit. The point is not that AI performs HARA; it does not. The point is that AI-DLC's artifacts give each human-owned safety activity a clean place to attach and a traceable record.

It is worth following that thread end to end for one hazard, because it shows the deterministic argument surviving intact through an AI-accelerated process. The hazard-analysis fragment behind CS-SR-002 fixes the top of the thread:

```
# HARA fragment - EV charge scheduling
↪ (aidlc-docs/inception/requirements/safety/hara.md)

Hazard:      uncontrolled charging during a battery thermal fault
Consequence: risk of thermal runaway
Classification: ASIL-C/D (severity x exposure x controllability)
Safety goal:  charging must never proceed while the BMS signals a thermal
↪ inhibit
Derived req:  CS-SR-002 (honor BMS inhibit immediately; non-overridable)
```

That derived requirement flows through the isolated BMS adapter design (U4) and lands at the verification end of the same thread, recorded in the build-and-test summary that gates the release:

```
# Build and Test Summary (excerpt)
↪ (aidlc-docs/construction/build-and-test/build-and-test-summary.md)

Safety-requirement verification:
  CS-SR-002  inhibit honored immediately      BMS boundary (HIL)  [human-reviewed]
↪ PASS
  CS-SR-003  safe-idle on interface loss      BMS boundary (HIL)  [human-reviewed]
↪ PASS
SOTIF scenarios:
  cold-ambient early-inhibit                  scenario suite
↪ PASS
  stale tariff data -> safe default window    scenario suite
↪ PASS
```

The figure draws this single hazard as one traceable thread, human-owned from end to end.

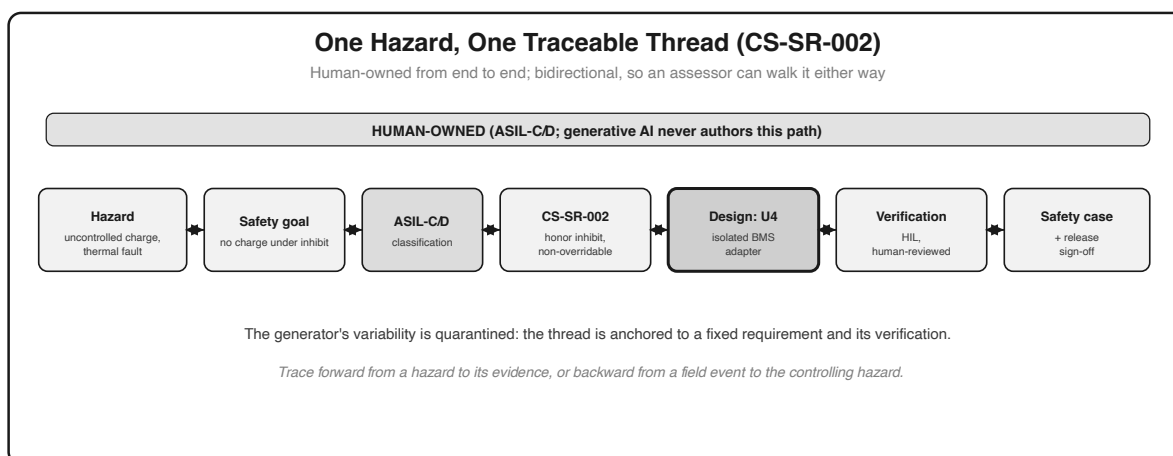


Figure 13: A single-row, left-to-right traceability thread for one hazard, with double-headed arrows between the nodes to show the link is bidirectional. The nodes are: Hazard (uncontrolled charging in a thermal fault), Safety goal, ASIL-C/D classification, Safety requirement CS-SR-002, Design (isolated BMS adapter U4), Verification (hardware-in-the-loop, human-reviewed), and Safety case plus release sign-off. A band across the top labels the whole thread human-owned, and a footer states that generative AI never authors this path and that an assessor can walk the thread backward from a field event to the controlling hazard.

12.3 UNECE regulation, type approval, and the AI-DLC audit trail

Type approval and the UNECE management-system regulations ask a question documentation-first development is well suited to answer: *can you prove, for every vehicle in the field, what software it runs and that the software was developed and deployed under a controlled process?* AI-DLC accumulates exactly that record as a by-product of doing the work.

The two UNECE regulations map directly onto the chain. R155 (the cyber security management system) is supported by the security requirements and their validation (CS-SEC-005, the TARA-to-control links). R156 (the software update management system) is supported by the release record: every VIN's deployed version traces back through tests, design, and requirements to intent. Because the chain is bidirectional, an assessor can start from a field event and walk back to the controlling requirement, the question regulators actually ask.

What AI-DLC does *not* resolve is the open regulatory question of AI provenance: assessors have not yet settled how AI-generated code should be treated in a certification argument. The honest position from Chapter 4 holds, for homologation-blocking and ASIL-C/D code the answer today is human-authored, human-accountable, and the audit trail records who decided and approved, not just what was produced.

12.4 Using documentation-first artifacts as evidence

Because the artifacts already exist and already link to one another, assembling a compliance dossier becomes a matter of organizing evidence rather than reconstructing it after the fact. The safety case, the structured argument that the item is acceptably safe, draws its claims, arguments, and evidence (the GSN/CAE structure from Chapter 3) from the same spine: safety goals from the requirements, the freedom-from-interference argument from the design, and the verification evidence from the test summaries and HIL results.

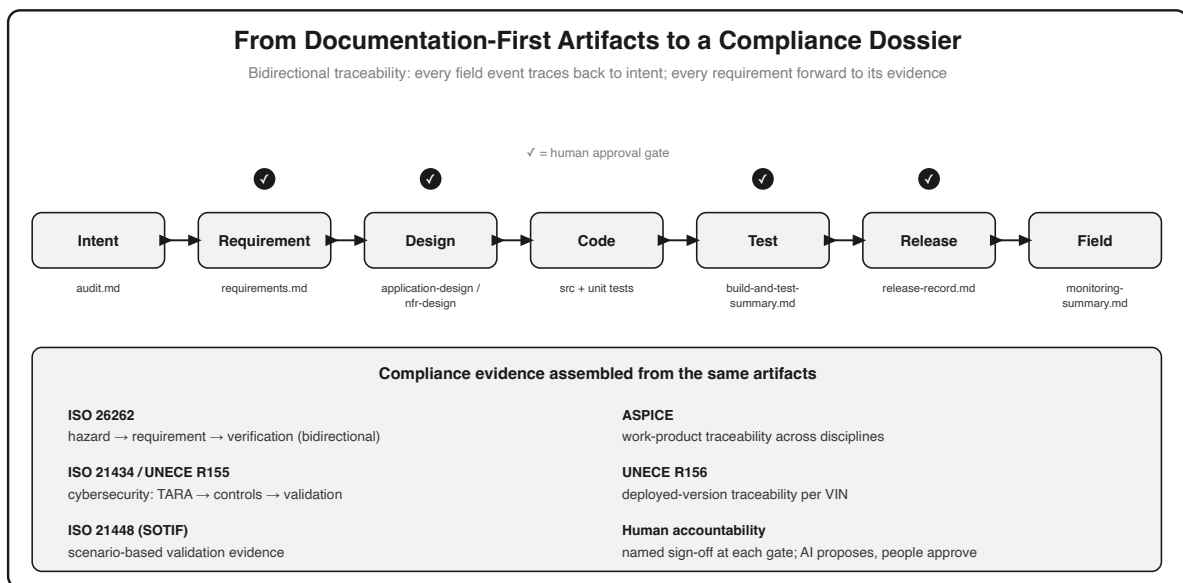


Figure 14: A horizontal evidence chain of seven linked nodes — Intent, Requirement, Design, Code, Test, Release, Field — joined by double-headed arrows to show bidirectional traceability. Human approval-gate check marks sit above Requirement, Design, Test, and Release. Below each node is the AI-DLC artifact that supplies its evidence: audit.md, requirements.md, application-design and nfr-design, source plus unit tests, build-and-test-summary, release-record, and monitoring-summary. A lower panel lists the compliance evidence assembled from the same artifacts: ISO 26262 hazard-to-verification traceability, ISO 21434 and UNECE R155 cybersecurity, ISO 21448 SOTIF scenario validation, ASPICE work product traceability, UNECE R156 deployed-version traceability per VIN, and human accountability with named sign-off at each gate.

A fragment of the safety case shows how the same artifacts become claims, arguments, and evidence, without any new authoring:

```
# Safety case fragment (Claims-Arguments-Evidence)

Claim:      Charging never proceeds while the BMS signals an inhibit.
Argument:   The inhibit is authoritative and non-overridable, enforced at an
            isolated adapter (U4) that is the only path to the battery.
Evidence:   CS-SR-002 (requirement)
            nfr-design/logical-components.md (freedom from interference; isolation)
            build-and-test-summary.md HIL result [human-reviewed]
            release-record.md (named sign-off)
```

Assembling the dossier is then a matter of collecting and ordering these links, not producing them: the claims come from the safety goals, the arguments from the design artifacts, and the evidence from the verification records, each already carrying the requirement IDs that tie it to the rest. What would traditionally be a late, effortful reconstruction becomes a query against a trail that was built as the work was done.

This is the practical answer to “is AI-DLC safe for automotive?” The methodology does not lower the evidence bar; it makes the evidence accumulate continuously and stay traceable, while keeping humans accountable at every gate and out of the generator’s reach on the control path. The dossier is stronger precisely because it was never a separate, end-of-program effort.

A caution worth stating plainly: an audit trail proves *process*, not *correctness*. The artifacts show that a requirement was approved, a test ran, and a person signed off; they do not by themselves prove the safety argument is sound. That judgment remains human, and full assurance still requires the established safety and validation activities. AI-DLC makes those activities better-evidenced and traceable; it does not replace them.

12.5 Validation becomes the constraint — and must be resourced

There is an uncomfortable consequence of accelerating generation while holding the safety bar fixed: the bottleneck moves to validation. If AI produces designs, code, and candidate tests several times faster than before, but review, safety analysis, and hardware-in-the-loop (HIL) capacity stay the same, the program does not go several times faster; it queues at the gates. This is the productivity paradox of Chapter 11 seen from the safety side, and pretending otherwise is how an organization quietly erodes its own rigor under schedule pressure.

The honest response is to treat validation capacity as a first-class investment, not a fixed backdrop. Three moves help. First, **grow review throughput deliberately**: more qualified reviewers, better tooling, and generated evidence that makes each review faster (a generated test suite that already names its requirements is quicker to assess than one reconstructed by hand). Second, **shift verification left where it is safe to**: SIL and automated checks catch the routine issues early so scarce HIL time and expert attention concentrate on what genuinely needs hardware and human judgment. Third, **protect the gate from volume pressure**: the answer to a backlog is never to review less carefully, and the audit trail makes any weakening of review visible rather than silent.

The strategic point is that AI-DLC does not remove the validation work; it relocates the constraint onto it. In automotive that is arguably a healthy relocation, because validation was always where safety was actually established; the methodology just makes it the explicit, measured, and resourced center of the process rather than the implicit end of it. An organization that adopts the generation gains without funding the validation capacity to match will find the

safety gates doing exactly their job: refusing to let unvalidated work through.

12.6 Safety and compliance anti-patterns

A handful of failure modes recur, and each undermines the argument the rest of the chapter builds:

- **Treating the audit trail as proof of correctness.** The trail proves process, that a requirement was approved and a test ran; the judgment that the safety argument is sound stays human.
- **Letting a capable model author safety-critical code.** No model earns the ASIL-C/D control path, and a more capable model is not a review discount. The hardest safety argument is always made about human-written code.
- **Skipping or deferring HARA.** Safety goals and ASIL come first; a safety requirement with no hazard behind it cannot be defended to an assessor.
- **Weakening review under schedule pressure.** The gate is the safety mechanism, so a validation backlog is resourced, never waved through; the trail makes any weakening visible rather than silent.
- **Retrofitting the safety case at the end.** If the dossier is a separate end-of-program effort, the evidence will not line up. Let it accumulate at the gates as the work is done.

Key takeaways

- The fix for non-deterministic generation is a deterministic process: untrusted generator, trusted gates, and a control path that stays human-engineered.
- Functional-safety and SOTIF activities attach to AI-DLC's gates, each with a clean artifact to anchor to; AI does not perform HARA, but its artifacts give the human-owned safety work a traceable home.
- The accumulated, bidirectional audit trail supports type approval and UNECE R155/R156, every VIN's deployed software traceable back to intent, though AI provenance in certification remains an open regulatory question.
- Documentation-first artifacts assemble into a safety case and compliance dossier continuously; the trail proves process, not correctness, so human safety judgment and established validation remain essential.
- Accelerating generation relocates the bottleneck onto validation; the safety gates only hold if review, safety analysis, and HIL capacity are resourced deliberately rather than left as a fixed backdrop.
- A single hazard traces as one human-owned thread, from HARA and safety goal through the CS-SR-002 requirement and the isolated U4 adapter to the human-reviewed HIL result and the safety-case sign-off.
- Avoid the recurring anti-patterns: treating the trail as proof of correctness, letting a model author safety-critical code, skipping HARA, weakening review under pressure, and retrofitting the safety case at the end.

What's next

Chapter 13 covers securing the pipeline itself, plus privacy and responsible AI.

Chapter 13: Security, Privacy, and Responsible AI in the Workflow

After this chapter you can secure the AI-DLC pipeline and its artifacts, handle IP and provenance of generated code, protect vehicle and fleet data, and put sound governance around the workflow.

13.1 Securing the AI-DLC pipeline and generated artifacts

Earlier chapters secured the *product*, the charge-scheduling feature’s commands and data. This chapter secures the *pipeline* that builds it. An AI-led workflow adds attack surface a traditional one does not: the context fed to the AI, the generated artifacts, and the build path that turns them into a vehicle update. The governing principle is the one from Chapter 12, the generator is untrusted, applied to security rather than safety.

It is worth naming the new threats concretely, because they have no exact equivalent in a hand-written pipeline. **Context poisoning:** if the AI draws context from a repository, an issue tracker, or documentation, an attacker who can influence any of those sources can attempt to steer generation, a comment or file crafted to read as an instruction rather than as data. **Prompt-borne exfiltration:** because the AI’s context is a channel, anything sensitive placed in it (a credential, a key, real fleet data) can leak through generated output or logs. **Artifact tampering:** a generated design or code file that is altered between generation and human review would let unreviewed content ride through the gate on the reviewer’s approval. **Supply-chain drift:** generated code can pull in dependencies the team never vetted. None of these are hypothetical for an SDV program whose output is signed software that drives to millions of vehicles; each maps to a control below, and all of them assume the generator and its output are untrusted until a human and the checks say otherwise.

The controls fall into three groups. **At the input**, the context given to the AI must not leak secrets or personal data and must be access-controlled: a prompt is an exfiltration path if it carries credentials. **Around generation**, the output is treated as untrusted until a human reviews it, and artifact integrity is maintained so a generated design or code file cannot be tampered with between generation and review. **At the build path**, commits are signed, secrets are scanned for, and the OTA package is signed and delivered over mutual TLS (mTLS), the same CS-SEC-005 posture, now applied to the delivery pipeline itself.

AI-DLC supports this directly through its opt-in **Security Baseline** extension. When enabled at Requirements Analysis, fifteen SECURITY rules[§] (encryption at rest and in transit,

[§]SECURITY-NN identifiers denote the individual rules of the AI-DLC Security Baseline extension (for example SECURITY-01, encryption at rest and in transit). The full set of fifteen becomes blocking gate constraints when the extension is enabled at Requirements Analysis, each checked and logged by its rule ID.

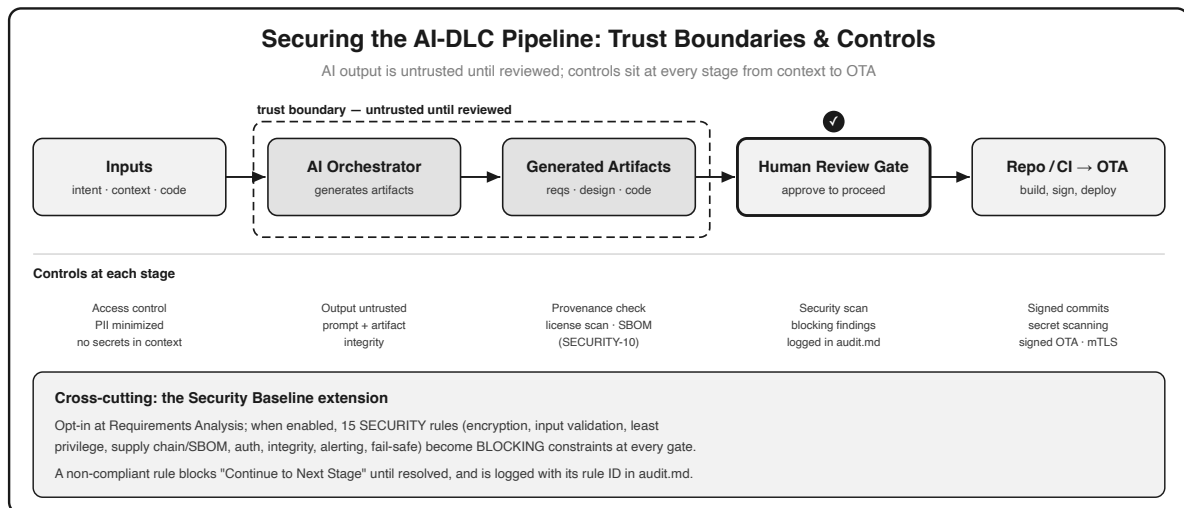


Figure 15: A left-to-right pipeline with five stages: Inputs (intent, context, code); AI Orchestrator; Generated Artifacts; a thick-bordered Human Review Gate with an approval check mark; and Repo/CI to OTA. A dashed trust-boundary box surrounds the AI Orchestrator and Generated Artifacts, labeled untrusted until reviewed. Below each stage are its controls: access control, PII minimized, and no secrets in context for inputs; output untrusted and prompt/artifact integrity for the orchestrator; provenance check, license scan, and SBOM for artifacts; security scan with blocking findings logged in audit.md at the review gate; signed commits, secret scanning, and signed OTA with mTLS for repo/CI to OTA. A footer panel describes the opt-in Security Baseline extension, whose fifteen SECURITY rules become blocking constraints at every gate.

input validation, least-privilege access, software supply-chain controls, authentication, integrity verification, alerting, and fail-safe defaults) become *blocking* constraints enforced at every gate:

```
# Stage Completion - Security Compliance (excerpt)
SECURITY-01 Encryption at rest/in transit ..... compliant
SECURITY-05 Input validation on API params ..... compliant
SECURITY-06 Least-privilege IAM policies ..... NON-COMPLIANT (blocking)
SECURITY-10 Supply chain / SBOM ..... compliant

# audit.md
2026-06-22T14:08Z SECURITY-06 finding: wildcard action in schedule-api role
-> "Continue to Next Stage" withheld until resolved
```

A non-compliant rule withholds the “Continue to Next Stage” option until it is fixed, and the finding is logged with its rule ID. Security stops being a late-stage scan and becomes a gate condition.

13.2 IP, licensing, and provenance of generated code

Generated code raises a question hand-written code usually does not: where did it come from, and what may it carry? Two risks matter for an automotive program. **Licensing:** generated code that echoes a restrictively licensed source could contaminate a proprietary ECU codebase or a supplier deliverable. **Provenance:** for a safety- or homologation-relevant artifact, “the AI wrote it” is not an acceptable lineage.

AI-DLC’s documentation-first trail and the Security Baseline’s supply-chain rule (SECURITY-

10) address both: dependencies are pinned, a software bill of materials (SBOM) is generated for production deployments, and the audit trail records what was generated, reviewed, and by whom. For the cloud-side charge-scheduling units, this is routine and well-supported (Chapter 4’s Accelerate band). For the in-vehicle and safety-relevant units, provenance is stricter precisely because the code is human-authored and human-accountable, the same boundary the book has drawn throughout. The practical rule: treat generated code as a contribution that must pass provenance and license review before it joins a regulated codebase, exactly as you would treat an external pull request.

13.3 In-cabin and fleet data privacy

Charge scheduling was flagged in Inception as privacy-sensitive because it handles location and usage data. Privacy is therefore both a product constraint and a workflow constraint.

As a **product** constraint, the design minimizes what it collects, encrypts data at rest and in transit (SECURITY-01), and keeps personal data out of logs (SECURITY-03); the monitoring pipeline tracks readiness accuracy and anomaly rates without needing to retain identifiable trip detail. As a **workflow** constraint, that same in-cabin and fleet data must not leak into the AI’s context during Inception or Construction: examples used to elaborate requirements or generate code should be synthetic or de-identified, never raw fleet records. The discipline is simple to state and easy to violate, a convenient real telemetry sample pasted into a prompt is a privacy incident.

13.4 Governance, roles, and auditability

The methodology’s gates only mean something if named people own them. AI-DLC’s human decision points map onto organizational roles, and the audit trail records who approved what:

AI-DLC gate	Accountable role	Recorded in
Requirements approved	Product Owner (+ functional safety for CS-SR-*)	<code>audit.md</code>
Design / NFR approved	Architecture Review Board; security engineer	<code>audit.md</code>
Security extension on/off	decided at intake	<code>aidlc-state.md</code> (Extension Configuration)
Build & test pass	Test/quality lead	<code>build-and-test-summary.md</code>
Release sign-off	Quality Steward (+ safety/regulatory)	<code>release-record.md</code>

The extension decision is itself recorded, so an auditor can see that security rules were enabled and enforced:

```
## Extension Configuration (aidlc-docs/aidlc-state.md)
| Extension           | Enabled | Decided At           |
|-----|-----|-----|
| Security Baseline   | Yes     | Requirements Analysis |
| Resiliency          | Yes     | Requirements Analysis |
```

This closes the responsible-AI loop the book has argued for: AI proposes, generates, and assembles evidence; named humans decide at every gate; security and privacy are blocking constraints rather than afterthoughts; and the whole chain is auditable. The methodology does not make these decisions for the organization; it makes them explicit, attributable, and recorded.

13.5 Responsible AI as a property of the workflow, not a policy on top

“Responsible AI” is often bolted on as a policy document, principles everyone signs and few can trace to a concrete control. The argument of this book is that AI-DLC makes responsible use a *property of the workflow itself*, which is a stronger position than a statement of intent. It is worth drawing the threads together, because the same handful of structural choices satisfy several responsible-AI principles at once.

Human accountability is structural: every gate has a named owner and every approval is recorded (Section 13.4), so there is never an action the organization cannot attribute to a person. **Transparency** is the documentation-first trail: the reasoning behind a feature (intent, options considered, decisions, and verification) is inspectable rather than locked inside a model’s weights. **Contestability** follows from the editable questionnaires: a safety or supplier engineer can change a question, reject an option, or send work back, so the human voice shapes the outcome rather than ratifying it. **Safety and security by default** come from making the hard constraints blocking gate conditions rather than best-effort guidance. And **data protection** is enforced at the point of risk, keeping personal and fleet data out of the AI’s context and out of logs.

For automotive this alignment is not incidental; it is what lets an AI-led workflow meet the expectations of frameworks like the EU AI Act, which asks high-risk AI systems for exactly these properties: human oversight, transparency, traceability, and risk management. AI-DLC does not guarantee compliance with any such framework, that remains the organization’s responsibility, but it produces the evidence and enforces the controls those frameworks ask for as a by-product of doing the work, rather than as a separate governance exercise layered on afterward. Responsible AI, in this telling, is less a promise than a shape the process is built into.

Key takeaways

- Securing the pipeline means treating AI context as a leak path and AI output as untrusted-until-reviewed, with controls at input, generation, and build/deploy.
- The opt-in Security Baseline extension turns fifteen SECURITY rules into blocking gate conditions, logged by rule ID in the audit trail.
- Generated code needs provenance and license review before joining a regulated codebase; SBOM and pinned dependencies support this, and safety-relevant code stays human-authored and accountable.
- In-cabin and fleet data is privacy-sensitive as both product and workflow: minimize and encrypt it, keep it out of logs, and never feed raw fleet data into AI context.
- AI-DLC’s gates map to named organizational roles and an auditable record, making security, privacy, and responsible-AI decisions explicit and attributable.
- The AI-led pipeline adds specific threats (context poisoning, prompt-borne exfiltration, artifact tampering, supply-chain drift), each answered by a control at input, generation, or build/deploy.
- Responsible AI is a property of the workflow, not a policy on top: human accountability, transparency, contestability, safe defaults, and data protection are structural, producing the evidence frameworks like the EU AI Act expect.

What's next

Chapter 14 turns to the reality that most SDV work is brownfield, showing how AI-DLC reverse-engineers and safely evolves existing vehicle software.

Chapter 14: Brownfield — Evolving Existing Vehicle Software with AI-DLC

After this chapter you can run AI-DLC against an existing SDV codebase: reverse-engineering legacy vehicle software into structured artifacts, grounding Inception in what already ships, and evolving the code in place without disturbing the certified safety-critical parts.

No such thing as a blank workspace

Every chapter so far has, for clarity, treated EV charge scheduling as if it began from a blank workspace. Almost no SDV work does. The feature you are asked to build lands in a vehicle that already has seventy-plus electronic control units (ECUs), a telematics stack a supplier shipped three model years ago, an AUTOSAR (AUTomotive Open System ARchitecture) configuration nobody fully remembers, and a Battery Management System (BMS) whose control code is already certified and frozen. The question is not “how do I generate this feature?” but “how do I add it to *that*, safely, without breaking traceability or re-opening a safety case?”

That is the brownfield problem, and it is the normal case in automotive. This chapter shows how AI-DLC handles it: it detects existing code, reconstructs an understanding of it as first-class artifacts, and then runs the same Inception → Construction → Operations loop from the earlier chapters, but anchored to the code that is already in the vehicle, and bounded by the parts that must not change.

14.1 Brownfield is the SDV default

Chapter 1 made the point that SDV software never really stops changing; Chapter 8 noted in passing that “little SDV work is greenfield.” This chapter takes that seriously. A production vehicle program is a decades-long accretion of code across suppliers, standards, and hardware generations. New value is almost always added *into* that estate, not beside it.

Brownfield changes the shape of the work in three ways that matter for AI-DLC:

- **Understanding precedes intent.** In greenfield, Inception starts with a blank idea. In brownfield, you cannot responsibly write requirements for a change until you know what the existing system does and how the new work will touch it.
- **The blast radius is real and pre-existing.** A change to a shared signal, a bus message, or an interface contract can ripple across ECUs and domains that were certified long ago. Change-impact analysis is not optional.

- **Traceability must be joined, not started.** The greenfield chain (intent → requirement → story → unit → code → test) has to graft onto whatever traceability the legacy code already carries, or reconstruct it where none exists.

AI-DLC is built for this. Its workflow is *adaptive* (Chapter 2): before it asks what you want to build, it detects whether code already exists and, if so, inserts a reverse-engineering stage ahead of Inception. The greenfield walkthrough of Chapters 5–10 was the simplified path; the brownfield path adds a front end and tightens the change discipline throughout.

14.2 Workspace detection and the reverse-engineering stage

The brownfield path opens with a cheap, automatic step: **workspace detection**. The workflow scans the workspace for source files and build files and classifies the project as greenfield (no code) or brownfield (code present). For a vehicle platform it finds C/C++ sources, AUTOSAR ARXML (AUTOSAR XML) and toolchain configuration, cloud-service code, and build descriptors, and records the finding before any feature work begins.

When code is found, the workflow runs **Reverse Engineering**, an analysis pass that reconstructs a structured understanding of the existing system and writes it under `aidlc-docs/inception/reverse-engineering/`. This is the brownfield counterpart to the intent bootstrap: instead of eliciting a new idea, it recovers the context an idea will have to fit into. Its outputs are documentation-first artifacts, each a fenced view the mob can review:

Artifact	What it captures
<code>business-overview.md</code>	The business the system implements and its key transactions
<code>architecture.md</code>	Packages, services, data stores, and how they relate
<code>code-structure.md</code>	Build system, module hierarchy, and an <i>Existing Files Inventory</i>
<code>api-documentation.md</code>	External and internal interfaces, and data models
<code>component-inventory.md</code>	Every package classified (application, infrastructure, shared, test)
<code>technology-stack.md</code>	Languages, frameworks, and toolchains in use
<code>dependencies.md</code>	Internal and external dependencies, with versions and licenses
<code>code-quality-assessment.md</code>	Test coverage, patterns, anti-patterns, technical debt

For the charge-scheduling feature, reverse engineering does not describe a feature we are inventing; it describes the vehicle software the feature must join. An excerpt of the reconstructed architecture:

```
# System Architecture    (aidlc-docs/inception/reverse-engineering/architecture.md)

## Component Descriptions
### telematics-gateway    (existing, supplier v2.4)
- Purpose:                off-board connectivity; MQTT to cloud; command routing
- Dependencies:           connectivity-mgr, secure-storage
- Type:                   Application (AUTOSAR Adaptive)

### bms-control            (existing, ASIL-C, certified - DO NOT MODIFY)
- Purpose:                cell protection, thermal limits, charge inhibit (authoritative)
- Interface:              ara::com BmsChargeControl (inhibit, permitted-envelope)
- Type:                   Application (AUTOSAR Classic; safety-critical)
```

```
## Integration Points
- Vehicle bus:  CAN + Automotive Ethernet (SOME/IP)
- Cloud:       existing fleet MQTT broker; OTA campaign service (R156)
```

Two entries carry the whole chapter. The `telematics-gateway` is existing code the new feature will route through and therefore *modify in place*. The `bms-control` component is certified, safety-critical, and marked **do not modify**; reverse engineering surfaces the safety boundary from the existing code rather than inventing it, exactly as Inception did in Chapter 5 for the greenfield case.

The `code-structure.md` artifact matters most for what comes next, because it names the files a change might touch:

```
# Code Structure    (aidlc-docs/inception/reverse-engineering/code-structure.md)

### Existing Files Inventory    (candidates for modification)
- telematics/src/command_router.cpp    - routes cloud commands to services
- telematics/cfg/SomeIpServiceIntf.arxml - service interface config (config = code)
- bms/src/bms_control.c                - certified control loop [FROZEN /
  ↪ review-gate]
- ota/campaign/manifest_schema.json    - R156 update manifest
```

Reverse engineering is not a one-time cost. The workflow timestamps its output and compares it against the codebase's last significant modification; if the artifacts have gone **stale**, or an engineer explicitly asks, it reruns and refreshes them. That keeps the reconstructed understanding honest as the estate keeps moving, the living-documentation habit the rest of the book relies on.

The reverse-engineering stage ends at a human gate: the mob reviews the artifacts and either requests changes or approves them before any requirements are written. On an SDV, the systems, safety, and supplier engineers are the ones who confirm that the reconstructed picture, especially the frozen boundary, is correct.

14.3 Grounding Inception in the existing system

With the reverse-engineering artifacts approved, the feature runs the same Inception phase as Chapters 5 and 6 (intent, Mob Elaboration, requirements, stories, units) but every step now reads the reconstructed context as input. The difference is one of grounding, not of mechanism.

Intent, for a brownfield change, is stated against what exists: *add departure-time charge scheduling to the current telematics and charge path, reusing the existing cloud MQTT channel and the certified BMS inhibit interface*. Mob Elaboration's questionnaire then asks brownfield-specific questions the greenfield run never needed, and, because the questionnaire is editable, the mob adds the ones the model missed:

```
# Requirements Clarification Questions - Charge Scheduling (brownfield)
# (aidlc-docs/inception/requirements/requirements-questions.md)

## Question 2
How should the new schedule commands reach the vehicle?
A) Reuse the existing telematics-gateway command router (modify in place)
B) Add a new parallel connectivity path
C) Other (please describe after [Answer]: tag below)
[Answer]: A
```

Question 4

The BMS charge-inhibit interface (ara::com BmsChargeControl) already exists and is certified. How does the new service use it?

- A) Consume the existing interface unchanged; never modify bms-control
- B) Extend the interface
- C) Other (please describe after [Answer]: tag below)

[Answer]: A

The answers flow into the same structured requirements and the same stable IDs the book has used throughout, so the traceability chain *joins* the existing system rather than restarting. CS-FR-021 (execute a cached schedule with no connectivity) is now written knowing the telematics-gateway already buffers commands; CS-SR-002 (treat BMS limits as authoritative) is written against the real `BmsChargeControl` interface reverse engineering recovered, not an idealized one. Unit decomposition likewise reuses the greenfield map (U1–U6) but tags each unit as *new*, *modify existing*, or *frozen*:

Unit	Brownfield disposition	Existing code touched
U1 cloud-schedule-api	New	Deploys beside existing fleet services
U3 charge-management-svc	New	Consumes telematics + BMS interfaces
U4 bms-interface-adapter	New (thin)	Wraps the frozen <code>BmsChargeControl</code>
telematics-gateway (existing)	Modify in place	<code>command_router.cpp</code> , <code>SomeIpServiceIntf.arxml</code>
bms-control (existing)	Frozen	none (certified, do not modify)

This is where the greenfield safety boundary and the brownfield frozen boundary line up: the certified BMS control code was out of the AI's scope in Chapter 9 because it is safety-critical, and it is out of scope here because it is *already certified*: two reasons, same line, and generative AI stays on neither side's control path.

14.4 Evolving the code: modify in place, never duplicate

Construction for a brownfield feature obeys one discipline above all: **modify in place, never duplicate**. When the workflow needs to change existing behavior, it edits the existing file rather than emitting `command_router_v2.cpp` beside `command_router.cpp`. Duplication is tempting because it feels safe, but in an SDV it quietly forks the codebase, breaks the traceability the standards demand, and leaves two versions of a signal handler for a future engineer to reconcile.

The code-generation plan therefore reads as a mix of *create* and *modify* steps, each traced to a story and to the reverse-engineered file it touches:

```
# Code-Generation Plan - charge-management-service (brownfield)
# (aidlc-docs/construction/plans/charge-management-service-code-generation-plan.md)

[x] Step 1 CREATE charge-management-service/ business logic + unit tests
    ↳ (US-03)
[x] Step 2 MODIFY telematics/src/command_router.cpp - route schedule cmds
    ↳ (US-11)
[x] Step 3 MODIFY telematics/cfg/SomeIpServiceIntf.arxml - add service intf
    ↳ (config=code)
```

```
[ ] Step 4 CREATE  bms-interface-adapter/ - consume BmsChargeControl      <-
  ↳ review-gated
[ ] Step 5 FROZEN  bms/src/bms_control.c   - NOT MODIFIED (certified)
```

Three brownfield realities shape this plan. First, **configuration is code**: much of an AUTOSAR integration lives in ARXML and toolchain files, so a “modify” step often edits configuration rather than a .cpp, and the generated service is one participant in a larger, vendor-shaped build. Second, **change-impact analysis is a gate, not an afterthought**: before a modify step runs, the workflow uses the reverse-engineered dependency and API artifacts to identify what else consumes the file being changed, so a touched bus message or interface does not silently break a neighbouring ECU. Third, **oversized legacy migrations are scoped, not attempted wholesale**: where a codebase is too large to hold in context, the work is bounded to the unit and its interfaces, the honest limit from Chapter 4, not a bulk rewrite.

The verification loop from Chapter 8 does extra duty in brownfield. Alongside the new feature’s tests, the build-and-test gate runs the existing regression suite to prove the change did not disturb what already shipped, and the human-reviewed hardware-in-the-loop (HIL) check confirms the frozen BMS boundary still behaves exactly as before:

```
# Build and Test Summary (brownfield)
# (aidlc-docs/construction/build-and-test/build-and-test-summary.md)

New feature tests:      58 passed / 0 failed
Regression (legacy): 431 passed / 0 failed  <- no disturbance to existing behavior
Change-impact:         telematics command_router + 1 ARXML intf; no other ECUs
  ↳ affected
BMS boundary (HIL):    inhibit honored; bms-control unmodified  [human-reviewed]
```

Only a green summary (new tests, untouched regressions, and the human-reviewed frozen boundary) advances the feature to Operations, where the R156 update manifest that reverse engineering found is reused to package the change.

The figure below places the whole brownfield path next to the greenfield one, so the single added front end (detect, then reverse-engineer) and the tightened change discipline are visible at a glance.

14.5 Brownfield governance and pitfalls

Brownfield sharpens both the value and the risk of AI-DLC. The value is that reverse engineering turns an opaque legacy estate into reviewable artifacts and a joined traceability chain, often the single most useful thing a team inherits when picking up unfamiliar vehicle software. The risk is that the same speed can push unreviewed change into code that carries certification and field obligations.

A few durable practices keep the balance right:

- **Treat reverse-engineering artifacts as living evidence.** Approve them, keep them current with the staleness rerun, and cite them in change-impact analysis; they feed the ISO 26262 and ASPICE (Automotive SPICE) traceability the audit trail needs (Chapters 5 and 12).
- **Make the frozen boundary explicit and structural.** Mark certified, safety-critical modules do-not-modify in the artifacts, wrap them behind a thin adapter unit, and hold that adapter to the higher review bar, never regenerate the certified control code itself.
- **Gate on regression, not just new tests.** A brownfield change is only safe if it leaves existing behavior intact; the untouched-regression result is part of the evidence, not a

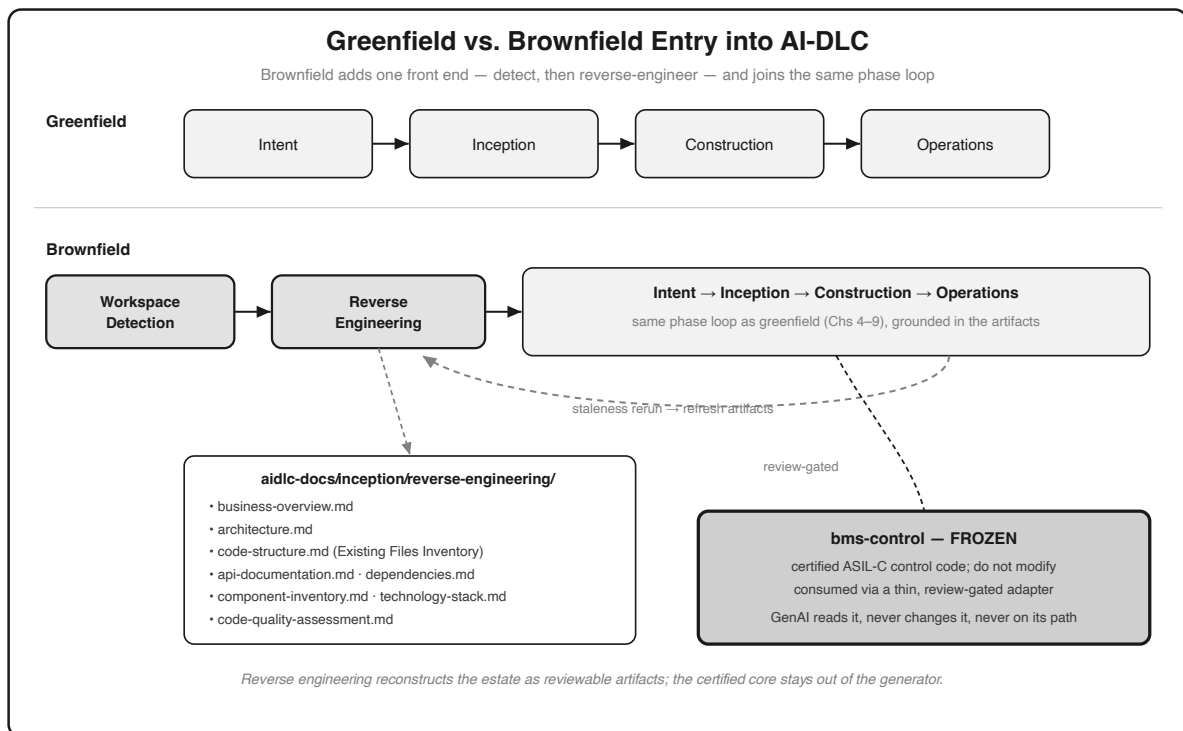


Figure 16: Two stacked horizontal tracks comparing paths into AI-DLC. The upper track, labeled Greenfield (lightest fill, solid border), runs Intent to Inception to Construction to Operations. The lower track, labeled Brownfield (mid-gray fill), begins with two extra boxes ahead of Intent: Workspace Detection and Reverse Engineering (which emits the reverse-engineering artifacts, listed beneath as business-overview, architecture, code-structure with an Existing Files Inventory, APIs, dependencies, and quality), then joins the same Intent to Inception to Construction to Operations flow. A dashed feedback arrow labeled staleness rerun loops from Construction back to Reverse Engineering. Throughout the brownfield track, a darkest-fill, thick-bordered box marked bms-control FROZEN sits below the flow, connected to the BMS adapter by a review-gated link, with a footer stating the certified safety-critical control code is never modified and generative AI never enters the control path.

convenience.

- **Prefer edits to duplication, always.** One evolving codebase with intact history beats two divergent copies, for both engineering and compliance.

And the anti-patterns to refuse, most of them familiar from Chapter 4 in a brownfield key:

- Pointing AI at large legacy migrations as a single bulk rewrite instead of scoping to a unit and its interfaces.
- Letting AI “improve” or regenerate certified safety-critical code because it can read it; reading is not licence to change.
- Skipping change-impact analysis and discovering a broken neighbouring ECU on the bench, or worse, in the field.
- Trusting stale reverse-engineering artifacts that no longer match the code.

Used with that discipline, AI-DLC does for brownfield SDV what it does for greenfield: it moves the human effort off reconstruction and boilerplate and onto the decisions that carry safety, timing, and regulatory weight, while the certified core of the vehicle stays exactly where the engineers put it.

Key takeaways

- Brownfield is the SDV default: new features are added into an existing estate, so understanding must precede intent, blast radius is pre-existing, and traceability must join rather than start.
- AI-DLC’s adaptive workflow detects existing code and inserts a **reverse-engineering** stage that reconstructs the system as structured artifacts under `aidlc-docs/inception/reverse-engineering/`, refreshed when they go stale.
- Inception then runs as before but **grounded** in those artifacts: intent, requirements, and units are written against real interfaces and reuse the same stable IDs, so the chain grafts onto the legacy code.
- Construction obeys **modify in place, never duplicate**, treats configuration as code, gates on change-impact and on untouched legacy regressions, and scopes oversized migrations rather than bulk-rewriting them.
- The frozen, certified safety-critical core (the BMS control code) is surfaced by reverse engineering and marked do-not-modify; generative AI reads it but never changes it and never enters its control path.

What’s next

Chapter 15 zooms out from a single feature to the standing operational framework that runs a whole fleet, and shows how AI-DLC embeds into each pillar of it.

Chapter 15: The SDV Operational Framework and How AI-DLC Embeds Into It

After this chapter you can describe the operational framework an organization runs to keep a fleet of software-defined vehicles safe, updatable, and compliant over their whole life, and map how AI-DLC embeds into each pillar of that framework rather than sitting beside it.

The standing machinery

Every earlier chapter followed one feature, EV charge scheduling, through a single pass of AI-DLC. A production program never runs just one pass. At any moment it is operating millions of vehicles on dozens of software versions, fielding defects, shipping campaigns, watching safety signals, and proving compliance, all at once and for years. That standing machinery is the *SDV operational framework*, and a methodology that stops at “we generated the feature” has not finished the job.

This chapter describes that framework and shows where AI-DLC fits inside it: not as a separate activity bolted on for delivery, but as the per-change engine whose documentation-first trail, human gates, and continuous loop become the framework’s evidence, control points, and improvement cycle.

15.1 What the SDV operational framework is

The operational framework is the standing operating model, the roles, processes, and tooling, that an original equipment manufacturer (OEM) and its partners run to deliver, operate, and assure vehicle software across the whole fleet and the whole life of each vehicle. It is not a project that finishes. It is the always-on system that keeps every deployed version known, monitored, and accountable from the day a car leaves the factory until it is retired.

It helps to hold this chapter apart from Chapter 10. Chapter 10 walked the Operations *phase* of a single Bolt: deploy one feature, monitor it, document it. This chapter is the standing *framework* that many Bolts flow through at the same time. If Chapter 10 is the engine that turns one change into a safe field release, this chapter is the vehicle that engine runs inside: the release trains, the fleet telemetry, the security operations, and the compliance evidence that all keep running whether or not a particular feature is shipping today.

In an SDV this framework is a safety and regulatory obligation, not merely an operations convenience. UNECE Regulation No. 156 (the Software Update Management System, or SUMS) requires that the software version on every vehicle identification number (VIN) is known and

traceable for the vehicle's life; ISO 26262 expects field monitoring against the safety assumptions the design was built on. A team can generate features quickly, but if it cannot say which version each car runs and why, it cannot legally keep those cars on the road. The framework is where that obligation is met, continuously.

15.2 The pillars of the framework

The framework is best understood as a set of pillars that run continuously and interlock. Each owns one standing responsibility and answers to a driving standard.

Pillar	What it owns	Key standard / driver
Operating model & roles	OEM/Tier-1/platform split; accountable owners (Quality Steward, safety, security)	ASPICE, org governance
Lifecycle & configuration management	The per-VIN truth of which software each vehicle runs	UNECE R156 / SUMS
Release & OTA management	Campaign packaging, staged rollout, proven rollback	UNECE R156
Fleet monitoring & observability	Telemetry, product vs. safety signals, key performance indicators (KPIs), anomaly detection	ISO 26262, ISO 21448 (SOTIF)
Incident, problem & change management	Severity triage, rollback vs. fix-forward, recall triggers	ISO 26262, regulatory reporting
Security operations	Vehicle Security Operations Center (VSOC), vulnerability management, software bill of materials (SBOM)	UNECE R155 / CSMS, ISO 21434
Compliance & assurance operations	Continuous evidence and audit readiness	ISO 26262 / 21434 / 21448, R155/R156
Data & privacy operations	Fleet data pipelines, consent, retention	Privacy regulation, EU AI Act
Cost & FinOps	Cloud and inference spend, per-layer attribution	Chapter 4 (measure, cap, alert)
Continuous improvement	Field learnings become the next change	The AI-DLC loop

These pillars are not organizational silos; they share one backbone. A single released change touches the lifecycle pillar (a new per-VIN version), the release pillar (the campaign that delivered it), the monitoring pillar (the signals it is watched by), the assurance pillar (the evidence it carries), and the security pillar (its supply-chain and vulnerability posture). The framework works only when those views are drawn from the same underlying record rather than reconstructed separately in each pillar, which is exactly the opening AI-DLC exploits.

The figure below places the pillars around the AI-DLC loop that feeds them, showing the change engine at the center of the standing framework.

15.3 Where AI-DLC embeds: the Operations phase as the per-change engine

AI-DLC does not replace the operational framework, and it is not a rival to it. It supplies the per-change flow that each pillar consumes. Every pillar needs a disciplined, evidenced stream of changes to operate on; AI-DLC is what produces that stream, one Bolt at a time, with the human gates and documentation the pillars depend on.

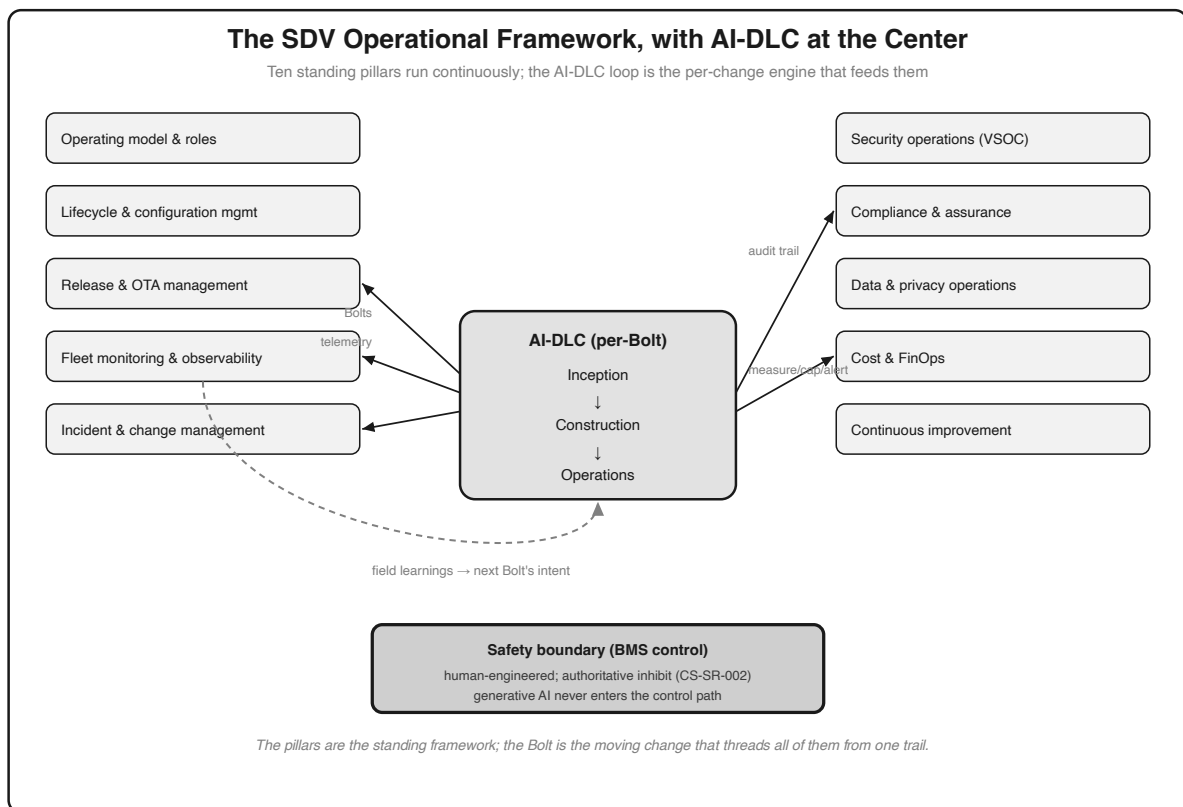


Figure 17: The SDV operational framework drawn as ten pillars flanking a central AI-DLC per-Bolt loop. The center shows the loop Inception, Construction, Operations with a feedback arrow. Five pillars sit on the left (operating model and roles, lifecycle and configuration management, release and OTA management, fleet monitoring and observability, incident and change management) and five on the right (security operations, compliance and assurance, data and privacy, cost and FinOps, continuous improvement). Arrows run from the central loop out to the pillars, labeled with the mechanism each receives (Bolts to release, the audit trail to assurance, telemetry to monitoring), and a return arrow runs from monitoring back into the loop. A footer states that the BMS safety boundary stays human-engineered and generative AI never enters the control path.

Framework pillar	AI-DLC mechanism that embeds into it
Release & OTA management	Each Bolt is the unit of change; the release gate is a human-approved AI-DLC gate
Lifecycle & configuration management	The audit trail records which requirements and tests apply to each deployed version, per VIN
Monitoring & observability	AI proposes anomaly detection and KPIs; humans judge; product and safety signals stay separated
Incident & change management	Staged-rollout halt, rollback to a retained version, or gated fix-forward, never an ungated patch
Security operations	The opt-in Security Baseline makes SECURITY rules blocking gate conditions; the CS-SEC-005 posture extends to the pipeline
Compliance & assurance	The documentation-first trail is the evidence spine (see 15.4)
Cost & FinOps	Measure, cap, and alert per Bolt; right-size models (Chapter 4)
Continuous improvement	Field learnings become the next Bolt's intent (see 15.6)

The charge-scheduling feature shows the embedding end to end. A cold-weather orchestration anomaly seen by the monitoring pillar (charging that finishes late in sub-zero ambient temperatures) becomes the intent for a new Bolt. That Bolt runs through Inception and Construction, then reaches the release pillar as a staged over-the-air (OTA) campaign, and lands in the lifecycle pillar as a new per-VIN version with its own evidence. Throughout, the Battery Management System (BMS) inhibit that the service must honor (CS-SR-002) stays human-engineered and outside the AI's loop; the improvement is in the QM-rated orchestration around it, never in the certified control path.

The deeper point is that the Bolt is the atomic unit that threads every pillar. The framework supplies the standing context (which vehicles, which versions, which obligations); AI-DLC supplies the moving change and the record that lets each pillar act on it without inventing its own bookkeeping. Remove AI-DLC and the pillars still exist, but they are fed by hand-assembled changes and hand-maintained evidence; embed it, and the same change simultaneously satisfies release, lifecycle, monitoring, security, and assurance because they all read from one trail.

15.4 The audit trail as the framework's evidence spine

The compliance and assurance pillar is the most expensive to run because it demands continuous, per-version evidence: for any VIN and any deployed version, an OEM must be able to show the requirements it met, the tests it passed, and the human who approved it. Done the traditional way, that evidence is reconstructed after the fact, which is slow and error-prone. AI-DLC produces it as a by-product of doing the work, so assurance becomes a query against the trail rather than an archaeological dig.

Each pillar draws the slice of the trail it needs. A small operations artifact makes the mapping explicit:

```
# Operational Evidence Map (aidlc-docs/operations/ops-evidence-map.md)
# How the standing framework draws on the per-Bolt audit trail.

Release pillar    -> release-record.md      (R156 version baseline per VIN)
Monitoring pillar -> monitoring-summary.md  (product vs. safety signals, KPIs)
Assurance pillar  -> audit.md               (intent -> requirement -> test ->
↪ field)
```

Security pillar	-> security-compliance.md (SECURITY rules; SBOM; CS-SEC-005)
Boundary	-> BMS control unchanged; HIL evidence human-reviewed

The figure below shows this spine: the per-Bolt artifacts at the base feed directly into the framework pillars above them, so a regulator's question, "show me that CS-SR-002 is honored on the version this VIN runs," resolves to a link, not a reconstruction.

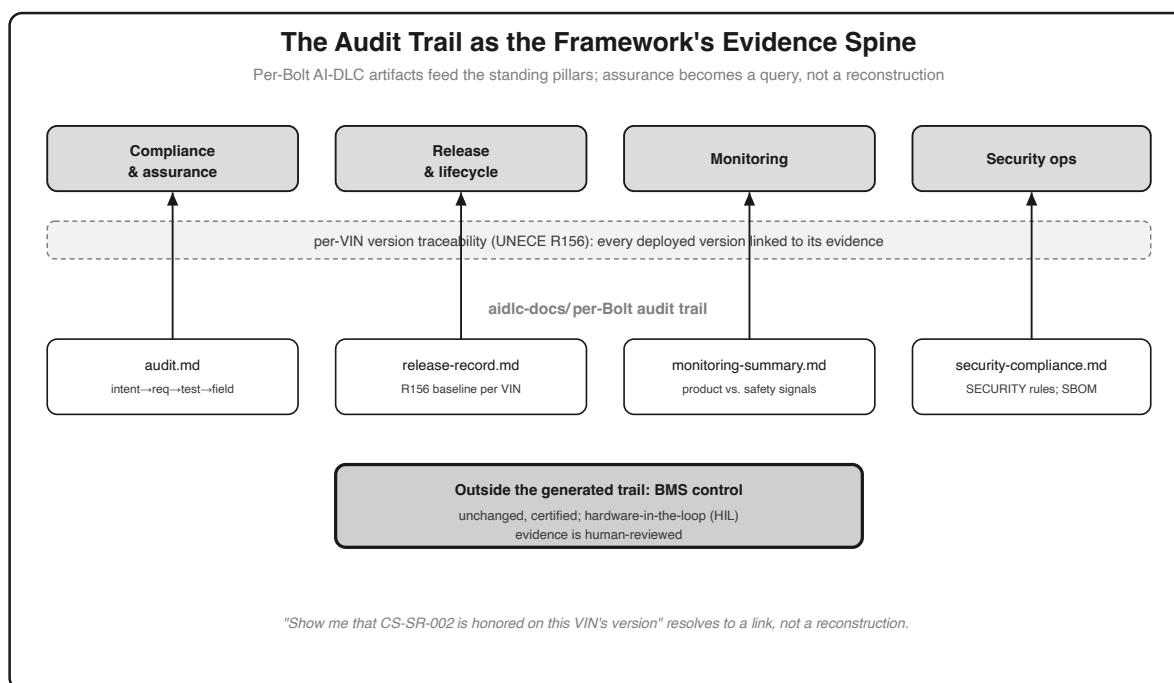


Figure 18: An evidence-spine diagram. Along the base sit the per-Bolt AI-DLC artifacts (audit.md, release-record.md, monitoring-summary.md, security-compliance.md), each drawn as a document node. Arrows rise from each artifact to the framework pillar it feeds above it: audit.md to Compliance and assurance, release-record.md to Release and lifecycle, monitoring-summary.md to Monitoring, and security-compliance.md to Security operations. A horizontal band labeled per-VIN version traceability (UNECE R156) links the pillars. A separate boxed note on the right, drawn with a thick dark border, states that the BMS control code is unchanged and its hardware-in-the-loop evidence is human-reviewed, sitting outside the generated trail. A footer notes that assurance becomes a query against the trail rather than a reconstruction.

15.5 Human gates, roles, and the safety boundary in operations

The framework's accountable roles map directly onto AI-DLC's human decision points, and this is what makes an AI-led workflow defensible in operations. The Quality Steward owns the release gate that authorizes a campaign; the functional-safety owner owns any safety-relevant signal or change; the VSOC owns security events and the response to them. AI-DLC does not invent these roles; it gives each one a concrete, recorded gate to stand at, so "who approved this deployment" always has a named answer in the trail.

The safety boundary the book has drawn since Chapter 9 holds with full force in operations, and operations is where it is most tempting to breach. The framework's safety-critical elements, the BMS thermal and cell-protection control and its authoritative inhibit, stay human-engineered, and generative AI never enters the control path, even under the time pressure of a live incident. When a field problem appears, the response runs through the framework's disciplined options

(halt a staged rollout, roll back to a retained known-good version, or ship a gated fix-forward), never through an ungated emergency patch and never by putting a generator on the safety loop.

Where AI does have a place in operational decisions, it proposes rather than decides. A rollout strategy or an incident response is presented as an editable, multiple-choice questionnaire; the recorded human choice, not the generated suggestion, is authoritative. The framework's roles supply exactly the human judgment those gates require.

15.6 Continuous improvement: closing the loop to the next Bolt

The framework is a loop, not a pipeline. Field learnings from the monitoring pillar do not end as a dashboard; they re-enter Inception as fresh intent, and the next Bolt carries them back through Construction and Operations. The cold-weather charge-scheduling anomaly of Section 15.3 is the pattern in miniature: observed in the field, turned into intent, and delivered as the next improvement, with the safety boundary untouched throughout.

Two forces keep the loop healthy as it scales. Cost discipline is one: because each turn of the loop spends inference and cloud budget, the measure, cap, and alert practice from Chapter 4 rides along, attributing spend per Bolt and per layer so a busy improvement cycle does not quietly become an expensive one. Review capacity is the other: the loop can only turn as fast as the framework can validate what it produces, which is why the validation pillar, not the generator, is the constraint to invest in. Run with that discipline and the SDV operational framework becomes what traditional post-production support never was: an engine that makes the fleet measurably better with every turn, quickly, traceably, and safely.

Key takeaways

- The SDV operational framework is the standing operating model (roles, processes, tooling) that runs the whole fleet continuously; it is distinct from the Operations phase of a single Bolt, which handles one change.
- AI-DLC embeds into the framework rather than sitting beside it: the Bolt is the unit of change that threads every pillar, the human gates are the framework's control points, and the audit trail is its shared evidence spine.
- Continuous assurance becomes a query rather than a reconstruction because the documentation-first trail supplies per-version evidence for ISO 26262, ISO 21434, and UNECE R155/R156.
- Cost control (measure, cap, alert) and security operations embed as blocking gate conditions, and the BMS safety boundary stays human-engineered throughout, even under incident time pressure.
- The framework is a loop: field learnings become the next Bolt's intent, so the fleet keeps improving safely, bounded only by validation capacity.

What's next

Chapter 16 turns to a practical cross-cutting decision the framework depends on: how to select and evaluate the large language models that power AI-DLC for each kind of workload.

Chapter 16: Selecting and Evaluating LLM Models for SDV Workloads

After this chapter you can match a large language model (LLM) to the kind of AI-DLC workload it will run, weigh the selection criteria that matter in an automotive setting (quality, context, latency, cost, privacy, and safety-relevance), and stand up an evaluation harness that keeps the choice honest as models and tasks change.

The wrong default

AI-DLC is model-agnostic by design, but a program still has to choose which models actually run it, and “use the biggest model for everything” is the wrong answer on both cost and fit. A frontier model asked to scaffold a REST (representational state transfer) endpoint burns budget it did not need to; a small model asked to reason about a timing-critical design quietly produces plausible, wrong output. The work of an SDV program spans a wide range, from boilerplate and tests to architecture reasoning to reverse-engineering a legacy estate, and those workloads have genuinely different model needs.

This chapter treats model selection as a first-class engineering decision, not a default. It gives a workload taxonomy, the criteria that decide a fit, an evaluation harness that makes the decision evidence-based, and the one rule that overrides all of them: no choice of model moves the safety boundary.

16.1 Why model selection is a first-class decision

Chapter 4 established two ideas this chapter builds on. The first is the maturity gradient: AI-DLC’s effectiveness varies by workload, high on cloud and cross-cutting work, lower toward safety-critical layers. The second is cost discipline: measure, cap, and alert, and right-size the model to the task. Model selection is where those two ideas become a concrete, recurring choice. Every workload sits somewhere on the gradient and carries a cost, and the model is the lever that sets both the quality ceiling and the price.

Getting it wrong is expensive in two directions. Over-provisioning (a frontier model for routine generation) inflates the inference bill that the cost pillar of Chapter 15 has to carry, with no quality gain the reviewer can see. Under-provisioning (a weak model for reasoning-heavy work) is worse, because the failure is silent: the output looks fine and fails in review or, missed there, in the field. The goal is fit, the smallest, cheapest model that clears the quality bar for that specific workload, chosen on evidence rather than reputation.

16.2 A workload taxonomy for AI-DLC

The first step is to stop treating “AI-DLC work” as one thing. The workloads it runs have distinct dominant demands, and naming them makes the model fit obvious.

Workload	What the model must do well	Dominant selection criteria
Requirements elaboration (Inception)	Reason over intent, surface ambiguity, ask good questions	Reasoning quality, instruction following
Cloud / QM code generation	Produce correct, idiomatic code and infrastructure-as-code	Coding quality, tool/function calling
Test generation	Cover requirements and edge cases; name the IDs	Coding quality, reliability (mid-tier is often enough)
Documentation & summaries	Faithful, concise prose from structured artifacts	Faithfulness, low hallucination (small model often suffices)
Reverse engineering (brownfield)	Hold a large legacy codebase in view at once	Long context window, retrieval grounding
Classification & routing	Triage telemetry, tag anomalies, route commands	Small, fast, cheap; determinism
Retrieval & similarity	Find related requirements, code, prior decisions	Embedding quality, not a chat model

The point of the table is that the right model is workload-specific. Reverse engineering the legacy telematics estate of Chapter 14 needs a long-context model above all; generating the `schedule-api` unit tests (US-03) needs solid coding ability but rarely a frontier tier; triaging fleet telemetry for the monitoring pillar needs a small, cheap, fast model run at volume. The figure below places the workloads on the maturity gradient and shows the model tier each tends to call for.

16.3 The selection criteria

Once the workload is named, a consistent set of criteria decides the fit. In an SDV program several of these carry more weight than they would in general software.

Criterion	What to ask	Why it matters in an SDV
Capability / quality	Does it clear the task’s quality bar on our own evals?	Under-provisioning fails silently in review or the field
Context window	Can it hold the artifacts or codebase the task needs?	Brownfield reverse engineering is context-bound (Chapter 14)
Latency & throughput	Fast enough for interactive Mob work or high-volume triage?	Interactive elaboration vs. fleet-scale classification differ
Cost per token	What does a representative task cost?	Feeds measure/cap/alert and per-layer attribution (Chapter 4)
Privacy & data residency	Can prompts contain fleet or personal data? Self-host or API?	Data-privacy pillar; keep sensitive data out of third-party context
IP & licensing	Are provenance and license terms acceptable for shipped code?	Supply-chain and IP posture (Chapter 13)
Tool use / function calling	Does it drive the toolchain reliably?	Construction and Operations depend on structured tool calls
Reproducibility & provenance	Can we pin a version and record which model produced what?	Assurance and audit (the trail must name the model)
Safety-relevance	Is this workload on or near the control path?	If yes, the model does not decide; a human does (16.5)

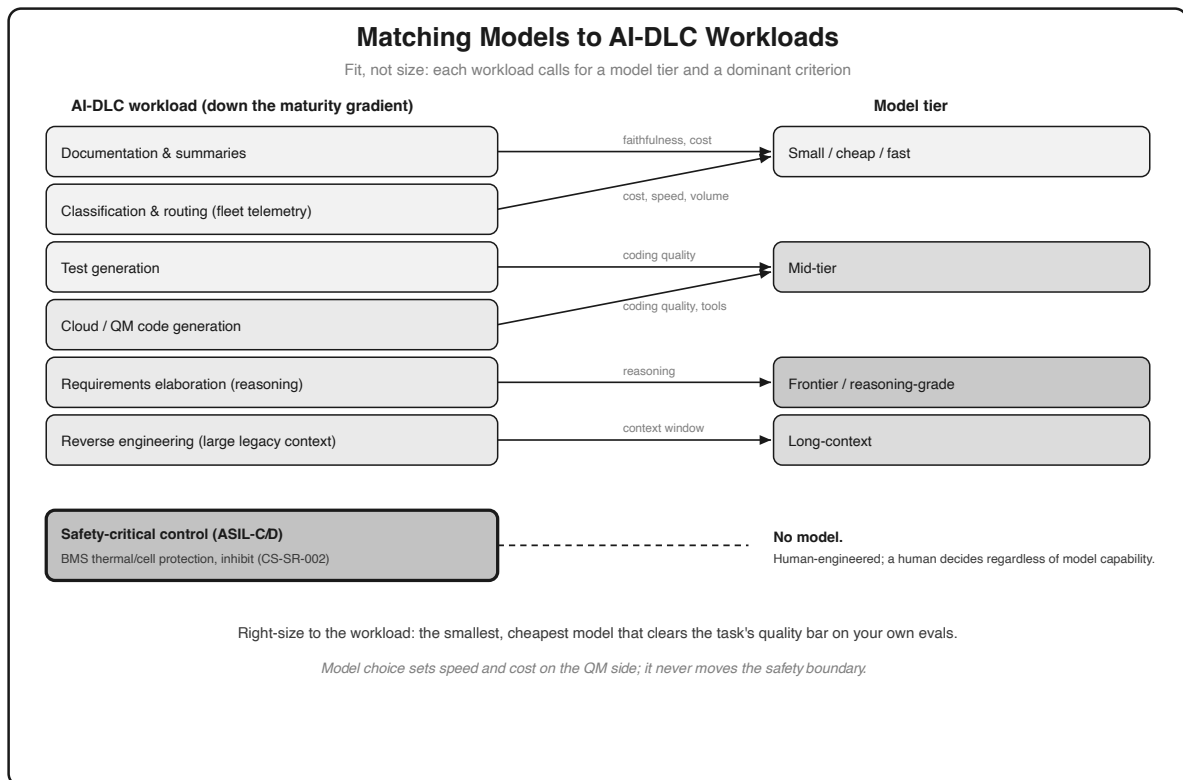


Figure 19: A matching diagram. The left column lists AI-DLC workloads ordered down the maturity gradient (documentation and summaries, classification and routing, test generation, cloud/QM code generation, requirements elaboration, reverse engineering with large context, and safety-critical control). The right column lists model tiers (small/cheap, mid-tier, frontier/reasoning, long-context, embeddings). Arrows connect each workload to its typical model tier, with the dominant criterion labeled on the arrow (cost, coding quality, reasoning, context window). The bottom row, safety-critical control, is drawn in a dark, thick-bordered box connected to no model tier, with a note that no model authors the ASIL-C/D control path regardless of capability. A footer states: fit, not size, is the goal.

Two criteria deserve emphasis because they are automotive-specific. Privacy and data residency can force a self-hosted or smaller on-premises model where a workload must process fleet telemetry or personal data, rather than sending it to a third-party endpoint. And reproducibility matters more than in typical software: the audit trail should record which model and version generated an artifact, so that assurance can answer “what produced this” as precisely as “who approved it.”

16.4 Building an evaluation harness

Reputation and vendor benchmarks do not tell you how a model performs on your workloads. An evaluation harness does. The idea is simple: assemble task-representative evaluation sets with known-good outcomes, run candidate models against them, score automatically where possible and with human review where it matters, and record the result as an artifact before adopting the model.

What to measure depends on the workload. For code generation, functional correctness against generated and hand-written tests (for example a pass rate across a suite of representative tasks) is the primary signal. For requirements elaboration, requirement-coverage and the rate of useful clarifying questions. For documentation, a faithfulness or hallucination check against the source artifacts. Across all of them, cost per task and latency are recorded alongside quality, because a model that is marginally better but far more expensive rarely wins. Safety-relevant and security-sensitive workloads add red-team and adversarial cases (for example prompt-injection resistance for anything that ingests fleet data, per Chapter 13).

The harness is not a one-time gate. Models change under you: providers update versions, and your own tasks drift. So the eval set is versioned and re-run as a regression suite whenever a model or version changes, and the results live in the trail like any other evidence. The figure shows the loop.

16.5 Matching models to the maturity gradient and the safety boundary

Putting the taxonomy and criteria together yields a simple operating pattern that mirrors the maturity gradient. Small, cheap models handle boilerplate, tests, documentation, and high-volume classification. Mid-tier models handle most cloud and QM code generation. Frontier or reasoning-grade models are reserved for genuinely hard reasoning: complex architecture, tricky requirements, thorny debugging. Long-context models are brought in specifically for brownfield reverse engineering. This is right-sizing from Chapter 4, made concrete per workload.

One rule overrides the entire chapter, and it is the same boundary the book has held throughout: model selection never changes the safety boundary. There is no model, however capable, that is allowed to author the safety-critical control path. The Battery Management System (BMS) thermal and cell-protection control and its authoritative inhibit (CS-SR-002) stay human-engineered regardless of which model powers the rest of the feature, and a more capable model does not earn a lighter review. Model choice affects speed and cost on the QM side of the line; it does not move the line, and it never substitutes for the human gate.

16.6 Governing model choice

Because models are now part of how the product is built, the choice is governed like any other consequential decision. Three practices keep it honest. First, a **model registry**: which models

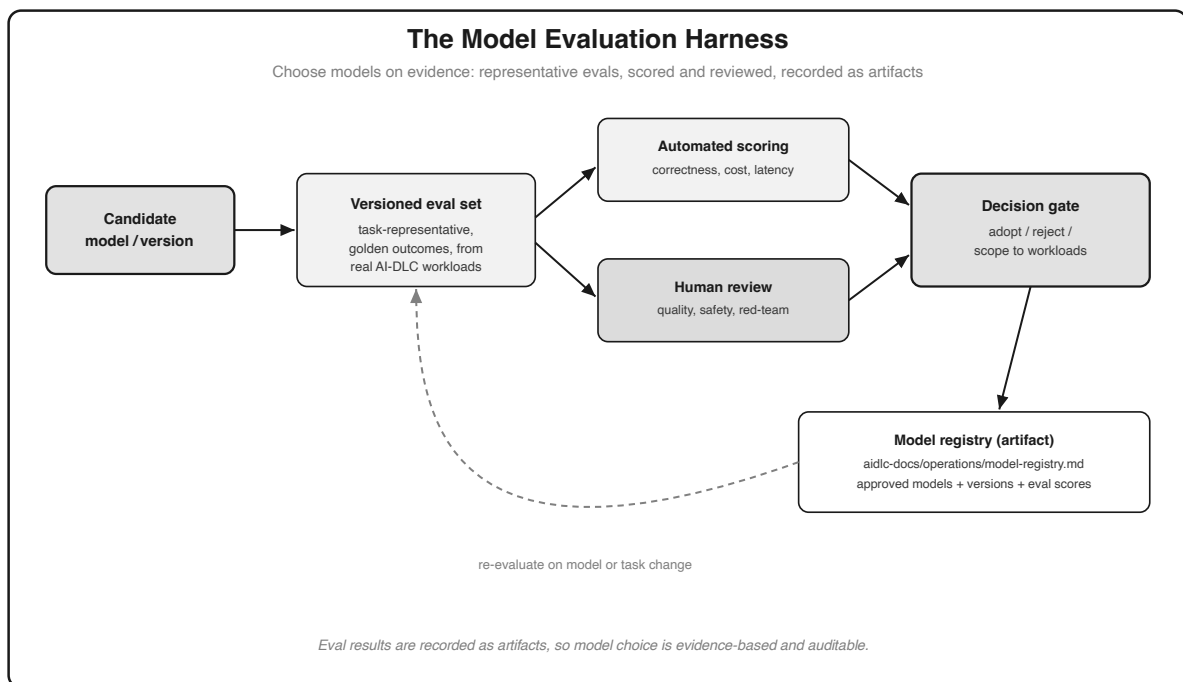


Figure 20: An evaluation-harness loop. A candidate model enters at the left. It runs against a versioned, task-representative evaluation set (with golden outcomes) drawn from real AI-DLC workloads. Outputs go through two scoring paths that merge: automated scoring (correctness, cost, latency) and human review (quality, safety-relevance, red-team cases). The merged result reaches a decision gate: adopt, reject, or scope to specific workloads. Adopted models and their scores are written to a model registry artifact under aidlc-docs. A dashed feedback arrow labeled re-evaluate on model or task change loops from the registry back to the evaluation set. A footer notes that eval results are recorded as artifacts, so model choice is evidence-based and auditable.

and versions are approved, for which workloads, with their evaluation scores attached, so a team cannot quietly adopt an unevaluated model. Second, a **re-evaluation cadence**: models and versions are re-run against the eval suite on a regular schedule and whenever a provider updates, because a version bump is a change that needs evidence, not a silent swap. Third, **cost attribution per model**: spend is tracked per model and per workload and folded into the measure, cap, and alert practice of Chapter 4 and the cost pillar of Chapter 15, so the economics of a model choice stay visible.

Treated this way, the model becomes a governed, evaluated, and attributable input to AI-DLC rather than an invisible dependency. The registry entry and its eval scores join the audit trail, so the same question the assurance pillar asks of code, “show me the evidence,” has an answer for the models too.

Key takeaways

- Model selection is a first-class, recurring decision: match the model to the workload rather than defaulting to the largest, because over-provisioning wastes budget and under-provisioning fails silently.
- Name the workload first (elaboration, code generation, test generation, documentation, reverse engineering, classification, retrieval); each has a dominant demand such as reasoning, coding quality, context window, or cost.
- Decide on criteria that carry extra weight in an SDV: privacy and data residency, IP and licensing, reproducibility and provenance, and safety-relevance.
- Choose on evidence: run candidate models against versioned, task-representative evaluation sets, score automatically and with human review, and record the result as an artifact; re-evaluate when models or tasks change.
- No model moves the safety boundary: generative AI never authors the ASIL-C/D control path, and a more capable model never earns a lighter human review.

What’s next

Chapter 17 ties the whole methodology together with an end-to-end worked example and an adoption roadmap for rolling AI-DLC out across an SDV organization.

Chapter 17: A Worked Example and an Adoption Roadmap

After this chapter you have an end-to-end picture of delivering an SDV feature as a single AI-DLC Bolt, plus a practical plan for rolling AI-DLC out across an automotive software organization.

17.1 End-to-end Bolt: building an SDV feature with AI-DLC

We have followed one feature, EV charge scheduling, through every phase of this book. This section collects that journey into a single picture: one Bolt carrying the feature from intent to field, accumulating artifacts and passing human gates along the way. The full artifact trail and source for this journey live in the book’s companion repository (see the Preface).

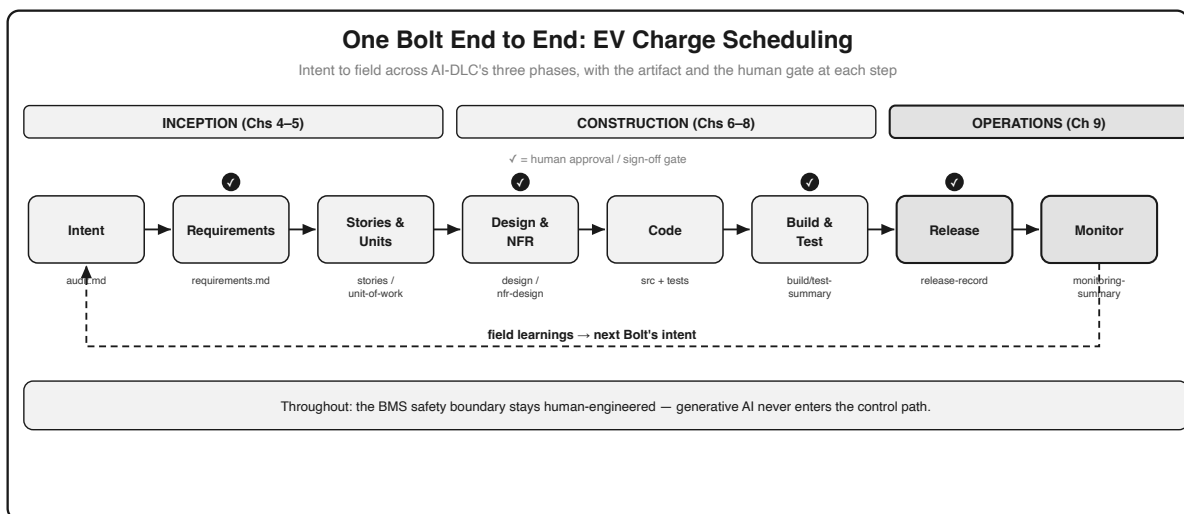


Figure 21: A left-to-right timeline of one Bolt across three phase bands. Inception (Chapters 5–6): Intent (audit.md), Requirements with an approval gate (requirements.md), Stories and Units (stories and unit-of-work). Construction (Chapters 7–9): Design and NFR with an approval gate (design and nfr-design), Code (src plus tests), Build and Test with an approval gate (build/test summary). Operations (Chapter 10), shown emphasized: Release with a sign-off gate (release-record) and Monitor (monitoring summary). Flow arrows connect the steps, and a dashed feedback loop runs from Monitor back to Intent labeled field learnings to next Bolt’s intent. A footer states that throughout, the BMS safety boundary stays human-engineered and generative AI never enters the control path.

The narrative is now familiar, and that familiarity is the point; the same shape repeats for any SDV feature:

- **Inception (Chapters 5–6).** A one-line intent became a captured intent record, then structured requirements (CS-FR/NFR/SR), user stories, and a unit decomposition that isolated the BMS adapter as a safety boundary. The team approved at the requirements gate.
- **Construction (Chapters 7–9).** Architecture split the feature across cloud, edge, and vehicle; NFR design turned safety, timing, reliability, and security into concrete components; code was generated per layer and the safety-relevant adapter was human-written; the build-and-test gate passed, including the hardware-in-the-loop BMS check.
- **Operations (Chapter 10).** A human-gated, staged OTA rollout deployed the feature; monitoring surfaced a cold-weather anomaly that became the next Bolt’s intent, closing the loop.

At every step the artifact and the human gate are the load-bearing elements, not the generation itself. And throughout, the safety boundary held: generative AI accelerated the orchestration and never touched the control path.

17.2 Lessons, anti-patterns, and a maturity model

The worked example teaches a few durable lessons, and warns of a few traps.

What works: - Start where verification is automated and stakes are bounded (cloud, tests, docs), then expand inward. - Make the safety boundary structural and small, so the high-rigor review is contained. - Treat artifacts and gates as the product; let traceability accumulate rather than reconstructing it.

Anti-patterns to avoid: - Letting generated volume outrun review capacity, the productivity paradox of Chapter 4; generating faster than you can validate helps no one. - Treating an audit trail as proof of correctness rather than process. - Pasting real fleet or in-cabin data into AI context for convenience. - Pointing AI at ASIL-C/D control code because it “worked” on the cloud side.

These cohere into a staged maturity model: organizations deepen adoption by layer, and the safety boundary does not move as they climb.

Most automotive organizations should expect to live at L2–L3 for cloud and middleware work while holding safety-critical control firmly outside the generator, and to revisit that line only as tooling and regulation mature.

17.3 Rolling AI-DLC out across an SDV organization

Rolling out AI-DLC is an organizational change, not just a tool install. A practical sequence:

- **Pick pilots with the VRF framework (Chapter 4).** Choose Accelerate-band workloads (cloud fleet APIs, data pipelines, test generation) where value is high, risk is low, and feasibility is real. Avoid starting with a Defer-band workload to “prove” the methodology.
- **Form cross-functional mobs, not an AI team.** The mob that elaborates and constructs must include software, systems, safety, security, test, and supplier voices, exactly as in Chapters 5–7. AI-DLC fails when treated as a side activity owned by a few enthusiasts.
- **Stand up the gates and roles.** Map the human decision points to accountable roles and a review cadence (Chapter 13’s governance table; Chapter 4’s quarterly re-scoring). Enable the Security Baseline extension for production-grade work.

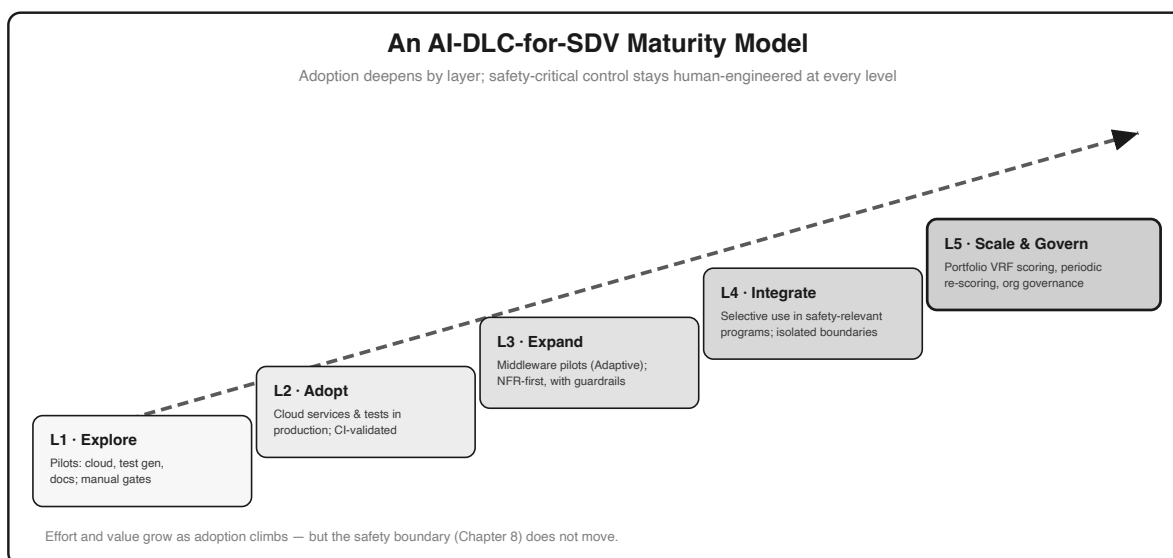


Figure 22: A five-step ascending staircase, each step darker than the last, with a dashed upward guide arrow. L1 Explore: pilots in cloud, test generation, and docs with manual gates. L2 Adopt: cloud services and tests in production, CI-validated. L3 Expand: middleware pilots on AUTOSAR Adaptive, NFR-first with guardrails. L4 Integrate: selective use in safety-relevant programs with isolated boundaries. L5 Scale and Govern: portfolio VRF scoring, periodic re-scoring, and organizational governance. A note states that effort and value grow as adoption climbs, but the safety boundary from Chapter 9 does not move.

- **Align suppliers.** Much SDV code crosses OEM/Tier-1 boundaries; provenance, IP, and interface contracts (Chapter 13) need agreement before generated code flows across organizations.
- **Invest in the validation bottleneck.** Because AI shifts the constraint from writing to validating, scale review, security scanning, and HIL capacity in step with generation, or the gains stall.

The goal is not maximal AI usage; it is maximal *value* at acceptable risk, which the framework is designed to find.

17.4 The road ahead: agents, smaller models, and evolving regulation

Three currents will shape what comes next. **Agents** will take on more of the orchestration, running more of the loop autonomously between gates, which makes the gates, not the generation, the thing to invest in, and raises the stakes on cost control (Chapter 4): the more autonomously an agent loops, the more a runaway can spend before a human notices. **Smaller, specialized models** may bring capable generation closer to proprietary toolchains and even on-premise, easing the IP and provenance concerns that keep some automotive work off shared services today. And **regulation will evolve**: the open question of how AI-generated code is treated in certification (Chapter 12) is under active discussion in UNECE working groups, and clarity there will move the safety boundary, carefully, and later than the cloud-side gains.

The durable advice outlasts any tool version. Elaborate intent well. Put human decisions where safety, timing, and regulation are at stake. Keep generative AI off the control path. Let traceability accumulate as the product. Re-score your boundaries as the technology and the

rules change. Do those, and AI-DLC is not a risk to manage but a way to build software-defined vehicles faster without giving up the rigor the domain demands.

17.5 A checklist for your first Bolt

The roadmap in Section 17.3 is organizational; this is the team-level companion, a concrete checklist for the first feature you carry through AI-DLC. It distills the whole book into steps you can act on, and it deliberately favors a small, bounded feature over an ambitious one.

Before you start:

- **Choose an Accelerate-band feature.** Cloud-side or cross-cutting, low safety exposure, a clear existing spec. Save the hard cases for after the team has run the loop once.
- **Assemble the mob.** Software, systems, test, and, even for a cloud feature, a security voice; add safety and supplier voices the moment the vehicle side is involved.
- **Decide the boundary up front.** Name anything safety-relevant the feature touches and plan to isolate it behind a thin, human-owned contract before any code is generated.

As you run the Bolt:

- **Write the intent as one honest sentence, then elaborate it.** Answer the questionnaires with real attention, and add the questions the model missed.
- **Approve at each gate deliberately.** Treat requirements, design, and build-and-test as real decisions, not rubber stamps; send work back when it is not right.
- **Let the artifacts accumulate.** Do not hand-maintain a separate traceability document. Check that requirement IDs carry into code and tests, and that the audit trail records who approved what.
- **Keep generation off the control path.** If the feature has an ASIL-C/D element, that code is human-engineered and reviewed on hardware, full stop.

After it ships:

- **Measure the right things.** Capture the DORA metrics per layer and the guardrail metrics (Chapter 11), and expect an early adoption dip before the steady-state gain.
- **Close the loop.** Feed the first field learnings back as the next Bolt's intent, and use what you learned about your own review capacity to plan the second feature.

Run that once on a forgiving feature and the team will have exercised every gate, artifact, and boundary the book describes, on a change where a mistake is cheap. The second Bolt is where the method starts paying off; the first is where the team earns the confidence to trust it.

Key takeaways

- One Bolt carried EV charge scheduling from intent to field; the artifacts and human gates, not the generation, are what made it trustworthy.
- Start where verification is automated and stakes are bounded, isolate the safety boundary, and treat traceability as the product.
- A staged maturity model (Explore → Adopt → Expand → Integrate → Scale & Govern) deepens adoption by layer while the safety boundary stays fixed.
- Roll out with VRF-chosen pilots, cross-functional mobs, real gates and roles, supplier alignment, and investment in the validation bottleneck.
- Agents, smaller models, and evolving regulation will shift what is feasible, but the durable practice is to elaborate intent, gate human decisions, keep AI off the control path, and re-score boundaries over time.

- Run a first Bolt on a small, forgiving Accelerate-band feature to exercise every gate, artifact, and boundary where a mistake is cheap; the second Bolt is where the method starts paying off.

Glossary

AI-DLC and SDV concepts

Core terms used throughout the book that are not abbreviations.

- **Bolt:** AI-DLC's unit of delivery, replacing the sprint; a focused pass through the phases for a coherent piece of work.
- **Inception:** AI-DLC phase ("What & Why") where AI generates requirements and stories; the team approves.
- **Construction:** AI-DLC phase ("How") where AI proposes architecture and writes code; the team decides.
- **Operations:** AI-DLC phase where AI deploys and documents from accumulated context; the team signs off.
- **Mob Elaboration:** AI-DLC practice: the team works with AI during Inception to elaborate intent into requirements.
- **Mob Construction:** AI-DLC practice: the team works with AI during Construction to steer architecture and code.
- **Zonal architecture:** E/E architecture organizing controllers by physical zone of the vehicle.

Abbreviations and acronyms

Term	Meaning
ADAS	Advanced Driver-Assistance Systems.
ADR	Architecture Decision Record; captures an architecture option, the decision, and its rationale.
AI	Artificial Intelligence.
AI-DLC	AI-Driven Development Life Cycle; an AWS methodology positioning AI as development orchestrator across Inception, Construction, and Operations.
API	Application Programming Interface.
ARXML	AUTOSAR XML; the description/interface format used by AUTOSAR tooling.
ASIC	Application-Specific Integrated Circuit.
ASIL	Automotive Safety Integrity Level (from ISO 26262).
ASPICE	Automotive SPICE; a process-maturity model for automotive software (work product traceability).
AUTOSAR	Standardized automotive software architecture; Classic (real-time) and Adaptive (high-performance, POSIX-based) platforms.
AWS	Amazon Web Services.
BMS	Battery Management System.

Term	Meaning
BSW	Basic Software; the lower layers of the AUTOSAR Classic platform.
CAN	Controller Area Network; an in-vehicle communication bus.
CI/CD	Continuous Integration / Continuous Delivery.
CSMS	Cyber Security Management System (UNECE R155).
DDS	Data Distribution Service; a service-oriented publish/subscribe middleware.
DORA	DevOps Research and Assessment; its four delivery-performance metrics (deployment frequency, lead time for changes, change failure rate, failed-deployment recovery time).
E/E	Electrical/Electronic (vehicle architecture).
ECU	Electronic Control Unit.
EV	Electric Vehicle.
FMVSS	Federal Motor Vehicle Safety Standards (United States).
GB	Guobiao; China's national standards.
GPU	Graphics Processing Unit.
GSN/CAE	Goal Structuring Notation / Claims-Arguments-Evidence; structured safety-argument notations.
HAL	Hardware Abstraction Layer.
HARA	Hazard Analysis and Risk Assessment.
HIL	Hardware-in-the-Loop (testing).
HMI	Human-Machine Interface.
HVAC	Heating, Ventilation, and Air Conditioning.
IaC	Infrastructure as Code.
IAM	Identity and Access Management.
IDL	Interface Definition Language.
IP	Intellectual Property.
ISO 21434	Road-vehicle cybersecurity engineering standard.
ISO 21448 (SOTIF)	Safety of the Intended Functionality.
ISO 26262	Functional safety standard for road vehicles.
ITAR	International Traffic in Arms Regulations (United States).
JV	Joint Venture.
KPI	Key Performance Indicator.
LiDAR	Light Detection and Ranging.
MCU	Microcontroller Unit.
MISRA	Motor Industry Software Reliability Association; C/C++ coding guidelines for safety.
ML	Machine Learning.
MQTT	Message Queuing Telemetry Transport; a lightweight telemetry/messaging protocol.
mTLS	Mutual Transport Layer Security.
NFR	Non-Functional Requirement (e.g., safety, timing, security, reliability).
OEM	Original Equipment Manufacturer.
OTA	Over-the-Air (software update).
PII	Personally Identifiable Information.
POSIX	Portable Operating System Interface.
PR	Pull Request.
QM	Quality Management; the ISO 26262 class for items with no safety relevance.

Term	Meaning
QoS	Quality of Service.
R155	UN Regulation No. 155; cyber security management (CSMS).
R156	UN Regulation No. 156; software update management (SUMS).
REST	Representational State Transfer.
RSS	Responsibility-Sensitive Safety; a formal motion-safety model.
SBOM	Software Bill of Materials.
SDLC	Software Development Life Cycle.
SDV	Software-Defined Vehicle.
SFF	Safety Force Field; a formal motion-safety model.
SIL	Software-in-the-Loop (testing).
SOA	Service-Oriented Architecture.
SoC	State of Charge (battery); also System-on-Chip, depending on context.
SOME/IP	Scalable service-Oriented Middleware over IP.
SOP	Start of Production.
SUMS	Software Update Management System (UNECE R156).
TARA	Threat Analysis and Risk Assessment.
TLS	Transport Layer Security (mTLS = mutual TLS).
UI	User Interface.
UNECE	United Nations Economic Commission for Europe.
VIN	Vehicle Identification Number.
VRF	Value, Risk, Feasibility; the AI-DLC adoption framework (Chapter 4).
WCET	Worst-Case Execution Time.
WP.29	World Forum for Harmonization of Vehicle Regulations (UNECE).